

ISEN ALL IS DIGITAL! OUEST | yncréa

Hardware Security

Hardware security - Cortex M

jean-jacques.meneu@isen-ouest.yncrea.fr

Agenda

1. Context: new cyber regulation
2. Memory Technologies in microcontrollers: Flash and SRAM
3. Memory organization, Microcontroller architecture, Assembly Language
4. Compilation Stages: Understanding of linker file
5. C programming and impact on variable location in memory
6. Microcontroller protections: Write Protection, Read Protection (PCROP, RDP), Firewall, TrustZone….



1: Hardware Security

Context: new cyber regulation

CRA

- CRA stands for Cyber Resilience Act
- It is a European cyber security law aiming at increasing security of connected digital elements.
- To sell a product in Europe en 2027, any device must comply with this law
- Apply to all products with a connected digital element

Enisa Requirements-Standard Mapping

Two sets of requirements

Product cyber security requirements

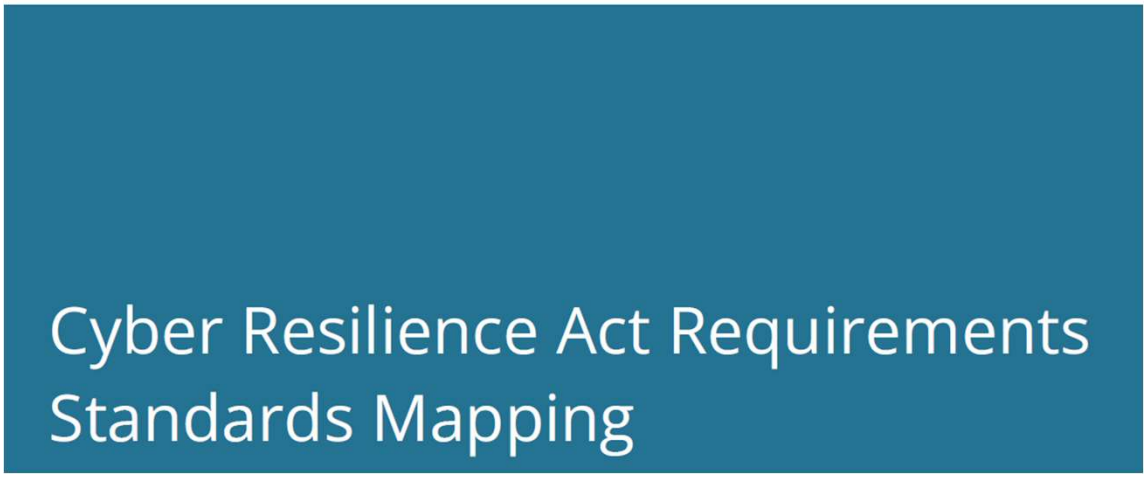
Annex I, section 1 of the CRA Proposal

Vulnerability handling Process requirements

Annex I, section 2 of the CRA Proposal

Enisa

1. Enisa link the CRA law voted by the UE parlement into technical requirement



https://www.enisa.europa.eu/publications/cyber-resilience-act-requirements-standards-mapping

ENISA

- ENISA (European Union Agency for Cybersecurity) est l'organe central qui définit les règles du jeu
- Rôle: conseille les états members, rédige des rapports sur les menaces et surtout, ellest chargée de créer les schémas de certification de cybersécurité
- C'est l'ENISA qui définit les standards que le produit à base de STM32 doit respecter pour être conforme au CRA. Si le CRA dit "il faut un haut niveau de sécurité", c'est l'ENISA qui explique techniquement ce que cela signifie (par exemple, en s'appuyant sur des standards comme PSA certified)

CRA TimeLine

Placing on the market date (first time sales)	Making available on the market date (each time import to EU or produce in EU)	Decision
Before December 2027 (no more sales on EU market)		CRA does not apply
Before December 2027 but modified (materially changed*)		CRA may apply
Before Dec'2027	**After** December 2027	CRA applies
After December 2027		CRA applies

Legacy products are also impacted

Note: *Material changes include **changes in intended use** or **functional enhancements**, such as software updates.
Spare parts are out of CRA scope

The CRA's timeline includes vulnerability and incident reporting starting in December 2026, with full compliance required by December 2027. Products must be compliant if produced within Europe or imported to Europe after the application date. This means that products developed before the law are concerned.

CRA Product Categories

1. Default class: All products not listed as security important nor critical
2. Class I : important products with security related functionalities
3. Class II : important products with security related functionalited and tamper resistance requirements
4. Critical class: Critical products

Categories are defined are defined and a tentative list of products or applications is provided with Annex II of the law. If products are not listed within Annex III, they are considered to belong to the default class

The law can be downloaded on this website
https://eur-lex.europa.eu/eli/reg/2024/2847/oj/eng

Class I

- Identity management systems and privileged access management software and hardware, including authentication and access control control readers, including biometric readers
- Standalone and embedded browsers
- Password managers
- Software that searches for, removes, or qurantines malicious software
- Products with digital elements with the function of VPN
- Nerwork management systems
- Security information and event management systems
- Boot managers
- Public key infrastructure and digital certificate issuance software

Class I

- Physical and virtual network interfaces
- Operating systems
- Routers, modems intended for the connection to the internet and switches
- Microprocessors and microcontrollers with security-related functionalities
- Application Specific Integrated Circuits (ASICS) and Field Programmable Gate Arrays (FPGA) with security related functionalities
- Smart home general purpose virtual assistants
- Smart home products with security functionalities, including smart door locks, security cameras, baby monitoring systems and alarm systems

Class I

- Internet connected toys covered by directive 2009/48/IEC of the European Parliament and of the council that have social interactive features (e.g. speaking or filming) or that have location tracking features
- Personal wearable products to be worn or placed on a human body that have a health monitoring (such as tracking) purpose and to which Regulation EU 2017/745 or EU No EU 2017/746 do not apply; or personal wearable products that are intended for the use by and for children

Class II

- Hypervisors and container runtime systems that support virtualized execution of operating systems and similar environments
- Firewalls, intrusion detection and prevention systems
- Tamper-resistant microprocessors and microcontrollers

CRA classification selon ST

For MCUs & MPUs we believe that products without security features will be classified in the default class; products with security features, such as cryptographic accelerators, random generators, isolation mechanisms, will be in Class I, while products with such security features and claiming physical attack resistance will belong to Class II.

Requirements Categories

The general principle is that for the products on the market, a self-assessment of compliance with the requirements specified in Annex I will be sufficient. For certain categories of more critical products, the application of harmonised standards will be required. For even more critical products, a third-party assessment will be mandatory

CRA Compliance (1/3)

- Risk Assessment: a deep risk analysis, to be done starting from the product definition, to analyze potential risks and propose mitigations, updated during the device lifecycle
- Secure by design: start implementing products security in mind, instead of just adding security as an on-demand feature
- No Known exploitable vulnerabilities: be able to track impacting vulnerabilities and mitigate them to a low-level and acceptable risk, within all the supply chain, including software and hardware

CRA Compliance (2/3)

- Regular security updates: the capatibility to mitigate the severity of vulnerabilities by providing patches and/or updates is expected in most cases
- Resistance to denial-of-service (DoS) attack: the capability to keep services running during DOS disruptions, by limiting the effects of disturbances
- Minimize negative impact on the availability of services provided by others
- Device Protection (authentication, confidentiality, integrity): ensure that devices are genuine and avoid firmware exploits and malware injection

CRA Compliance (3/3)

- Software Bill Of Materials (SBOM): provide an exhaustive list of software components used to build the products. The list shall include information such ownership, license, version and copyrights
- Vulnerabilities handling, monitoring, disclosure: vulnerabilities along the complete supply chain, including hardware and software, including open-source components, to be mitigated. Regular scans must be done to ensure that the product is safe over the product the product lifetime. In case of new vulnerabilities, the impact shall be assessed and mitigated. A robust security mitigation model is vital for identifying assets, threats and common vulnerabilities, and for defining a protection strategy based on countermeasures. Severe incidents must be reported to ENISA and CSIRT with an early warning due in 24 hours and a notification in 72 hours

Security Standards Around the World

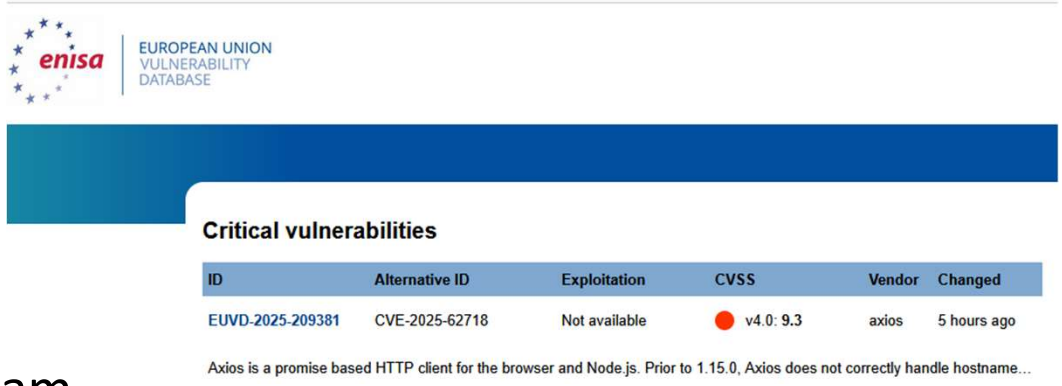
1. RED Directive: RED establishes a regulatory framework for placing radio equipment on the market to meet cybersecurity requirements for internet-connected devices, with compliance via self-assessment or notified bodies.
2. Cyber Resilience Act : The CRA is a European cyber security law aiming at increasing security of connected digital elements.
3. Cyber Trust Act: The cybersecurity act strengthens the EU cybersecurity certification framework, enhancing trust in digital products and services.
4. NIST Cybersecurity Framework: A voluntary framework developed by the U.S. national institute of standards and technology to provide guidance for organizations to manage and reduce cybersecurity risk.
5. China Cybersecurity Law: The China cybersecurity law regulates network security and data protection requirements in China, effective since June 2017

SBOM from ST

- Yes, STM32Cube packages will be delivered with SBOM, in CycloneDx format. The process of exposing the SBOM publicly on the STM32Cube packages has been started and over time be extended to most of them if they are updated.
- Other formats such us SPDX are not provided by ST but multiple commercial and open-source converters are publicly available.
- The SBOMs will be delivered within the STM32Cube packages, either on the downloadable version or on the GitHub version

Follow Vulnerabilities

European Union Vulnerability Database
https://euvd.enisa.europa.eu/

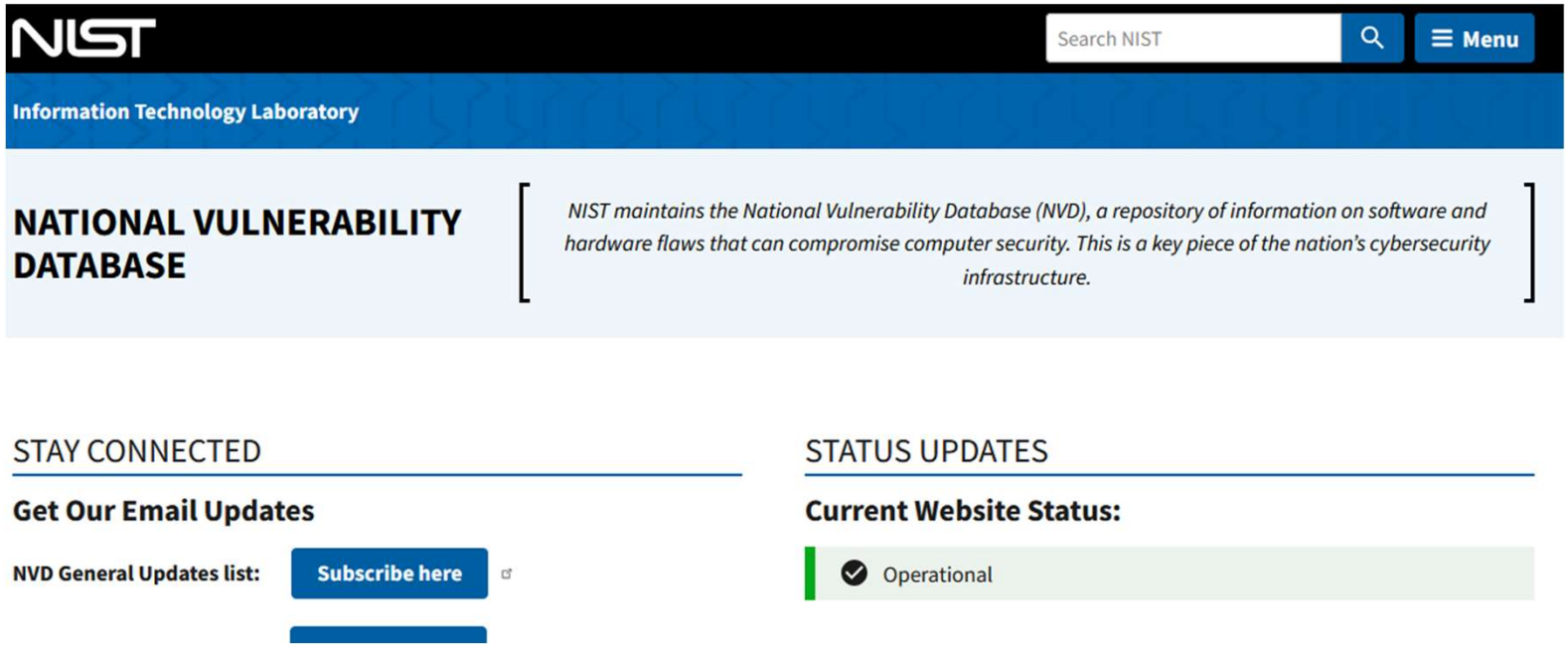


ST's Product Security Incident Response Team
https://www.st.com/content/st_com/en/about/security-and-privacy/psirt.html



Follow Vulnerabilities in the US

NIST Vulnerability Database
https://www.nist.gov/itl/nvd



PSA Certification

PSA Certified offers a framework for securing connected devices, from analysis through to security assessment and certification. The framework provides standardized resources to help resolve the growing fragmentation of IoT requirements and ensure security is no longer a barrier to product development

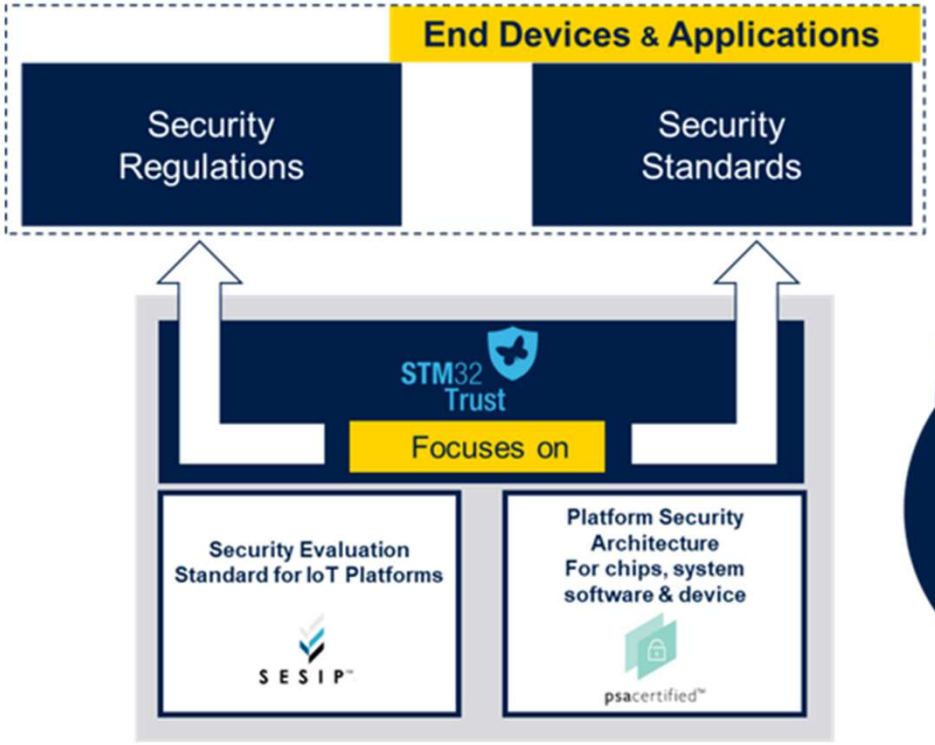
https://www.psacertified.org/

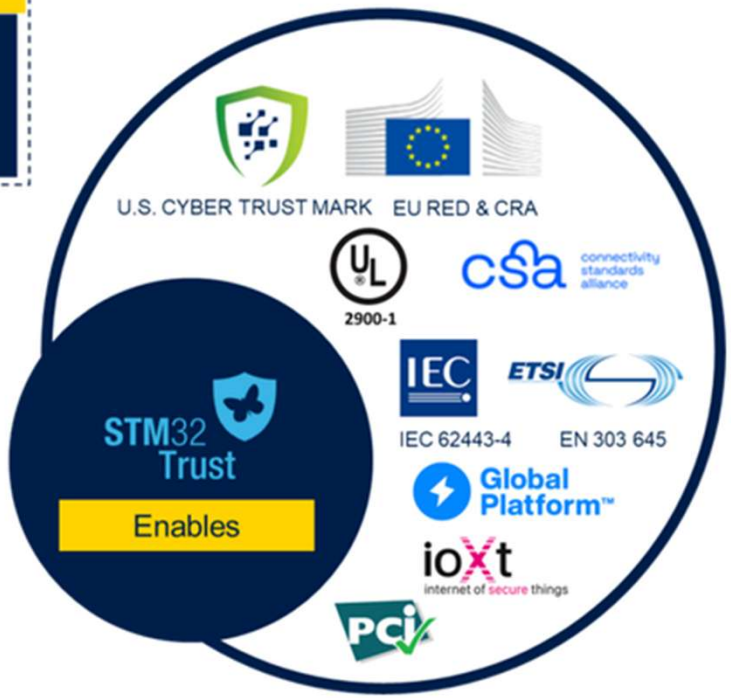
PSA Framework Example

5.1 Code Integrity

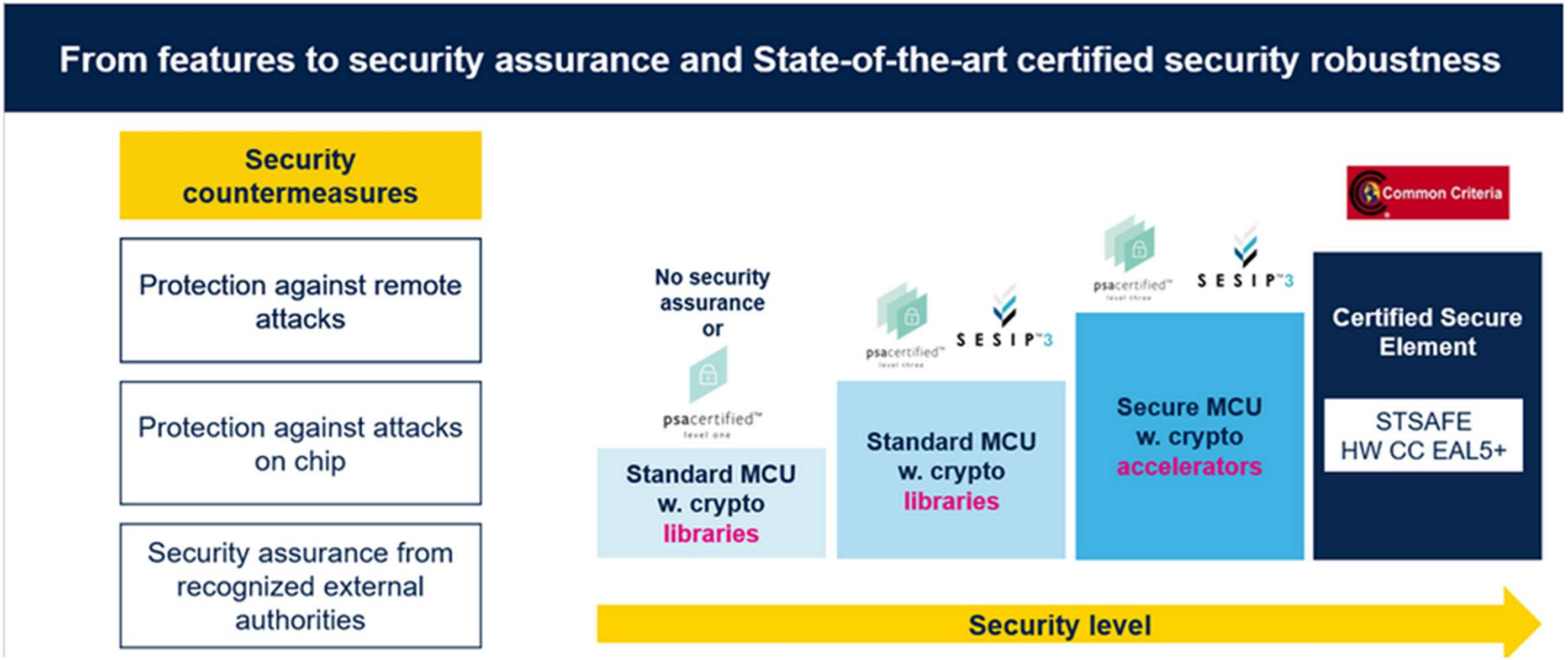
ID	Requirement	Supported?		
		Yes	Partial	N/A
S1.1	The System Software shall support update of the system software and the application specific software, either from locally connected devices (such as removable media) or from remote servers. Updates shall be validated by the system software or the PSA-RoT to check the integrity and authenticity immediately prior to execution and, optionally, at the time of download. This includes the executable code and any related data, such configuration data and version numbering. The cryptography used shall comply with requirement S2.3.			
	(Describe how updates are validated, including the cryptographic algorithms, the key sizes and where the keys used for validation are stored. Justification is required if local validation of an update from remote servers prior to installation cannot be supported, typically due to resource constraints.)			

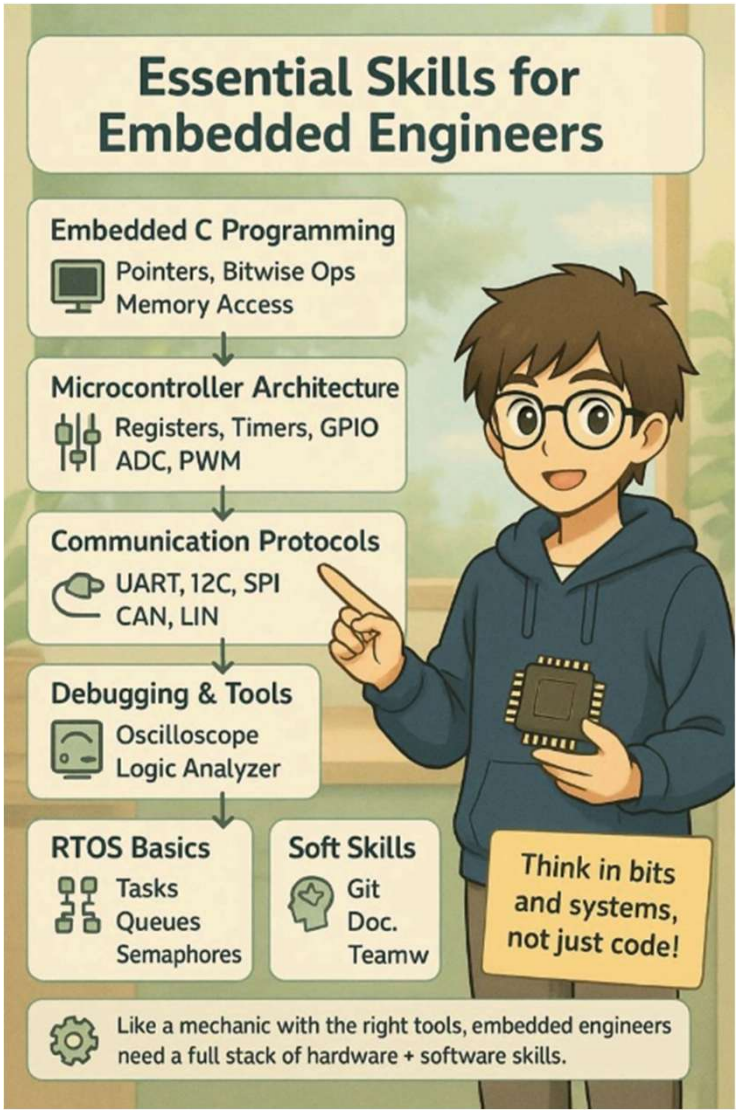
STM32 Trust


End Devices & Applications
Security Regulations
Security Standards
STM32 Trust
Focuses on
Security Evaluation Standard for IoT Platforms
SESIP
Platform Security Architecture For chips, system software & device
psacertified


U.S. CYBER TRUST MARK
EU RED & CRA
2900-1
connectivity standards alliance
IEC
ETSI
IEC 62443-4
EN 303 645
STM32 Trust
Enables
Global Platform
ioXt
internet of secure things
PCI

Risk Assessment





And much more nowadays…

✓Artificial Intelligence

✓Physics, sensors and analog electronics

✓Wireless technologies

✓Security (Software, Encryption)

✓**Hardware Security**

ISEN ALL IS DIGITAL! OUEST | yncréa

Microcontrollers Security/ New Requirements

Introduction to STM32 microcontrollers security

Introduction

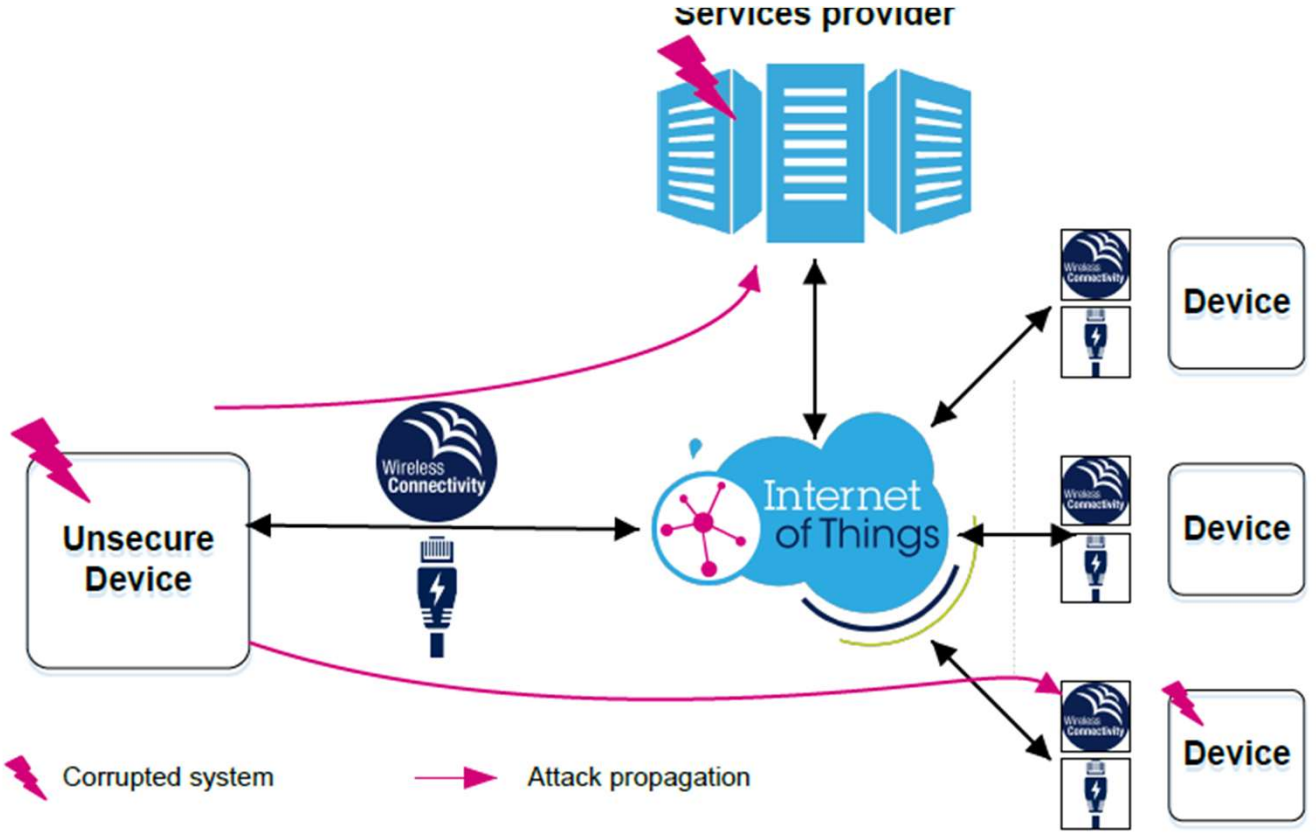
This application note presents the basics of security in STM32 microcontrollers.

Security in microcontrollers encompasses several aspects including protection of firmware intellectual property, protection of private data in the device, and guarantee of a service execution.

The context of IoT has made security even more important. The huge number of connected devices makes them an attractive target for attackers and several remote attacks have shown the vulnerabilities of device communication channels. With IoT, the security extends the requirements for confidentiality and authentication to communication channels, which often require encryption.

ISEN ALL IS DIGITAL! OUEST | yncréa

Securized Chain

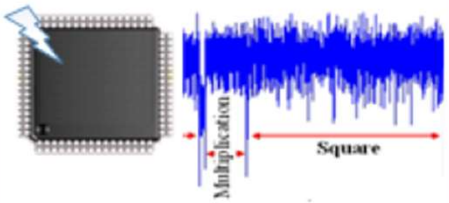
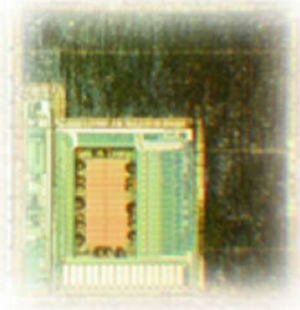


Elements to Protect

Target	Assets	Risks
Data	Sensor data (such as healthcare data or log of positions) User data (such as ID, PIN, password or accounts) Transactions logs Cryptographic keys	Unauthorized sale of personal data Usurpation Spying Blackmail
Control of device (bootloader, malicious application)	Device correct functionality Device/user identity	Denial of service Attacks on service providers Fraudulent access to service (cloud)
User code	Device hardware architecture/design Software patent/architecture Technology patents	Device counterfeit Software counterfeit Software modification Access to secure areas

ISEN ALL IS DIGITAL! OUEST | yncréa

Types of Attacks

Attacks types	Software	Hardware non–invasive	Hardware invasive
-			
Scope	Remote or local	Local board and device level	Local device level
Techniques	Software bugs Protocol weaknesses Trojan horse Eavesdropping	Debug port Power Glitches Fault injection Side-channels analysis	Probing Laser FIB Reverse engineering
Cost/ expertise	From very low to high, depending on the security failure targeted	Quite low cost. Need only moderately sophisticated equipment and knowledge to implement.	Very expensive. Need dedicated equipment and very specific skills.
Objectives	Access to confidential assets (code and data). Usurpation Denial of service	Access to secret data or device internal behavior (algorithm).	Reverse engineering of the device (silicon intellectual property) Access to hidden hardware and software secrets (Flash access)

Hardware Attack

A distinction is made between two types of attacks, that differ in cost, time and necessary expertise:

- Non-invasive attacks have only external access to the device (board-level attack) and are moderately expensive (thousands to tens of thousands US dollars in equipment).
- Invasive attacks have direct access to device silicon (after de-packing). They are carried out with advanced equipment often found in specialized laboratories. They are very expensive (more than 100k dollars, and often in the range of millions) and target very valuable data (Keys or IDs) or even technological patents.

In the IoT World, standard microcontrollers are used and are poorly protected against invasive attacks up to now.
Smart card microcontrollers have extra hardware protections

In this class, we only focus on non-invasive attacks

Malware

- Modify device configuration (such as option bytes or memory attributes).
- Disable protections.
- Read memory and dump its content for firmware and data cloning.
- Trace or log device data.
- Access to cryptographic items.
- Open communication channel/interface.
- Modify or block the device functionality.

-> Requirement to know hardware and software

-> microcontroller organisation, architecture, assembly langage, compilation stages…

Embedded Programming

- Embedded Programming is a low level programming that allows to change the value of a single bit and hence modify microcontroller behavior
- If an attacker is successful to access the program and has the capacity to change only one bit of the programm, he can change application behavior or take full control of the application.

Protections Summary

Memory	Types	Description	Protections
System Flash memory	. Internal . NVM . ROM	ROM part of the Flash memory. Embeds device bootloader and other ST services.	Cannot be updated (erase/written). A part may also be unreadable.
User Flash memory	. Internal . NVM	Flash memory for user application	Internal protections: • RDP • WRP (not for SRAM) • TrustZone • PCROP (not for SRAM) • OTP (not in SRAM) • Firewall • Secure hide protection (not for SRAM) • MPU
SRAM	. Internal . Volatile	Working memory for Stack, heap or buffers. Can be used to execute the firmware downloaded from internal or external non-volatile memories.	
NAND, NOR, Octo- or Quad-SPI Flash memory	. External . NVM	Additional memory for applications or data storage	Cryptography Write protection (on Flash device) TrustZone
SDRAM	. External . Volatile	Additional RAM for application execution	Cryptography

Ways to access memory

- From the core
- Via the DMA
- Via the bootloader
- Via the debugger
- Via communication (wired transmission: I2C/SPI/UART or wireless transmission: BLE, WIFI…)

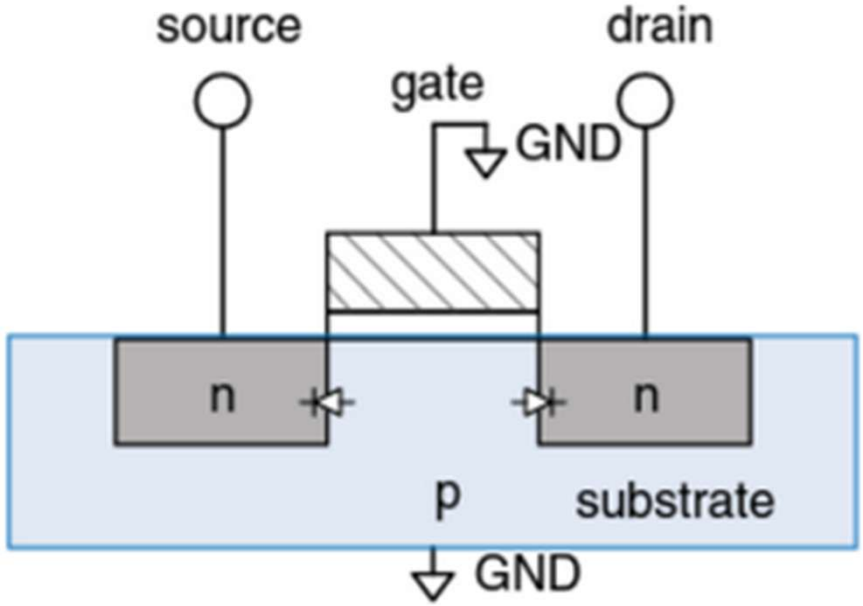


ISEN ALL IS DIGITAL! OUEST | yncréa

2: Hardware Security

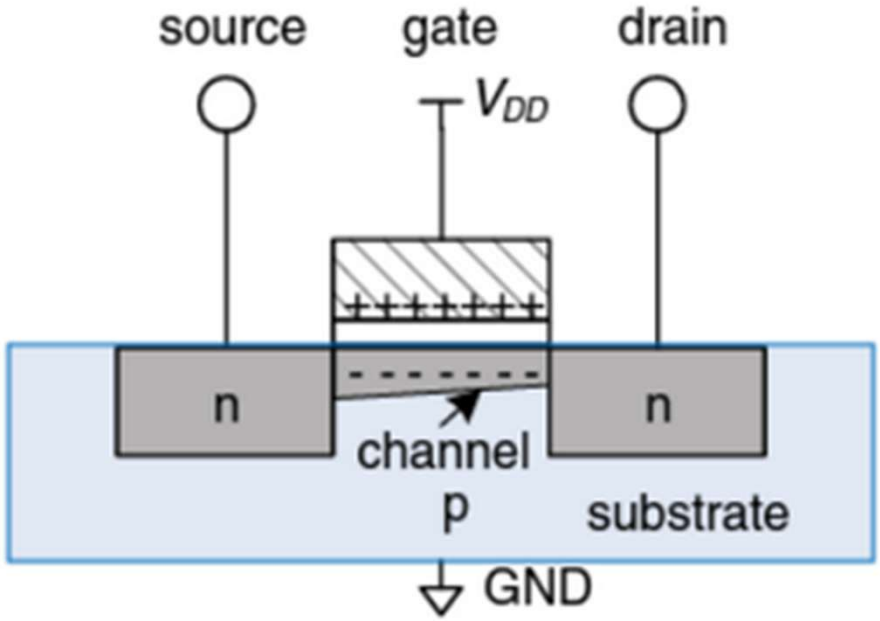
Memory Technologies in microcontrollers: Flash and SRAM

MOSFET / NMOS



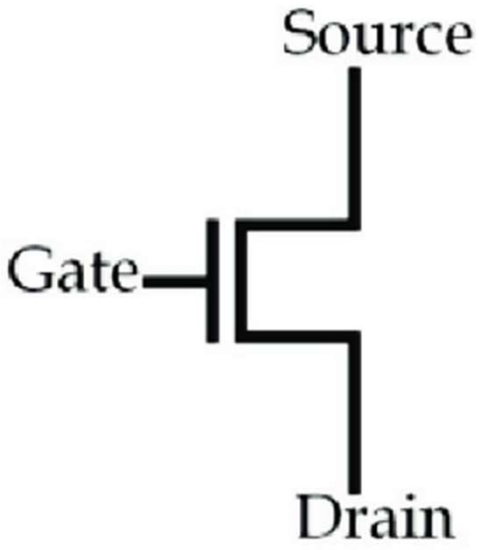
An NMOS transitor blocks the current between the source and the drain when no voltage is present on the gate.

MOSFET / NMOS



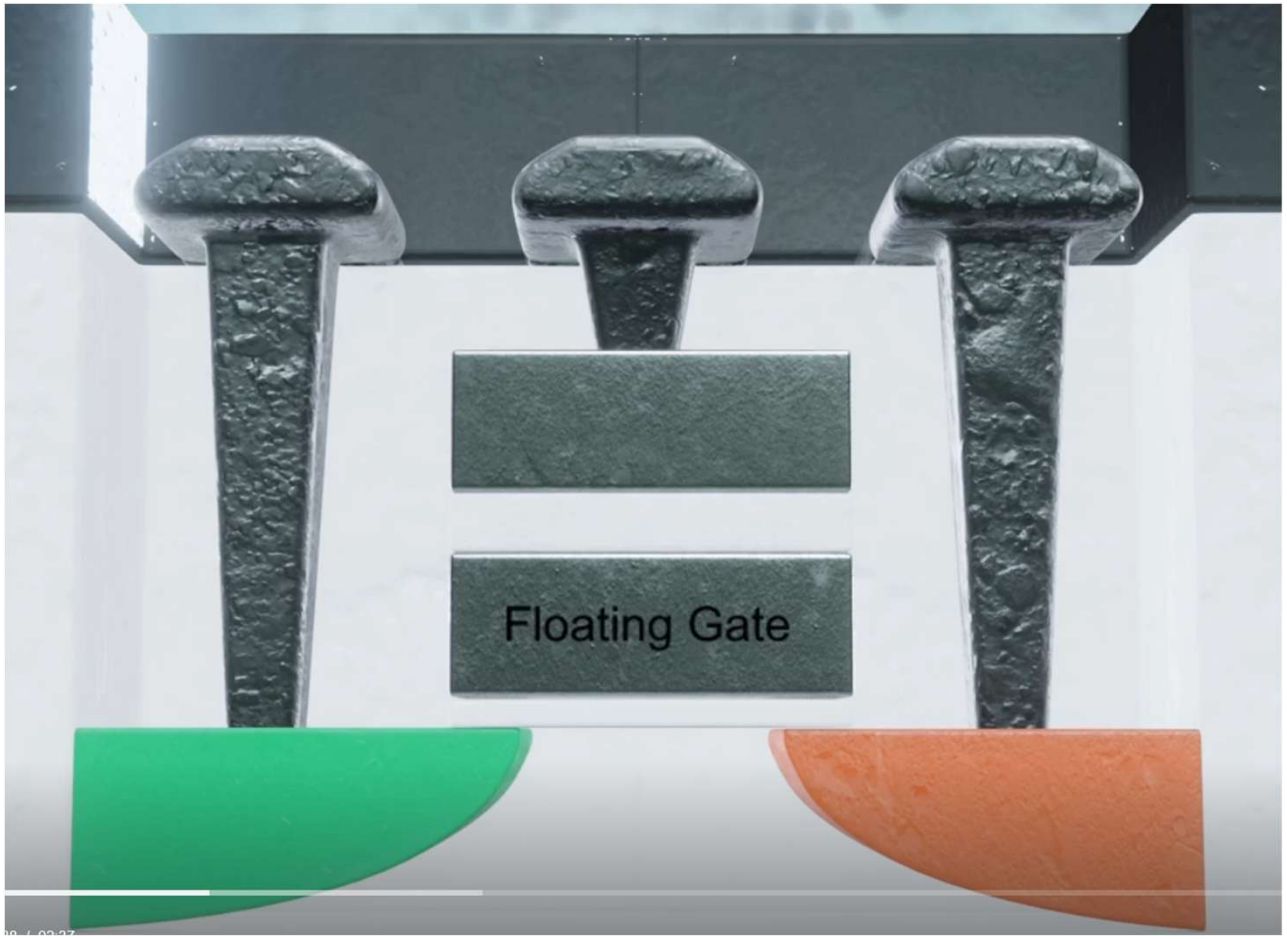
A voltage on the gate creates an electron channel between the source and the drain.

NMOS Representation



An NMOS does not keep any information when the voltage is removed.

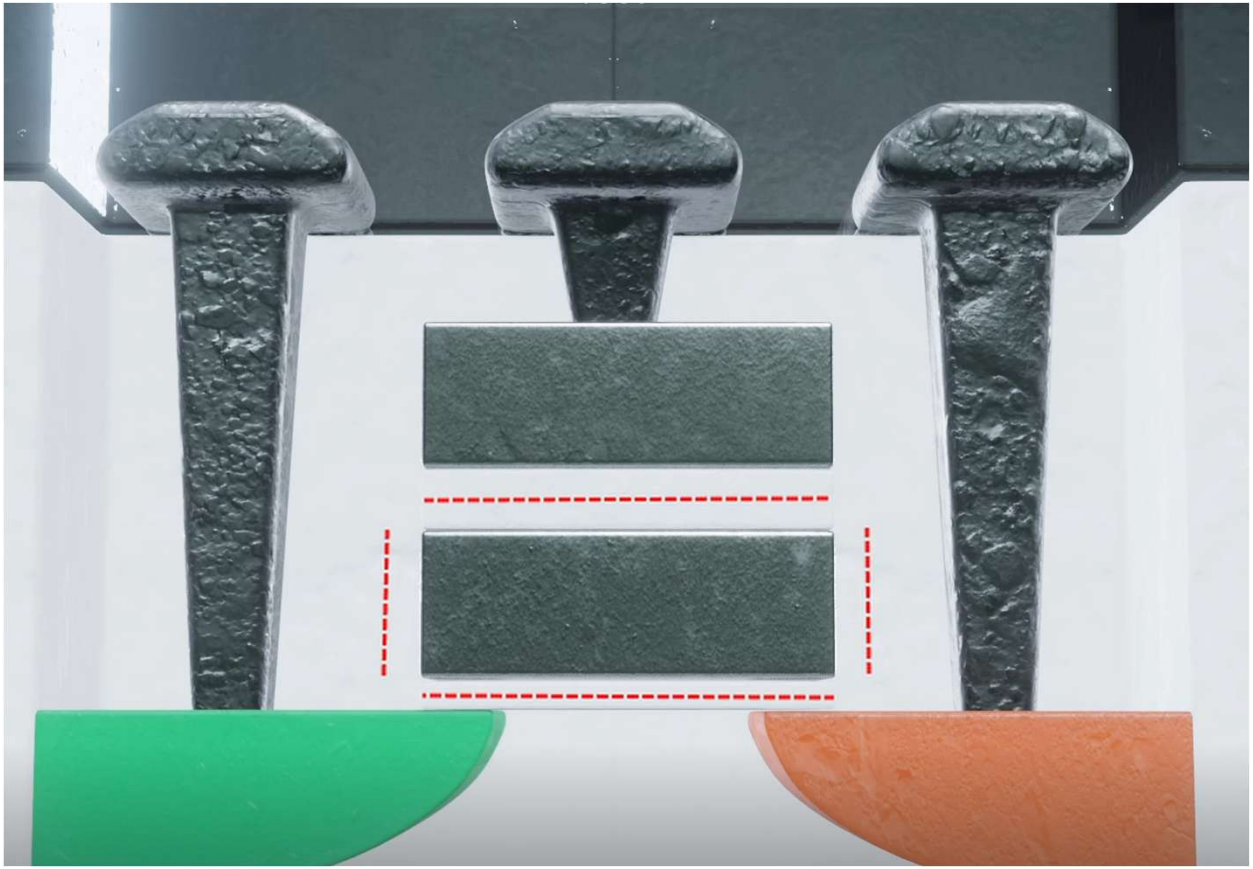
FLASH Memory Cell



A Flash memory cell is similar to an NMOS transitor but a floating gate is integrated between the gate and the channel.

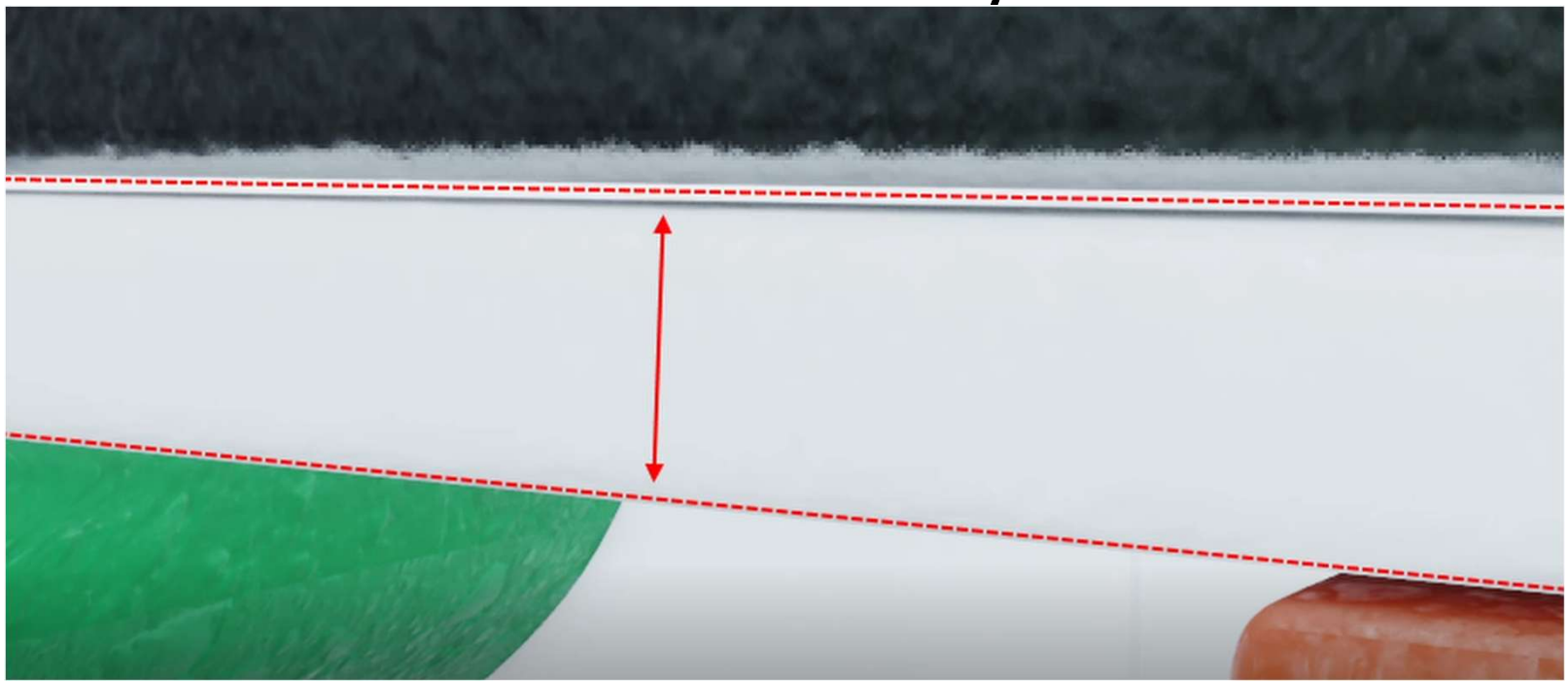
ISEN ALL IS DIGITAL! OUEST | yncréa

FLASH Memory Cell



The floating gate is completely isolated and electrons are trapped inside. Hence, it is possible to keep information even when the voltage is removed.

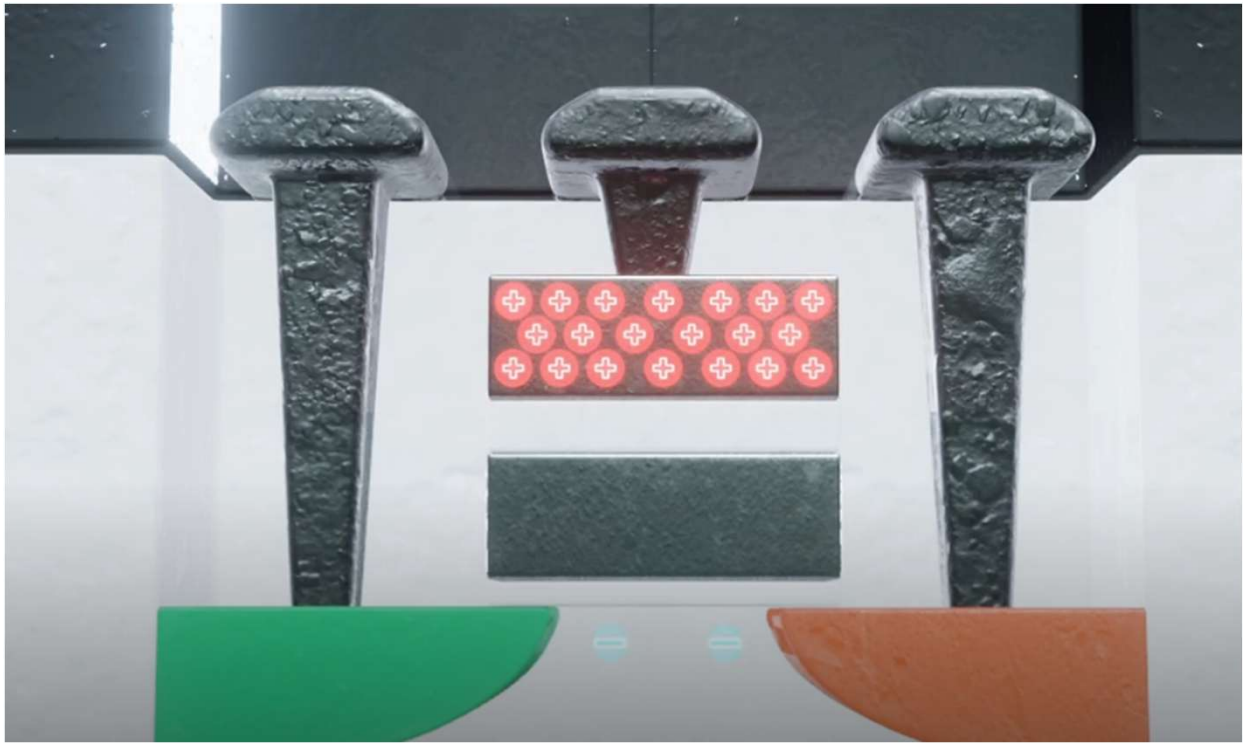
FLASH Memory Cell



The isolation between the floating gate and the channel is extremely thin, that allows to have electrons crossing this barrier in specific situations.

ISEN ALL IS DIGITAL! OUEST | yncréa

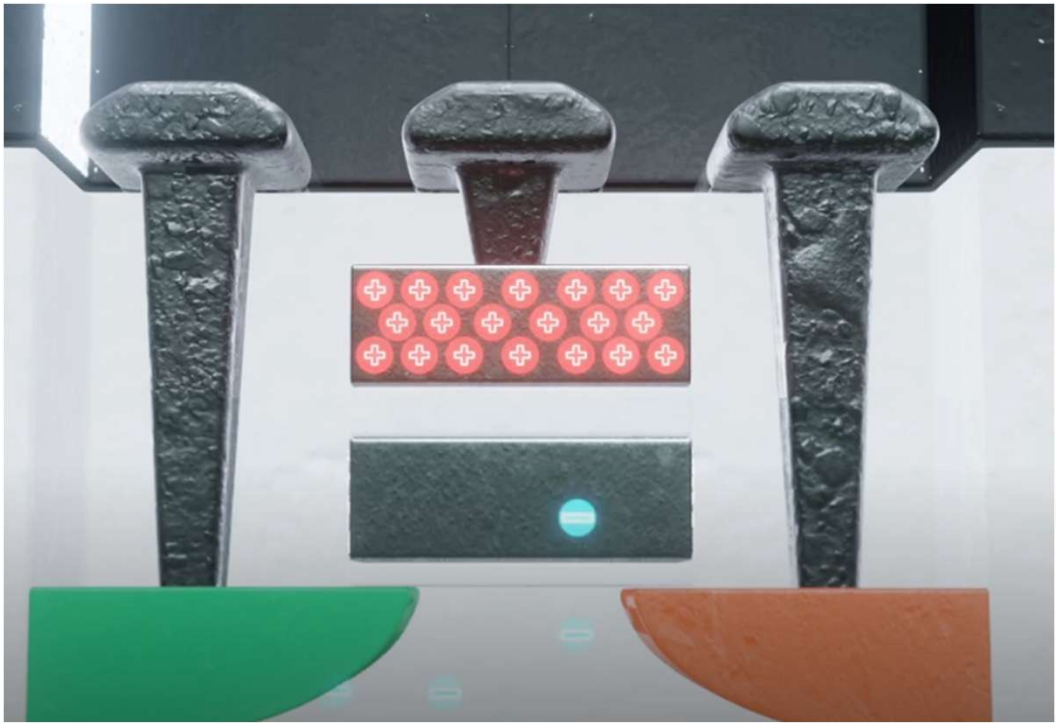
Flash Memory Cell



Electrons are inserted in the floating gate thanks to a technic called Hot Electrons Injection. A high current flowing through the channel coupled with a high vertical electrical field pushes a few electrons into the floating gate.

High current consumption to program flash memory!!!

Flash Memory Cell

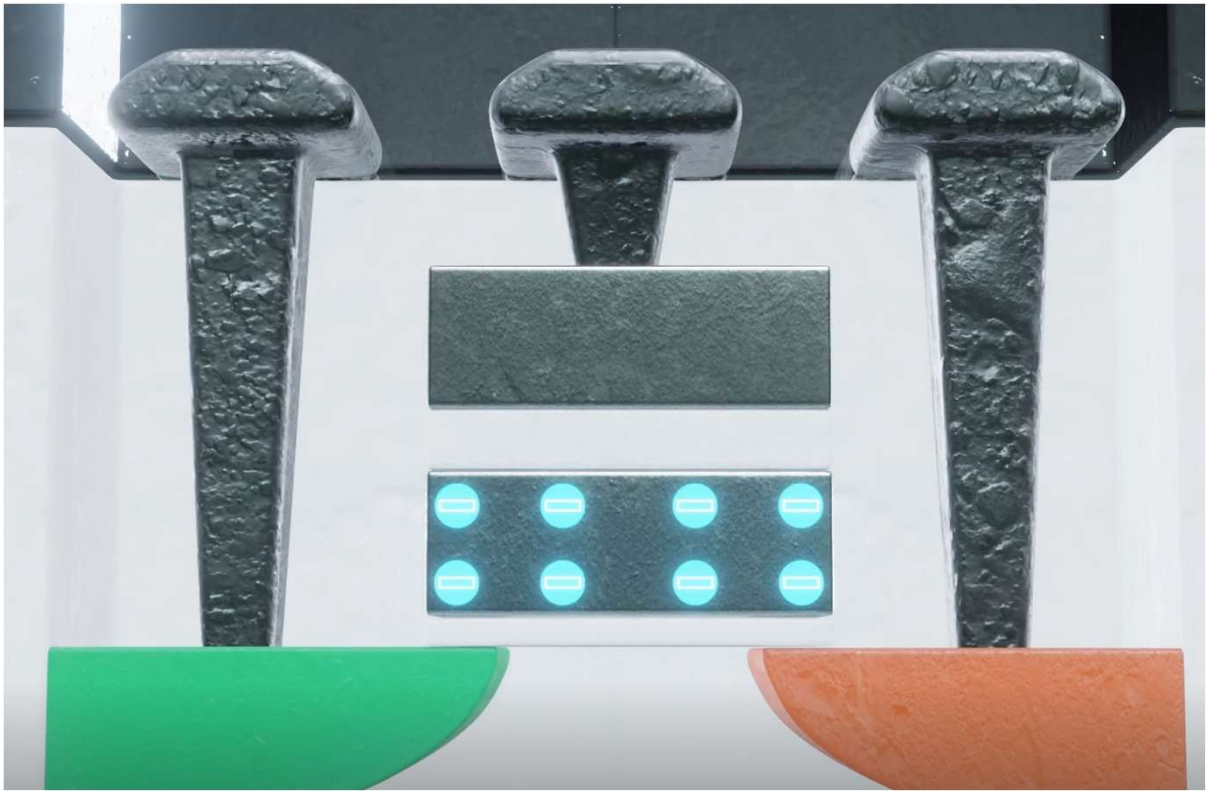


Electrons have enough speed to go through the isolation barrier.

Light destructions of the isolating barrier each time the chip is programmed -> the memory can be programmed only a limited number of times (100K for instance).

ISEN ALL IS DIGITAL! OUEST | yncréa

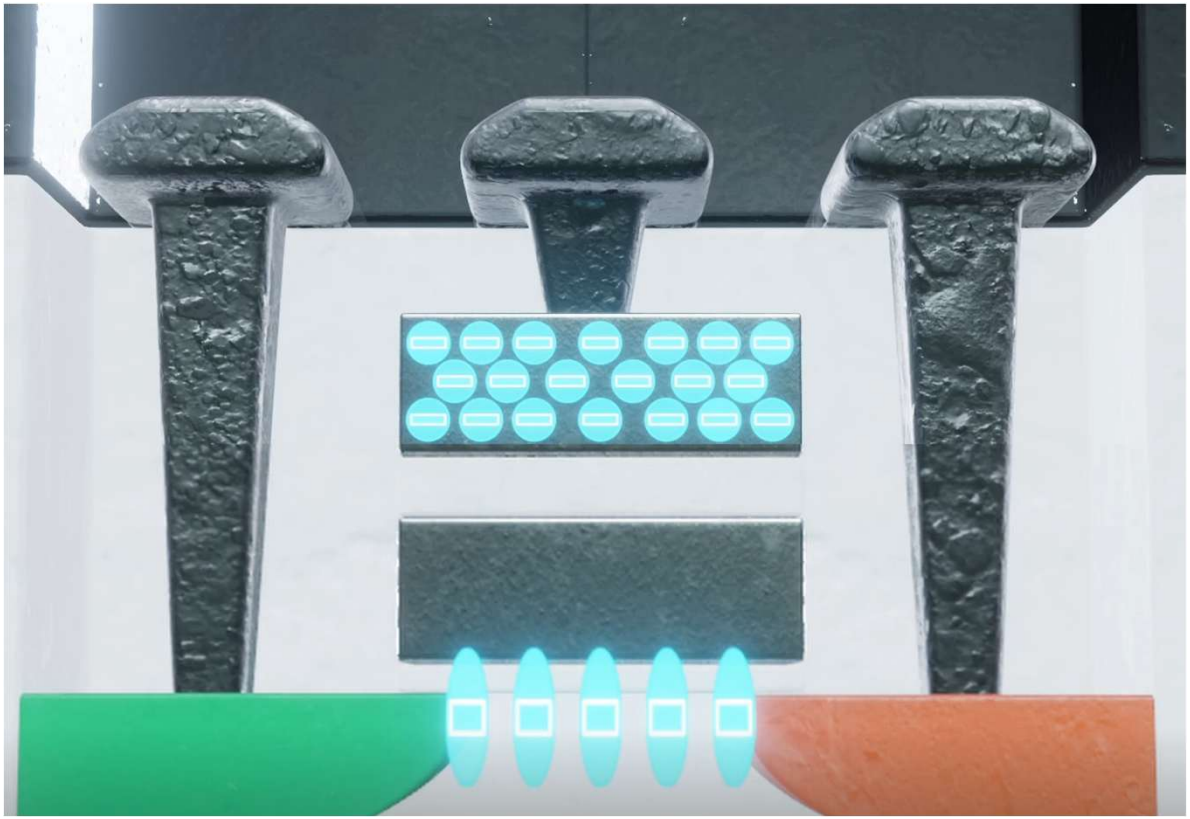
Flash Memory Cell



When the electrical field is not present anymore, electroncs are captured inside the floating gate.

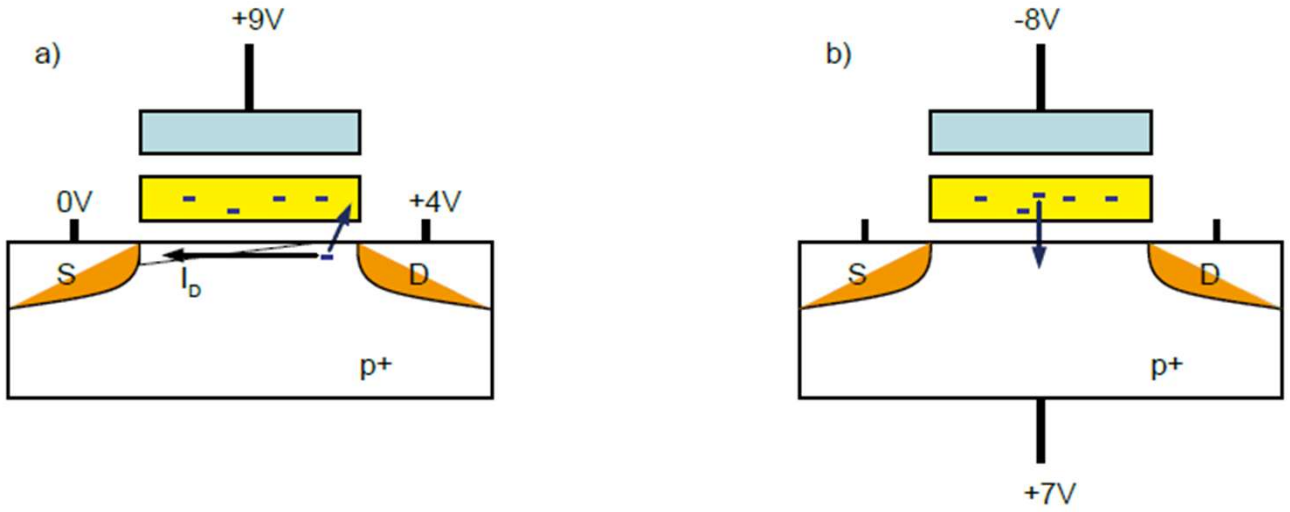
ISEN ALL IS DIGITAL! OUEST | yncréa

Flash Memory Cell



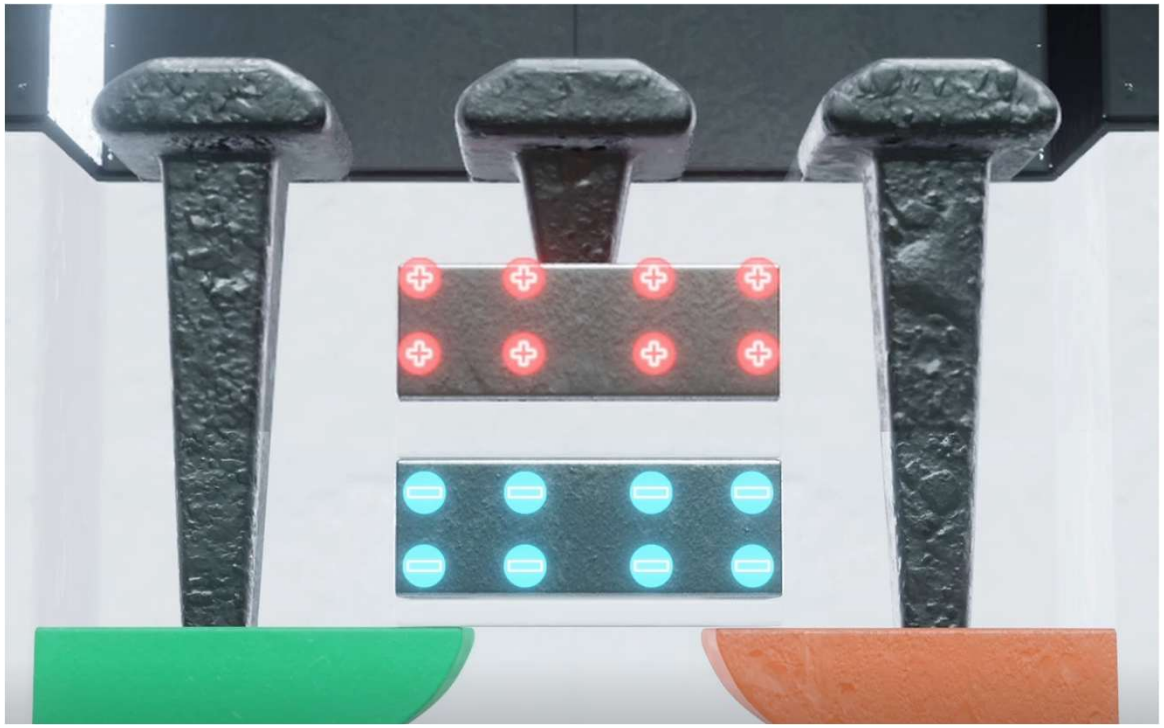
To remove electrons, a negative voltage is applied to the gate. This phenomenon is called Tunnel Fowler Nordheim effect.

Flash Memory Voltages



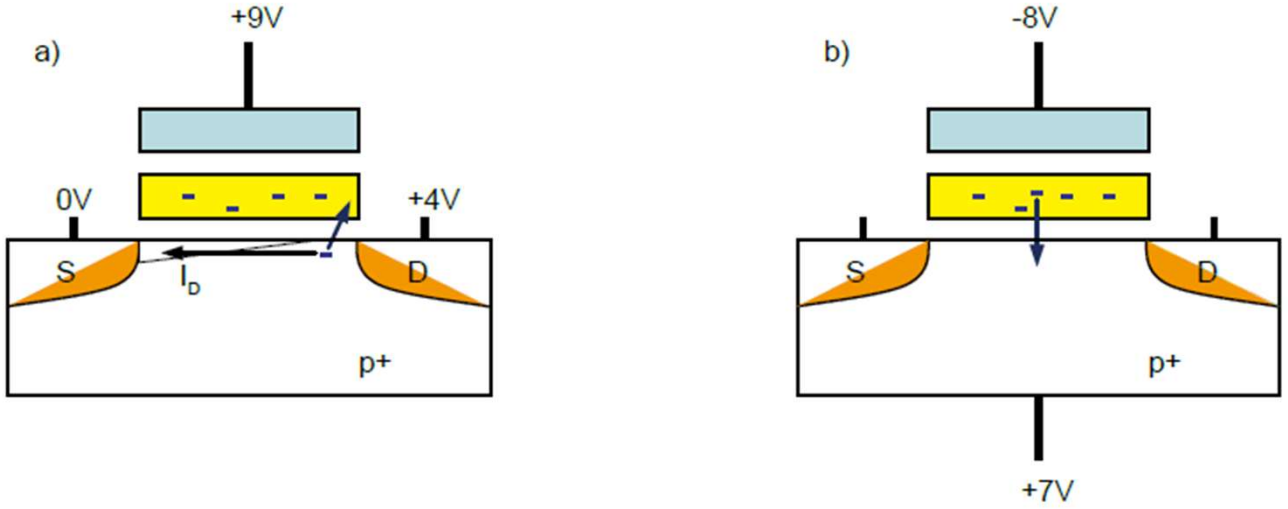
Even in a microcontroller running at 3V, voltages to write or erase memory are higher. In modern microcontrollers, a charge pump is integrated to generate these high voltages. Monitoring microcontroller consumption can detect when flash memory is programmed.

Flash Memory Cell



The presence of electrons prevents the formation of a channel between the source and the drain when a positive voltage is applied to the gate

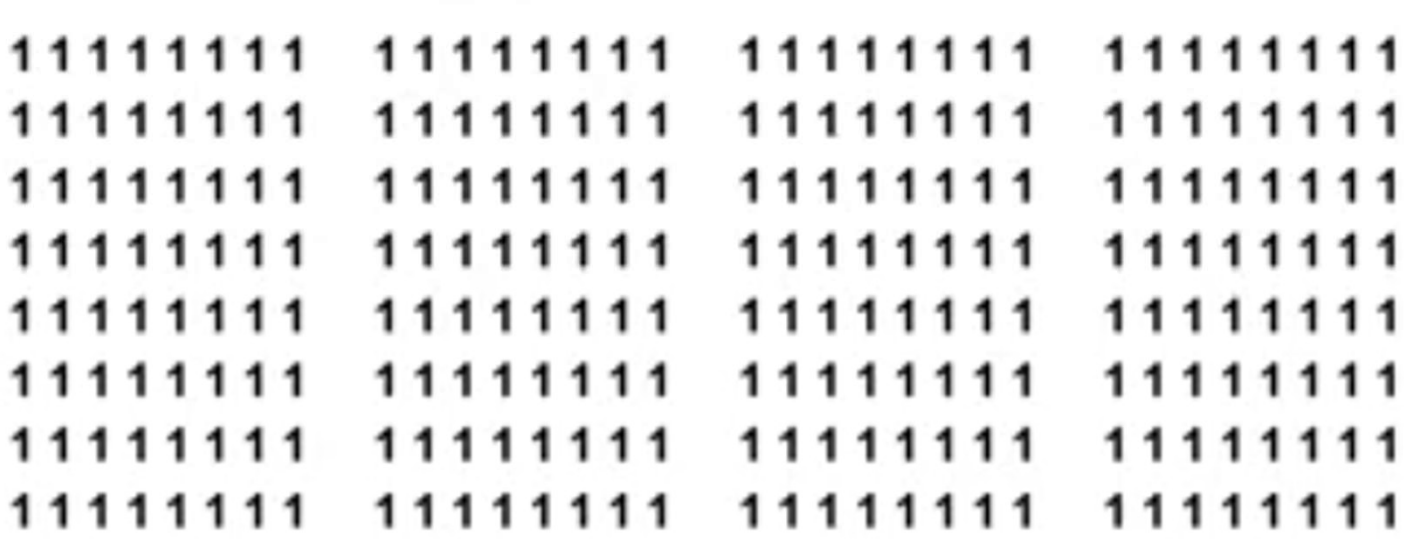
Cell Erasure



When a cell is erased, it means that electrons are removed from the floating gate -> A value of 1 is read.

Writing in Nor Flash (1/3)

When a microcontroller is fresh from the factory, its flash memory is fill in with ‘1’ as no electrons are trapped in the floating gate



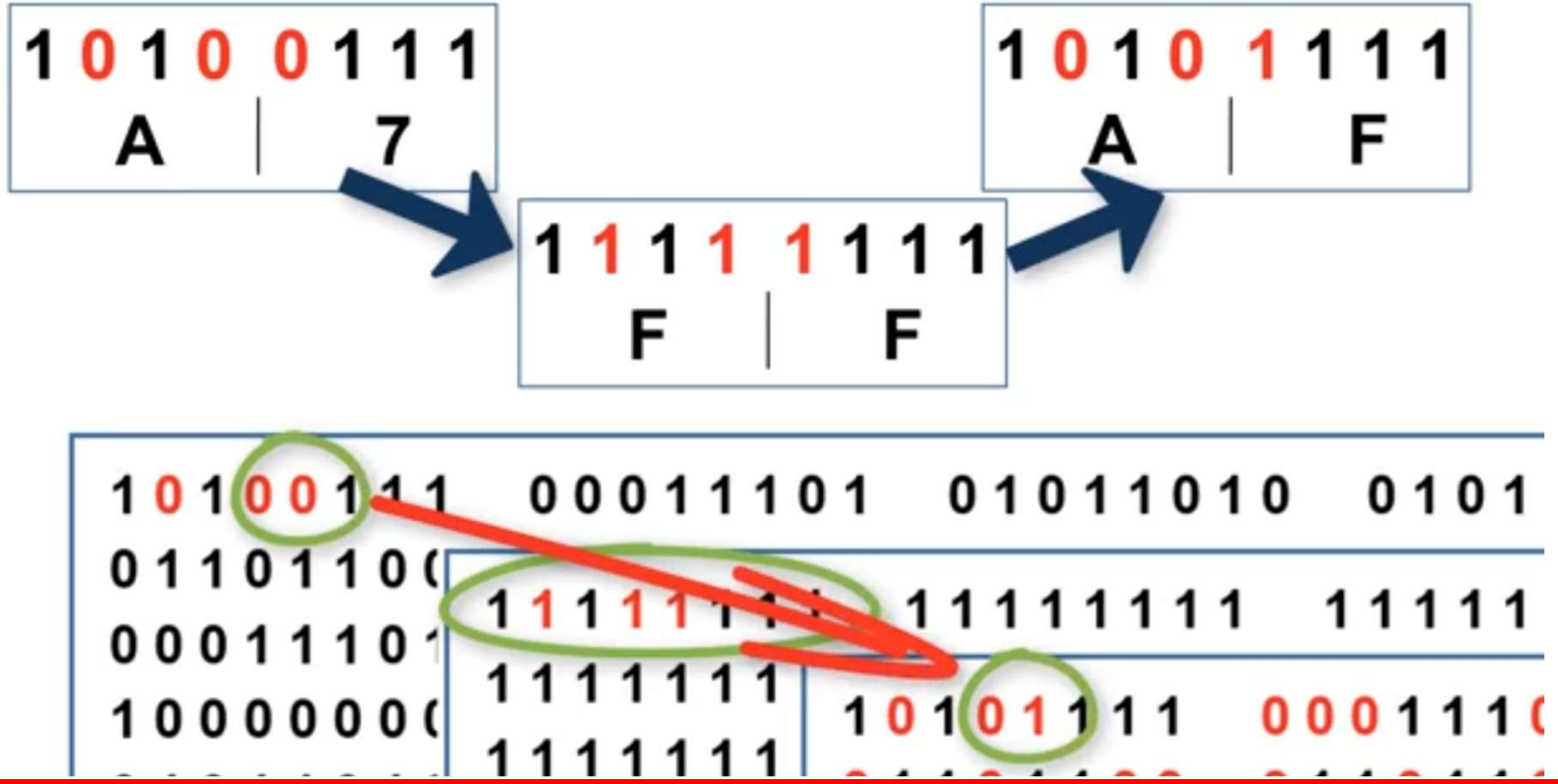
Writing in Nor Flash (2/3)

Zeroes can be programmed one by one

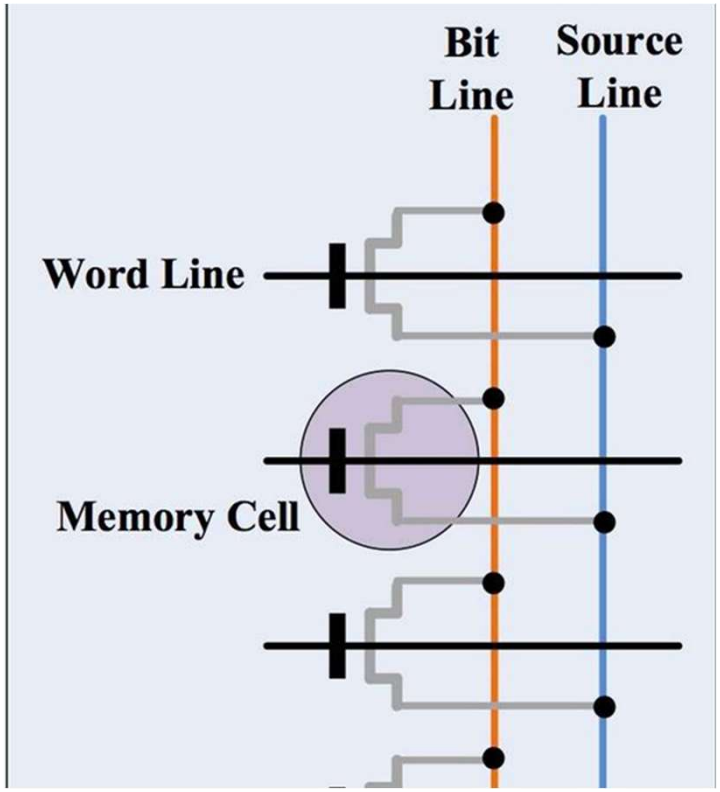
10010001	00011101	01011010	01011010
01101100	01101100	01101000	10000101
00011101	10011101	01011101	00000000
10000000	10000000	10010001	11011010
01011010	01010010	01101100	01101000
01101000	11101000	00011101	01101000
01011101	11010101	10010010	00011101
10010010	00010010	01011110	01101100
01011110	01011010	10011101	10011101
00011101	10000101	00000000	10000000

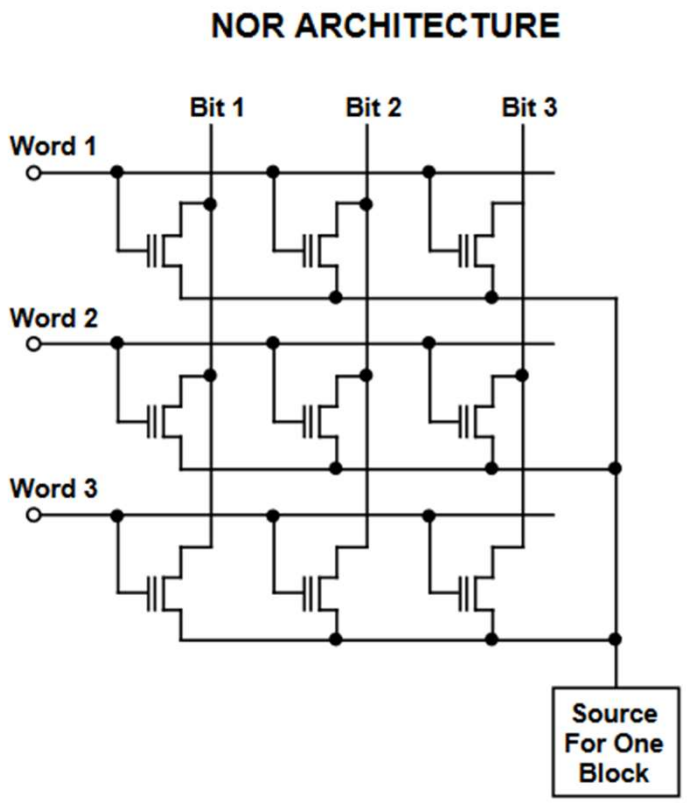
Writing in Nor Flash (3/3)

To switch from ‘0’ to ‘1’ it is mandatory to make a full erase of the flash memory

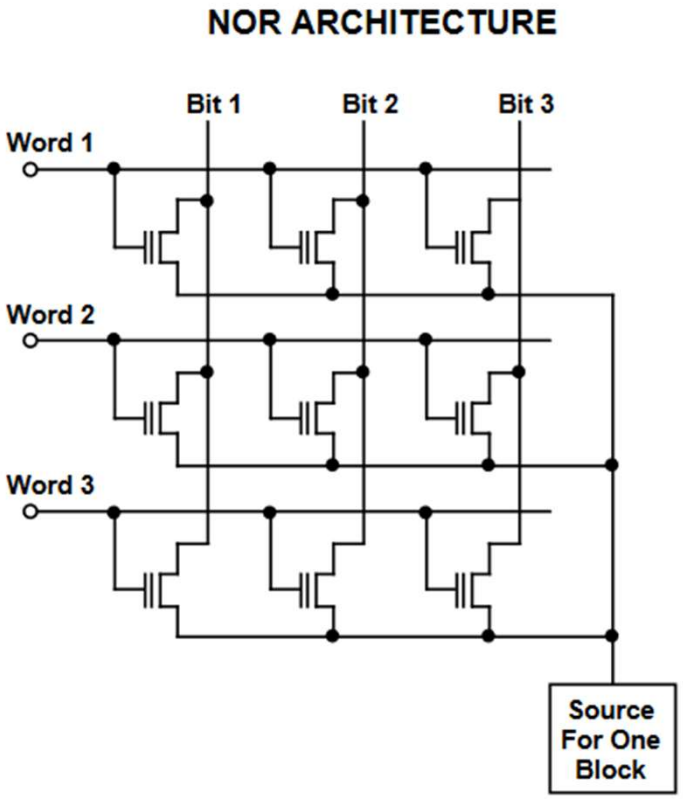


Flash Nor Memory Organisation



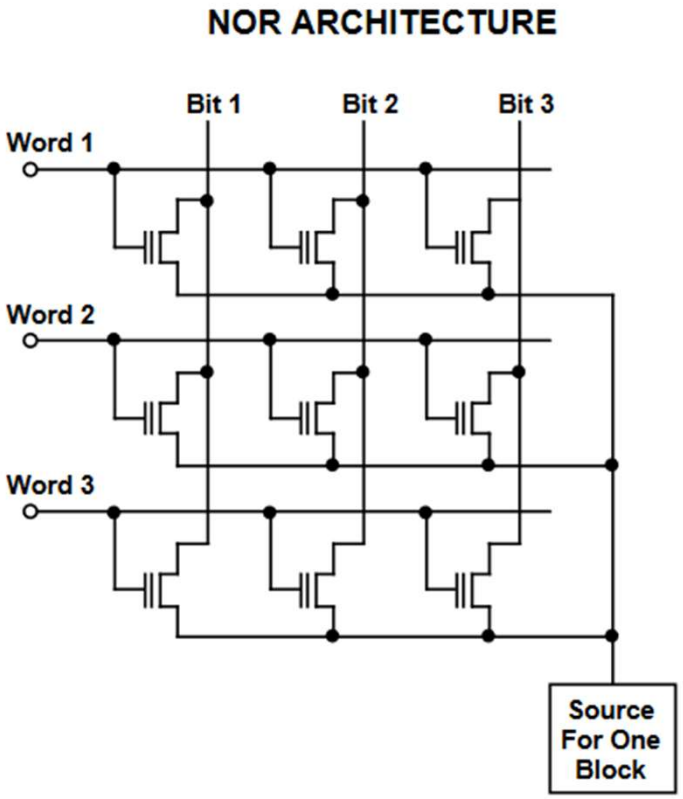


Flash Writing



A high voltage is applied to the gate. A positive voltage is applied to the drain to create or not an electrons channel

Flash Erasing



The source is powered with a high voltage, the gate is at 0 or 3V and the drain is left floating.

→ Erasing of a block
→ Check datasheet to know the size of the block (page)

Flash Program

Flash program and erase operations

The STM32L47x/L48x/L49x/L4Ax embedded Flash memory can be programmed using in-circuit programming or in-application programming.

The **in-circuit programming (ICP)** method is used to update the entire contents of the Flash memory, using the JTAG, SWD protocol or the boot loader to load the user application into the microcontroller. ICP offers quick and efficient design iterations and eliminates unnecessary package handling or socketing of devices.

In contrast to the ICP method, **in-application programming (IAP)** can use any communication interface supported by the microcontroller (I/Os, USB, CAN, UART, I^2C, SPI, etc.) to download programming data into memory. IAP allows the user to re-program the Flash memory while the application is running. Nevertheless, part of the application has to have been previously programmed in the Flash memory using ICP.

The contents of the Flash memory are not guaranteed if a device reset occurs during a Flash memory operation.

Write into Flash

Writing into a flash is not a straight forward process

Unlocking the Flash memory

After reset, write is not allowed in the *Flash control register (FLASH_CR)* to protect the Flash memory against possible unwanted operations due, for example, to electric disturbances. The following sequence is used to unlock this register:

1. Write KEY1 = 0x45670123 in the *Flash key register (FLASH_KEYR)*
2. Write KEY2 = 0xCDEF89AB in the FLASH_KEYR register.

Any wrong sequence will lock up the FLASH_CR register until the next system reset. In the case of a wrong key sequence, a bus error is detected and a Hard Fault interrupt is generated.

Flash main memory erase sequences

The Flash memory erase operation can be performed at page level, bank level or on the whole Flash memory (Mass Erase). Mass Erase does not affect the Information block (system flash, OTP and option bytes).

Page erase

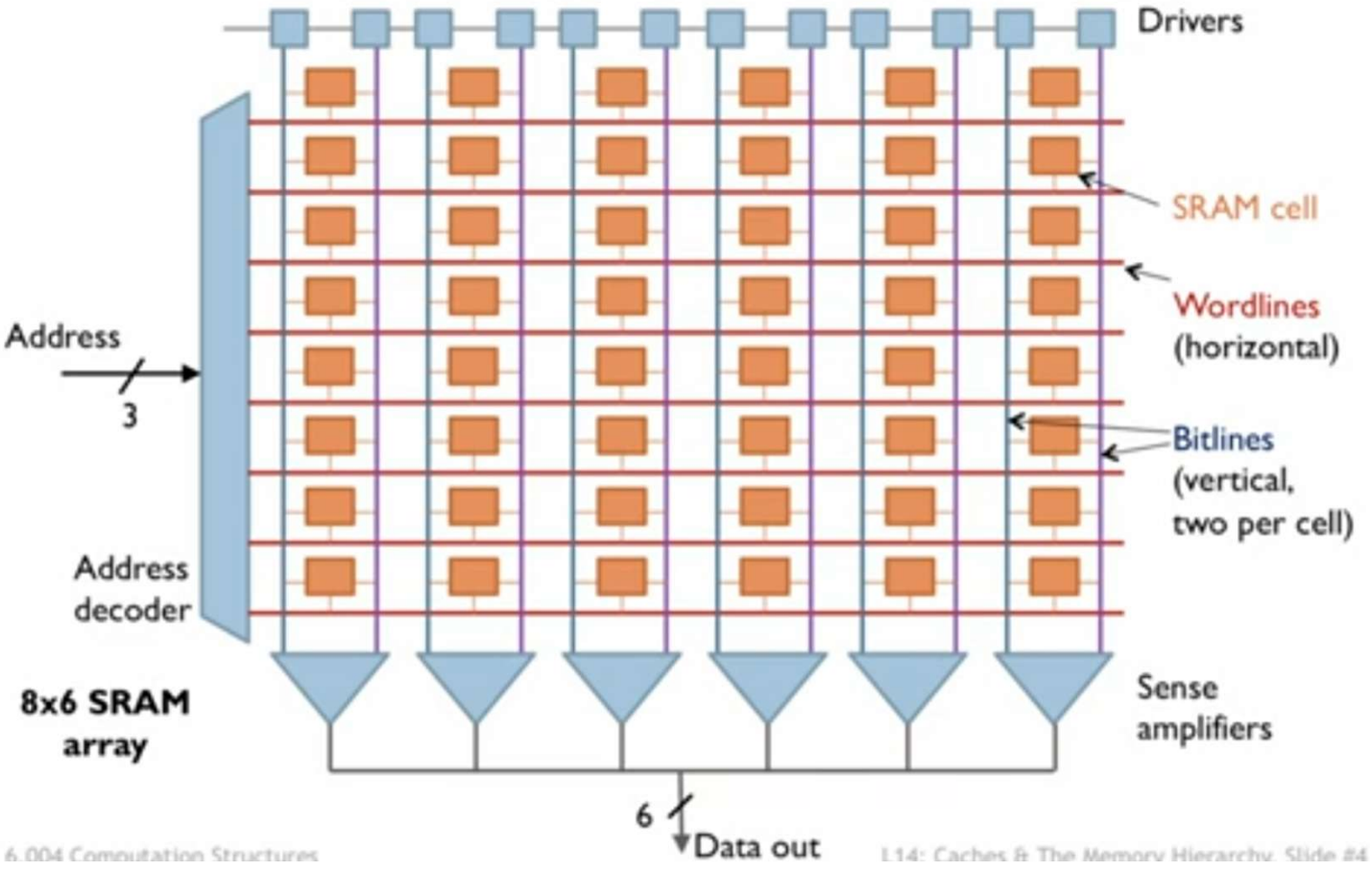
To erase a page (2 Kbyte), follow the procedure below:

1. Check that no Flash memory operation is ongoing by checking the BSY bit in the *Flash status register (FLASH_SR)*.
2. Check and clear all error programming flags due to a previous programming. If not, PGSERR is set.
3. Set the PER bit and select the page you wish to erase (PNB) with the associated bank (BKER) in the *Flash control register (FLASH_CR)*.
4. Set the STRT bit in the FLASH_CR register.
5. Wait for the BSY bit to be cleared in the FLASH_SR register.

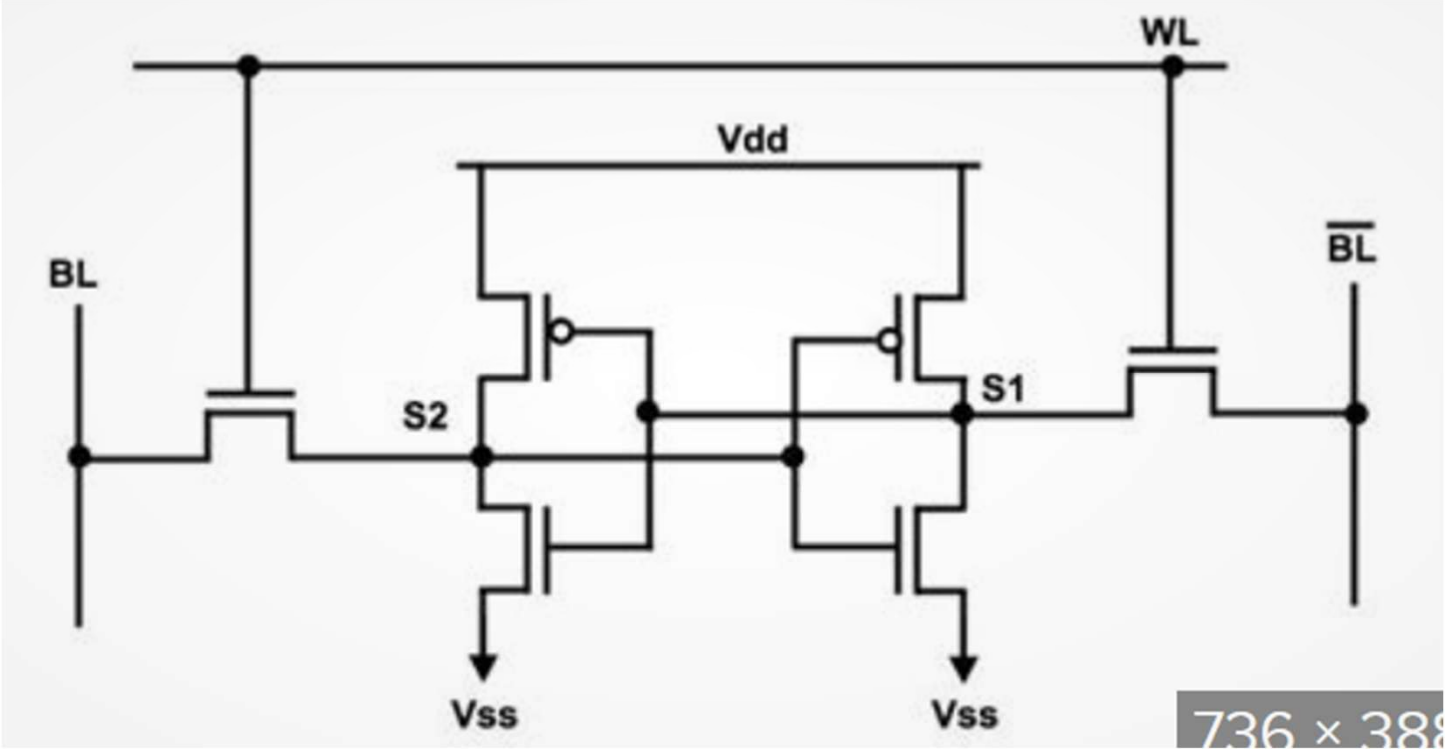
The internal oscillator HSI16 (16 MHz) is enabled automatically when STRT bit is set, and disabled automatically when STRT bit is cleared, except if the HSI16 is previously enabled with HSION in RCC_CR register.

If the page erase is part of write-protected area (by WRP or PCROP), WRPERR is set and the page erase request is aborted.

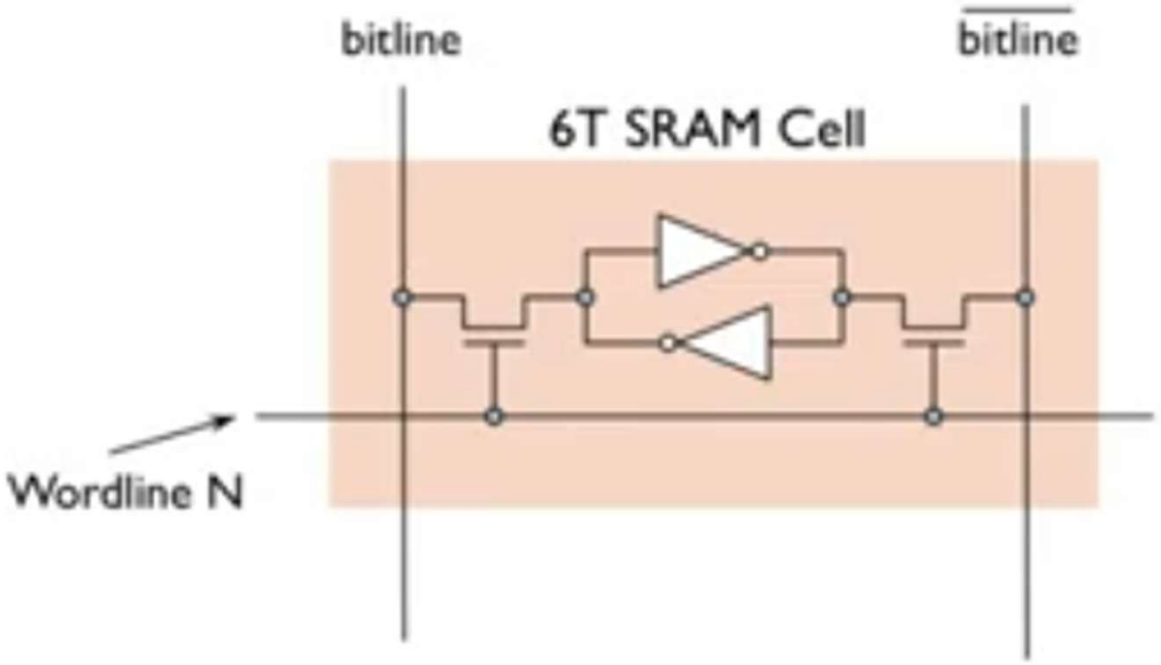
SRAM Organisation



SRAM Cell

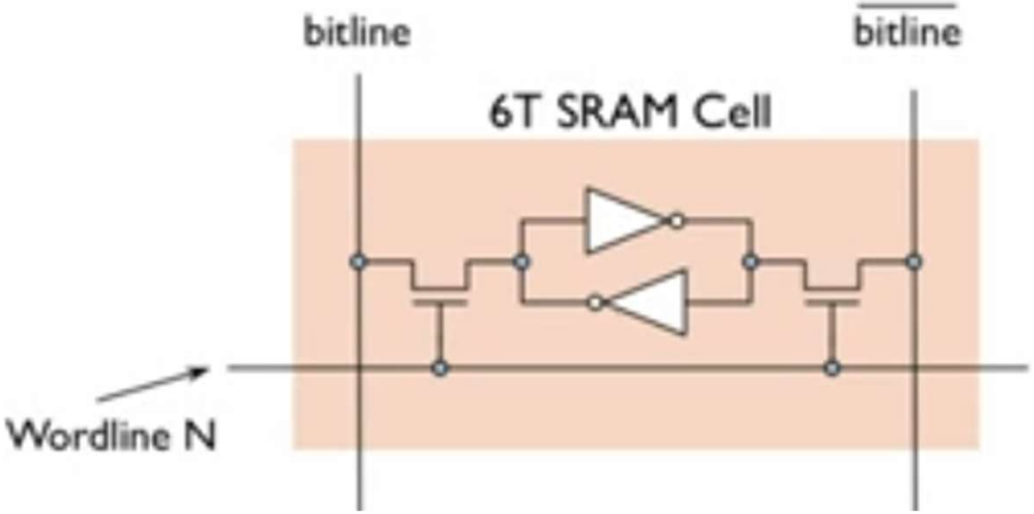


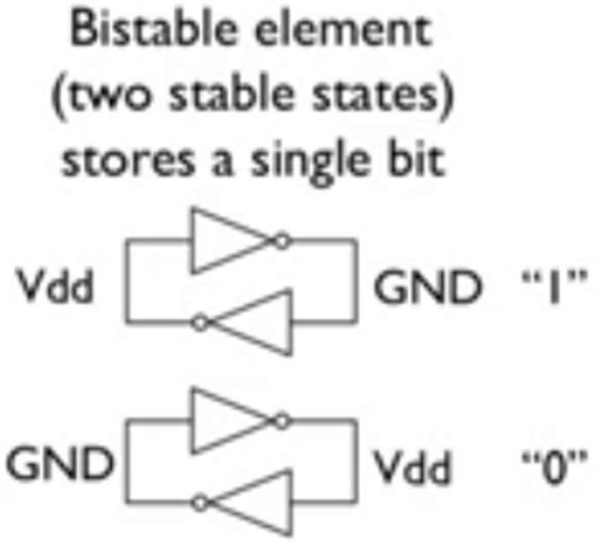
SRAM Cell



4 NMOS transistors sesrve as inverters

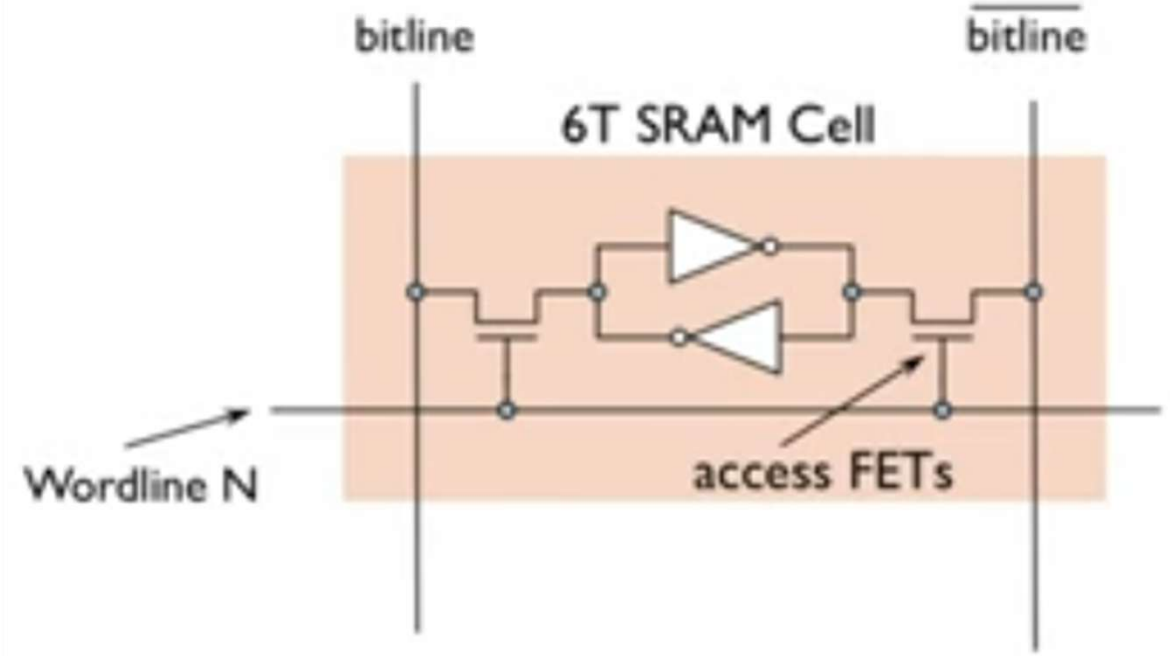
SRAM Cell





The inverter pair maintains the bit value. The cell needs a constant voltage to maintain its state

SRAM Cell



When the worldline is high, the access transistors are turned on.

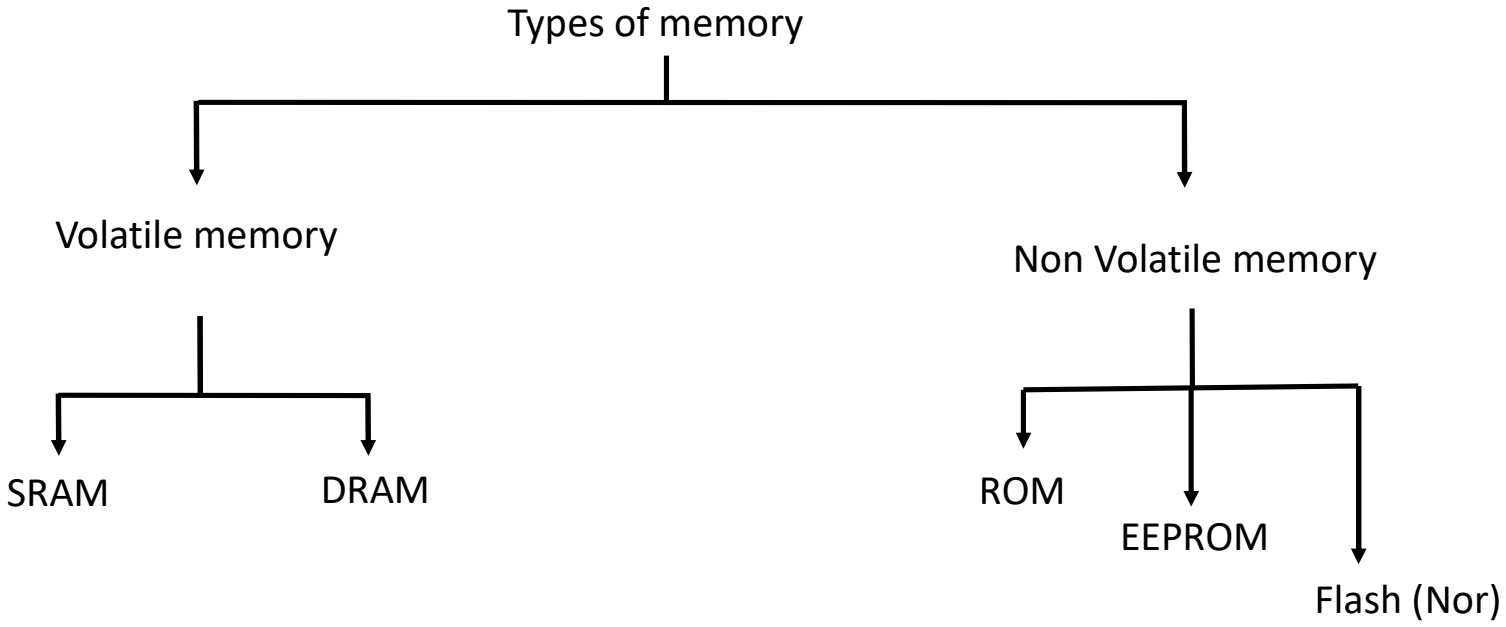
SRAM

- ✓ Without voltage, it is impossible to guarantee that the information is preserved
- ✓ !!!! Without voltage, it is impossible to guarantee that the information is lost !!!!
- ✓ At low temperatures, it is possible to preserve the values of the bits for a minute (depends on SRAMs)
- ✓ In ultra-secure applications, a negative voltage is applied to destroy the silicon in connection with an intrusion (tamper)

Memory Parameters

- ✓ Volatility (able to keep data if Vcc is On or Off)
- ✓ Granularity in read and write cycle
- ✓ Endurance
- ✓ Access time to read and write
- ✓ Size / Cost

Memory types

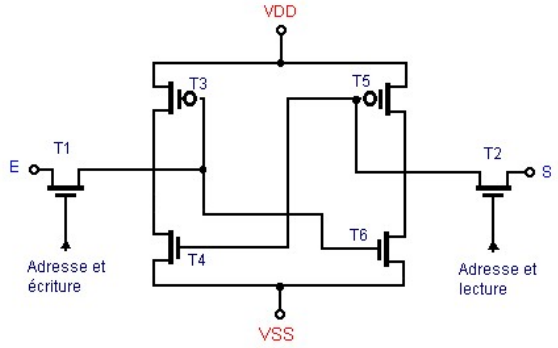


ISEN ALL IS DIGITAL! OUEST | yncréa

SRAM Characteristics

SRAM stands for Static Random Access Memory

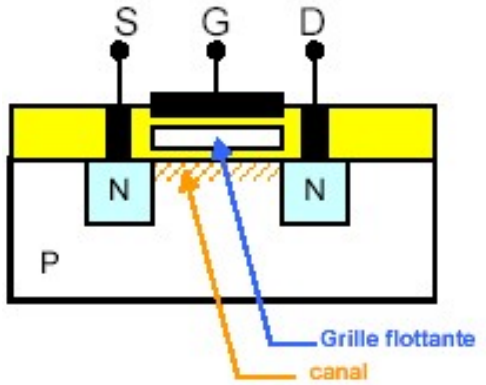
- ✓ Volatile
- ✓ 8-bit granularity in read and write cycle
- ✓ Unlimited Endurance
- ✓ Fast access time to read and write
- ✓ Large Size / High Cost
- ✓ Low Current Consumption



Always Used in µControllers

FLASH (NOR) Characteristics

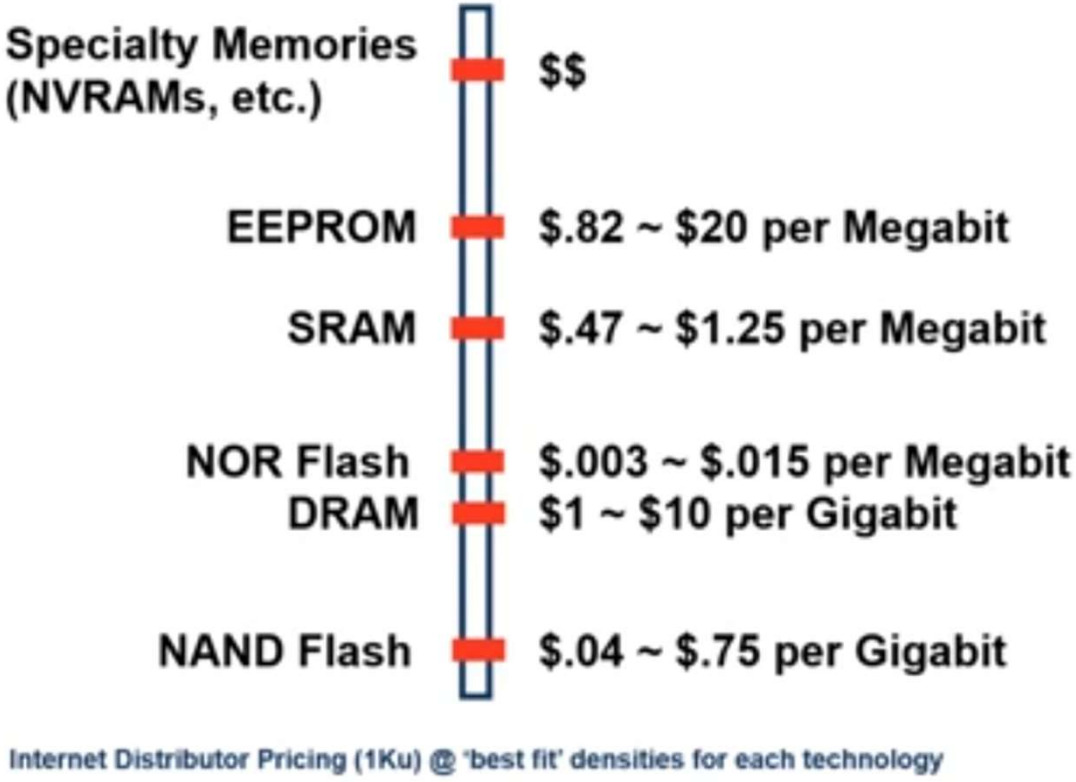
- ✓ Non-Volatile
- ✓ 8-bit granularity in read cycle. Must write in pages (block of bytes)
- ✓ limited Endurance in write cycle
- ✓ Very Slow access time in write cycle
- ✓ Medium in readnng (around 50MHz max)
- ✓ Medium Size / Lower Cost than EEPROM



Always used in µControllers

ISEN ALL IS DIGITAL! OUEST | yncréa

Memory cost



- ✓Cost by bit
- ✓Nor Flash is the most optimized to store a program

!!! Prix mémoire varie grandement

* Source Microchip 2021



ISEN ALL IS DIGITAL! OUEST | yncréa

3: Hardware Security

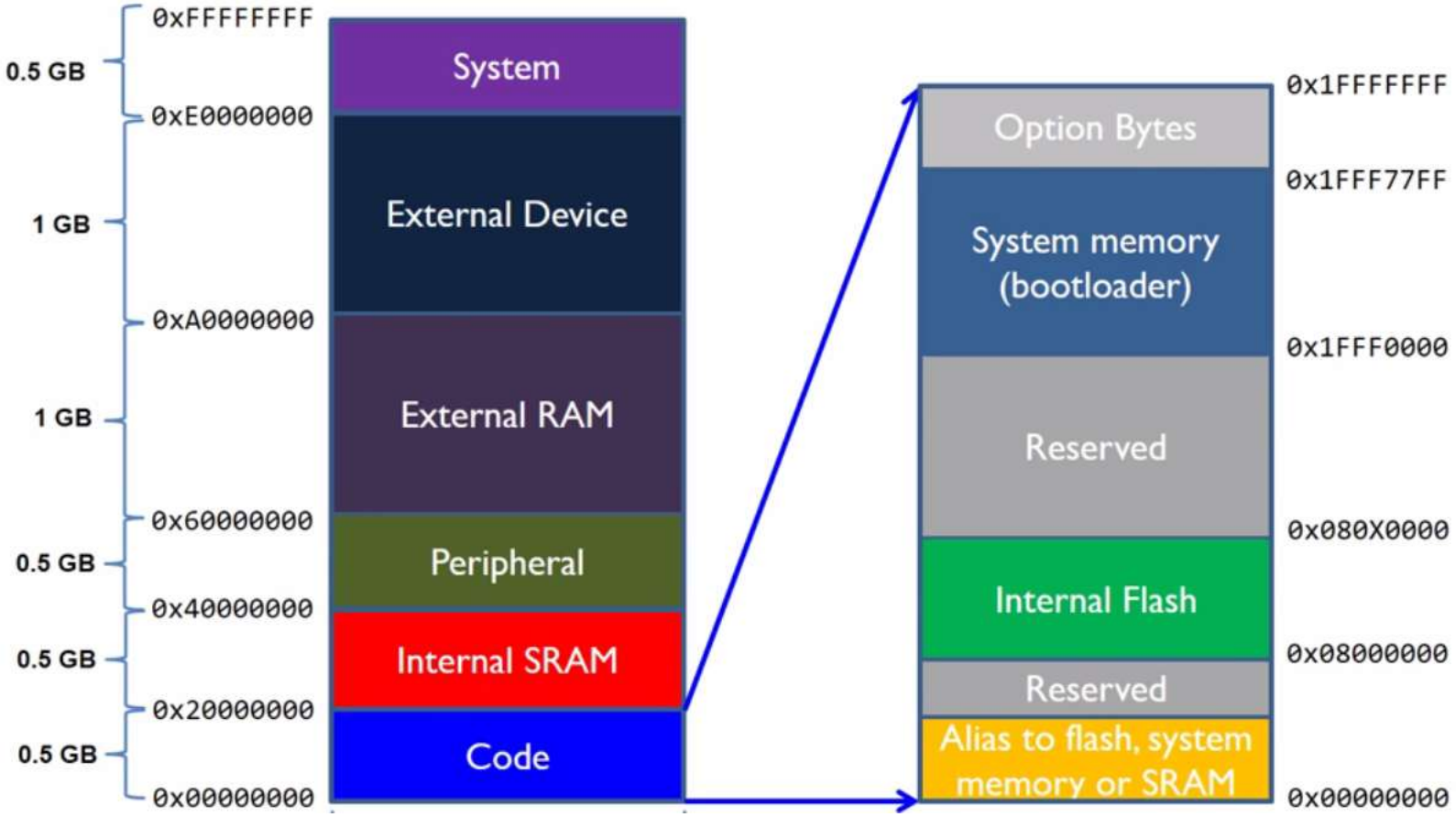
Memory organization, Microcontroller architecture, Assembly language

ISEN ALL IS DIGITAL! QUEST | yncréa

STM32Lx Architecture

- ✓ STM32Lx uses a Harvard architecture / separate program and data memory buses
- ✓ Logical address space is unified (from 0x0000 0000 to 0xFFFF FFFFF.)

STM32 Memory Organization



!!! addresses are different depending on components

Memory Map Registers vs Core Registers

4GB Memory Map

Vendor Specific	
Private peripheral bus - external	
Private peripheral bus- internal	
External Device	1.0GB
External RAM	1.0GB
Peripheral	0.5GB
SRAM	0.5GB
Code	0.5GB

Access by pointing to an address

Registers

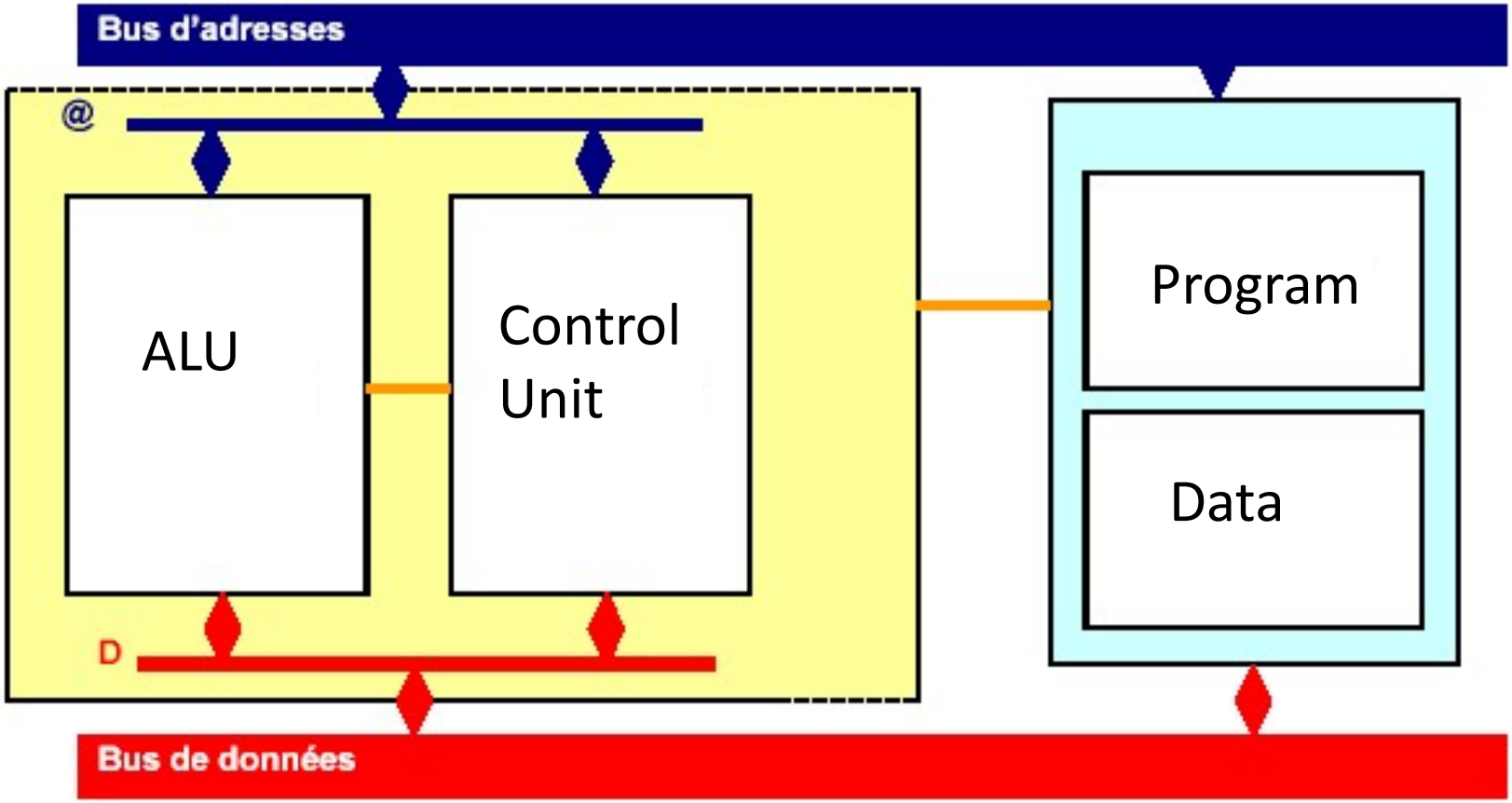
R0-R12 general purpose register
R13 (MSP/PSP)
R14 (LR)
PC
PSR
PRIMASK
FAULTMASK
BASEPRI
CONTROL

Not accessible by pointer

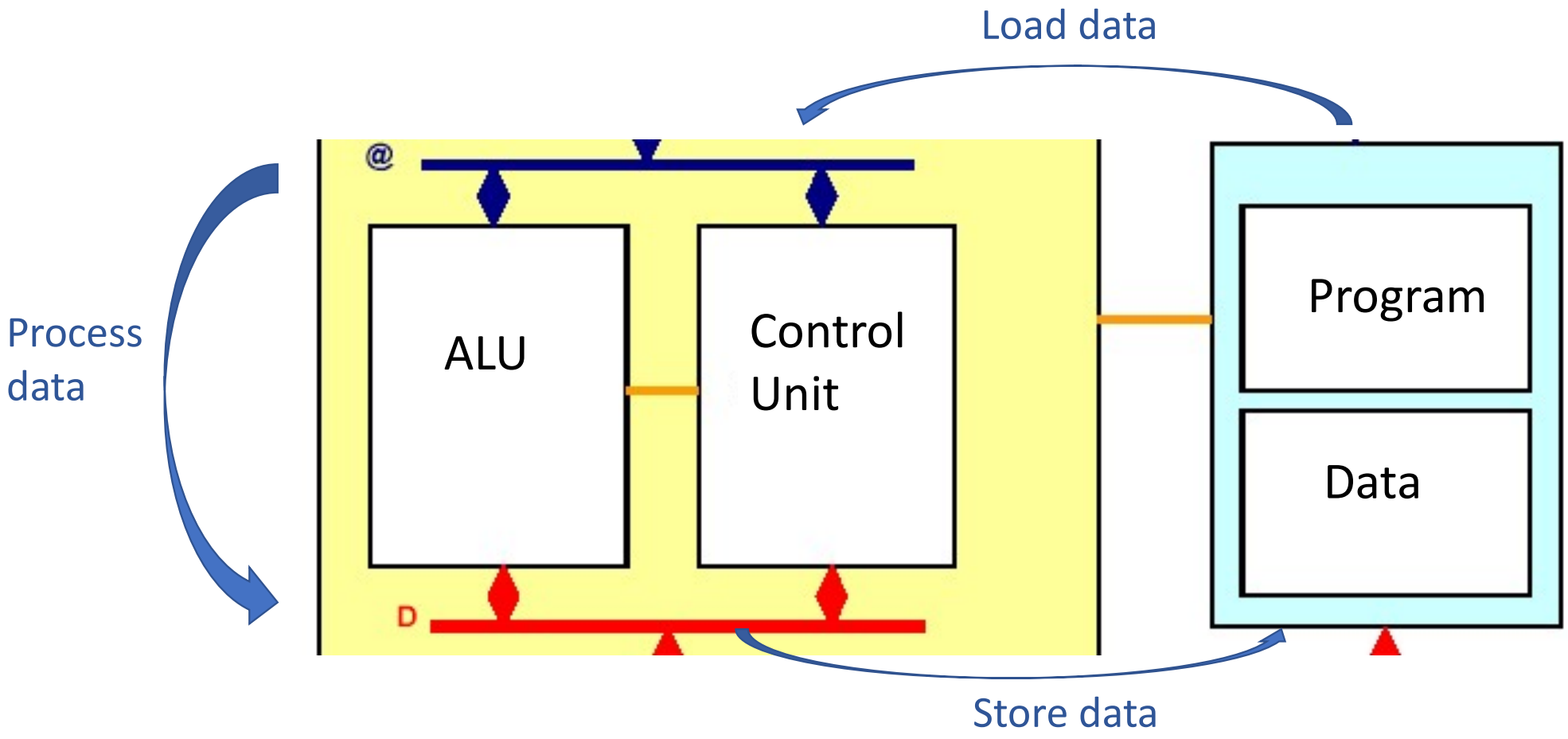
Cortex M Registers

- ✓ The Cortex-M processors apply a load and store method of operations.
- ✓ This means that any kind of data processing is performed from and to registers
- ✓ Instructions such as ADD and SUBTRACT: data must first be loaded into the CPU registers. The data processing is then executed, and the result is stored back in the main memory.

Microprocessors: Basic Architecture

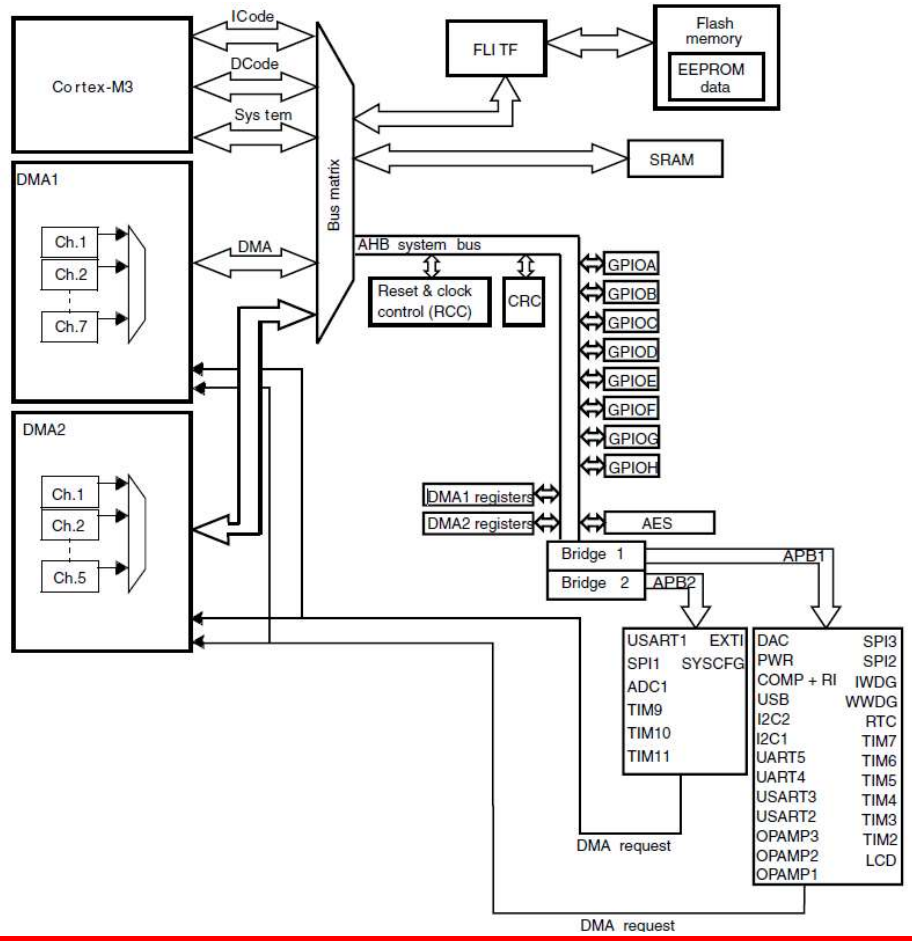


Load and Store Principle



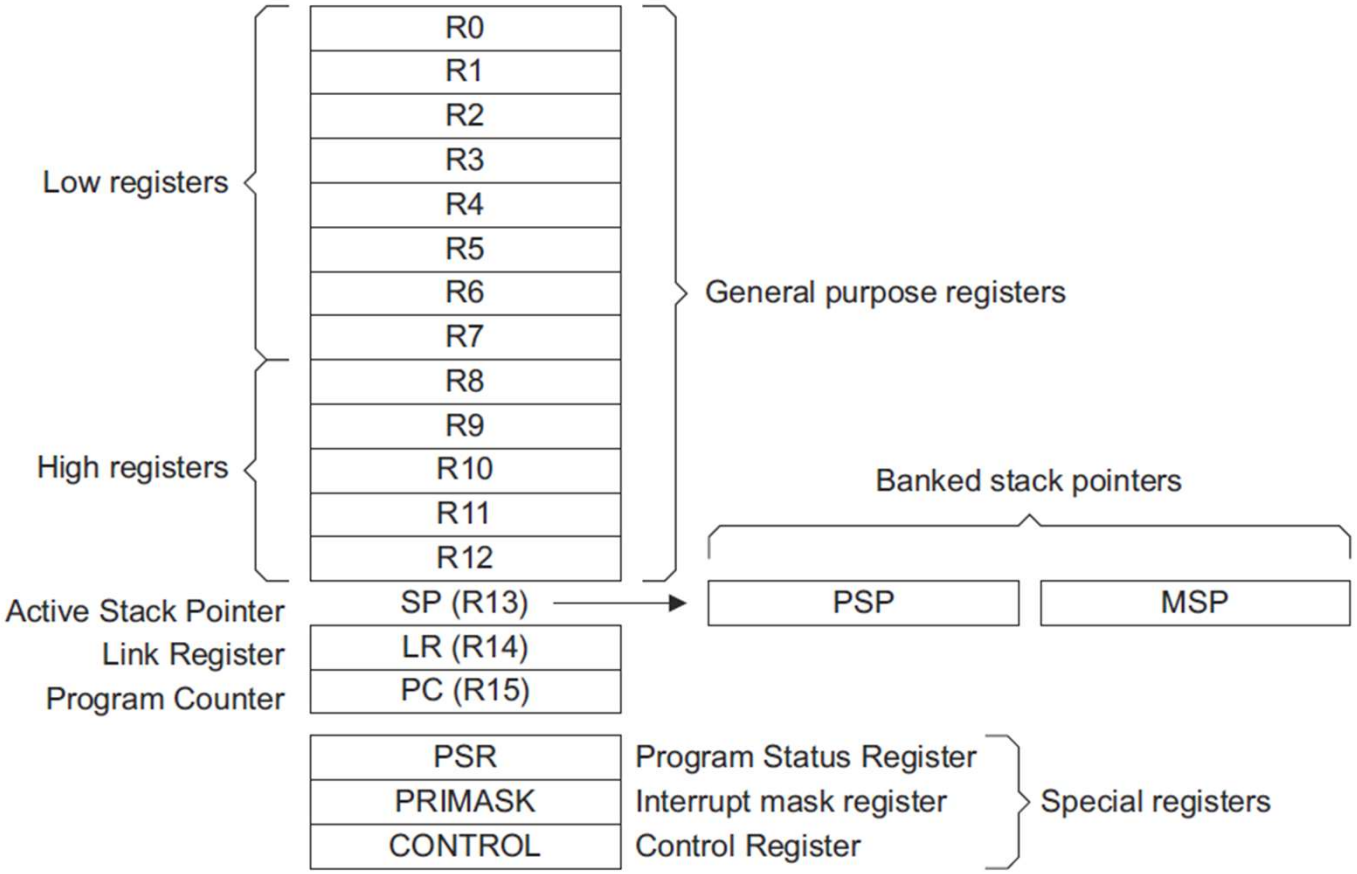
STM32L152 Architecture

Internal Registers and ALU



Memory map data

Cortex M Core Registers

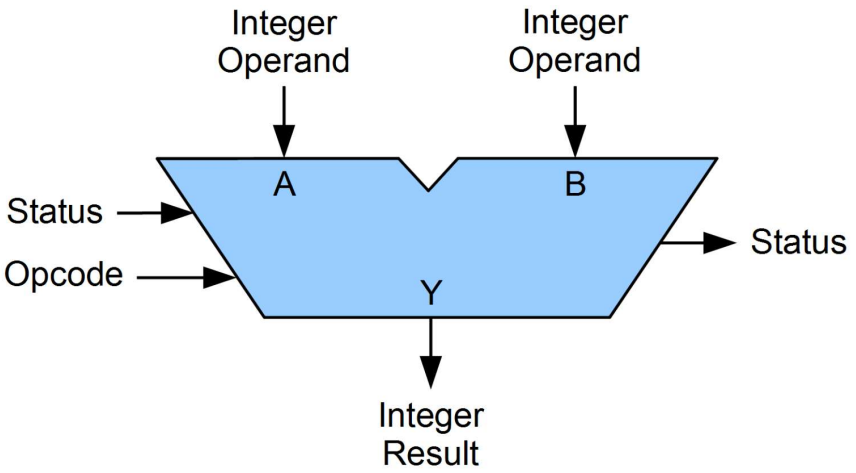


Arithmetic Logic Unit

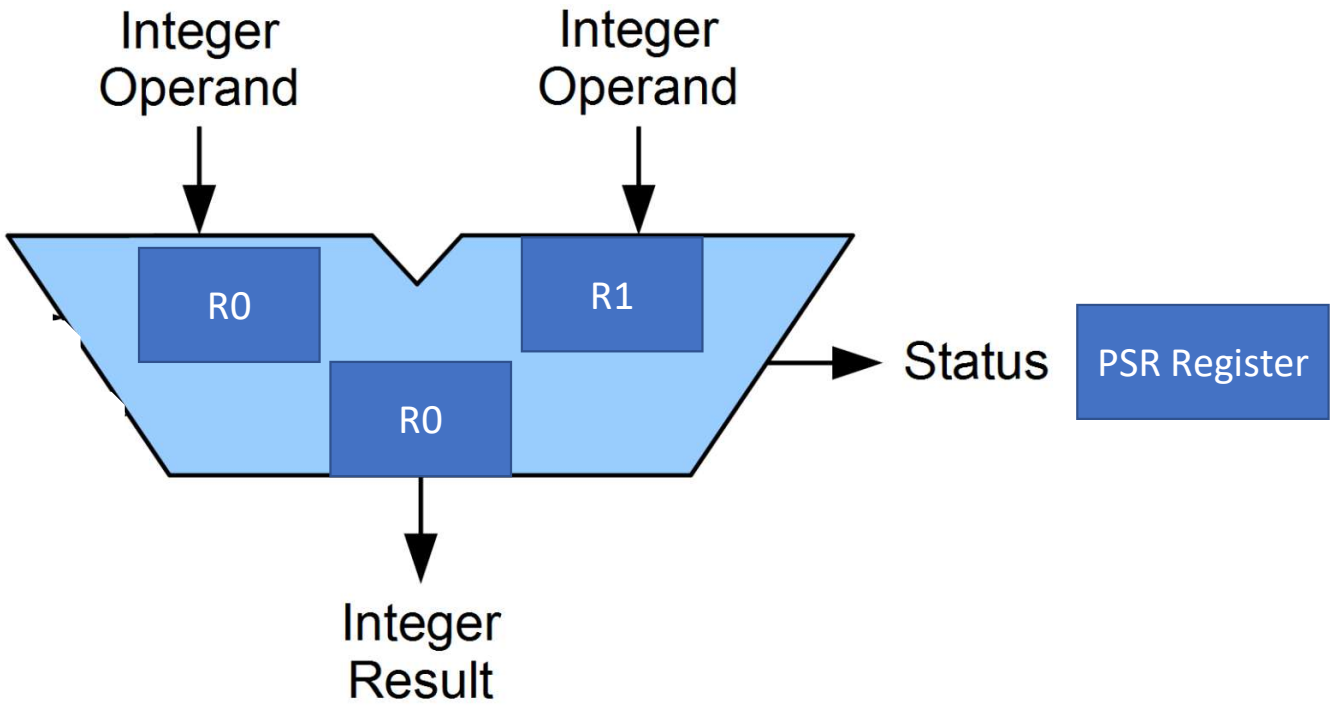
- ✓ An Arithmetic Logic Unit (ALU) is a combinational digital electronic circuit that performs arithmetic and bitwise operations on integer binary numbers
- ✓ ALU is a fundamental building block of a CPU.

Symbolic Representation ALU

- ✓ Input to an ALU are the data to be operated on, called operands
- ✓ A code indicating the operations to be performed (OpCode).
- ✓ ALU's output is the result of the performed operation
- ✓ Status of the previous or current operation



ALU Example : Addition



What is an Instruction?

- ✓ Each operation in the ALU is defined by a series of bytes, called an instruction
- ✓ An instruction has a length 16 or 32 bit on the Cortex M
- ✓ An instruction includes:
  - The operation code
  - Localisation of operands and result
- ✓ Details of the instruction set are written in the Programming Manual

Op-Code	OPER 1	OPER 2	RES

Load Operation

- ✓ A load operation allows to load a destination byte with a source byte
- ✓ It is called an instruction and in assembly language is referred as LD
- ✓ To copy data from a source to a destination, the source and the destinations must be defined
- ✓ The addressing modes give various ways to define the destination and the source

MOVS Instruction

- ✓ The programming manual explains the instruction set (syntax and examples are given)
- ✓ The MOVS instruction can store an immediate value in the range 0-65535 to a destination register

3.5.6 MOV and MVN

Move and move NOT.

Syntax

```
MOV{S}{cond} Rd, Operand2
MOV{cond} Rd, #imm16
```

Example

```
MOVSR11, #0x000B; write value of 0x000B to R11, flags get updated
MOVR1, #0xFA05; write value of 0xFA05 to R1, flags are not updated
MOVSR10, R12; write value in R12 to R10, flags get updated
MOVR3, #23; write value of 23 to R3
MOVR8, SP; write value of stack pointer to R8
MVNSR2, #0xF; write value of 0xFFFFFFF0 (bitwise inverse of 0xF)
    ; to the R2 and update flags
```

MOVS Instruction Encoding

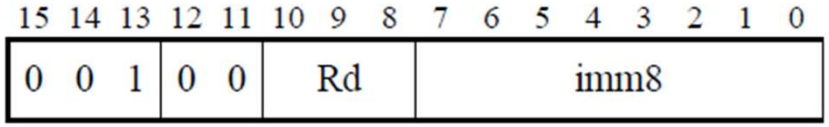
The ARMv7-M Architecture Reference Manual details the encoding of each instruction

A6.7.75 MOV (immediate)

Move (immediate) writes an immediate value to the destination register. It can optionally update the condition flags based on the value.

Encoding T1 All versions of the Thumb ISA.

`MOVS <Rd>,#<imm8>`	Outside IT block.
`MOV<c> <Rd>,#<imm8>`	Inside IT block.




```
d = UInt(Rd);  setflags = !InITBlock();  imm32 = ZeroExtend(imm8, 32);  carry = APSR.C;
```

MOV Instruction Encoding Example

With the example given in the Programming Manual

```
movs r3, #0xFA
```

The encoding must be as follow

- Rd = 3 = b011 = 0x3
- Imm8 = 0xFA = b1111 1010

The instruction is then coded as b0010 0011 1111 1010 = 0x23FA

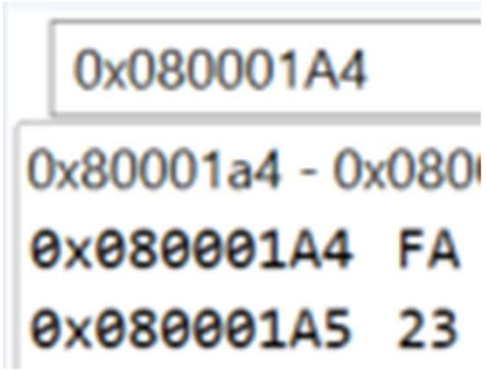
MOVs Instruction Encoding Example

In the disassembly window, we can check the compiler encoded as 0xFA23

```
127                          movs r3, #0xFA
080001a4: 0x0000fa23           movs    r3, #250          ; 0xfa
```

In the memory window:

- Address 0x8000 01A4 : 0xFA
- Address 0x8000 01A6 : 0x23



Instruction Sets

- ✓ Instruction sets depend on the microcontroller (even if basic instructions are present in all microcontrollers)
- ✓ When an instruction is not available, 2 options:
  - Find a microcontroller having this instruction
  - Program this instruction
- ✓ 8-bit microcontrollers have no instruction to add float numbers and must be done by software
- ✓ Cortex M0 has only basic instructions unlike Cortex M4 that can perform FPU (Floating Point Unit) instructions

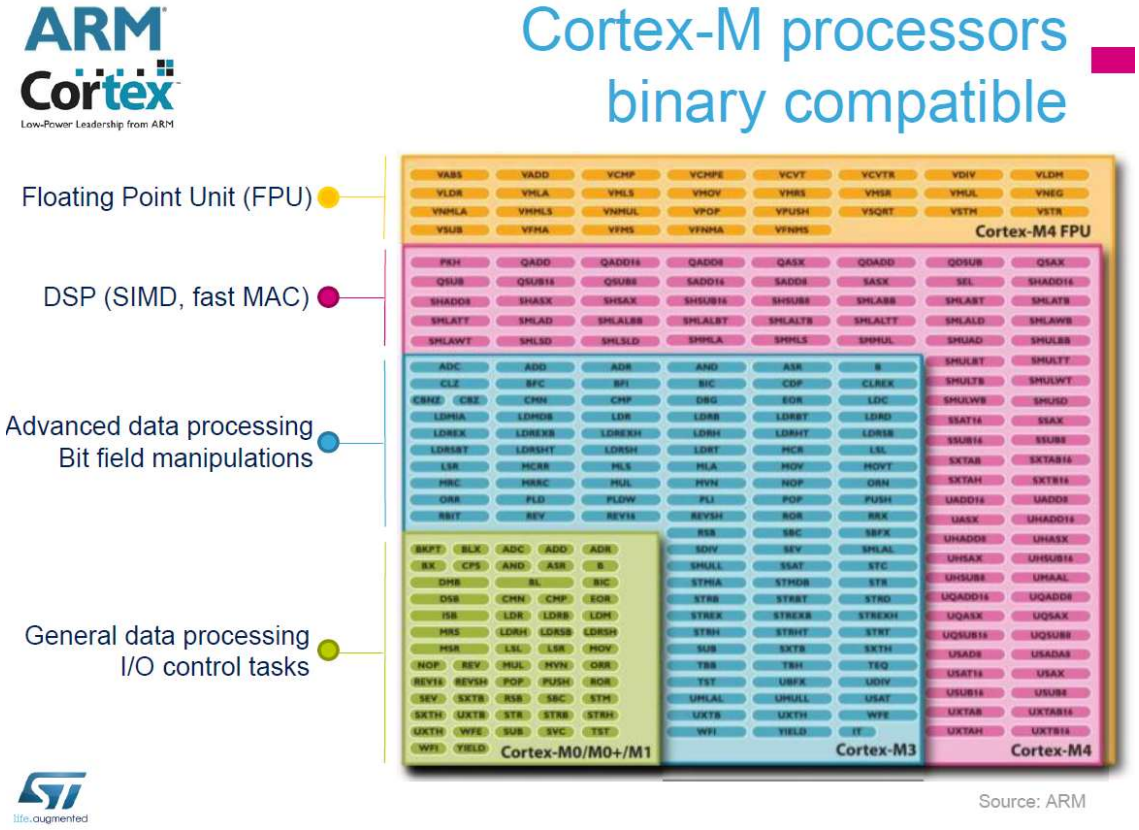
ARM Instructions

2 instruction sets

- ✓ ARM instruction set would allow code to be written for maximum performance as they are 32-bit instructions
- ✓ Thumb code would achieve a greater code density as it is 16-bit instructions
- ✓ Cortex-M processors are code compatible with the original Thumb instruction set, but they are designed to execute an extended version of the Thumb instruction set called Thumb-2.
- ✓ Thumb-2 is a blend of 16 and 32 bit instructions that has been designed to be very C friendly and efficient.

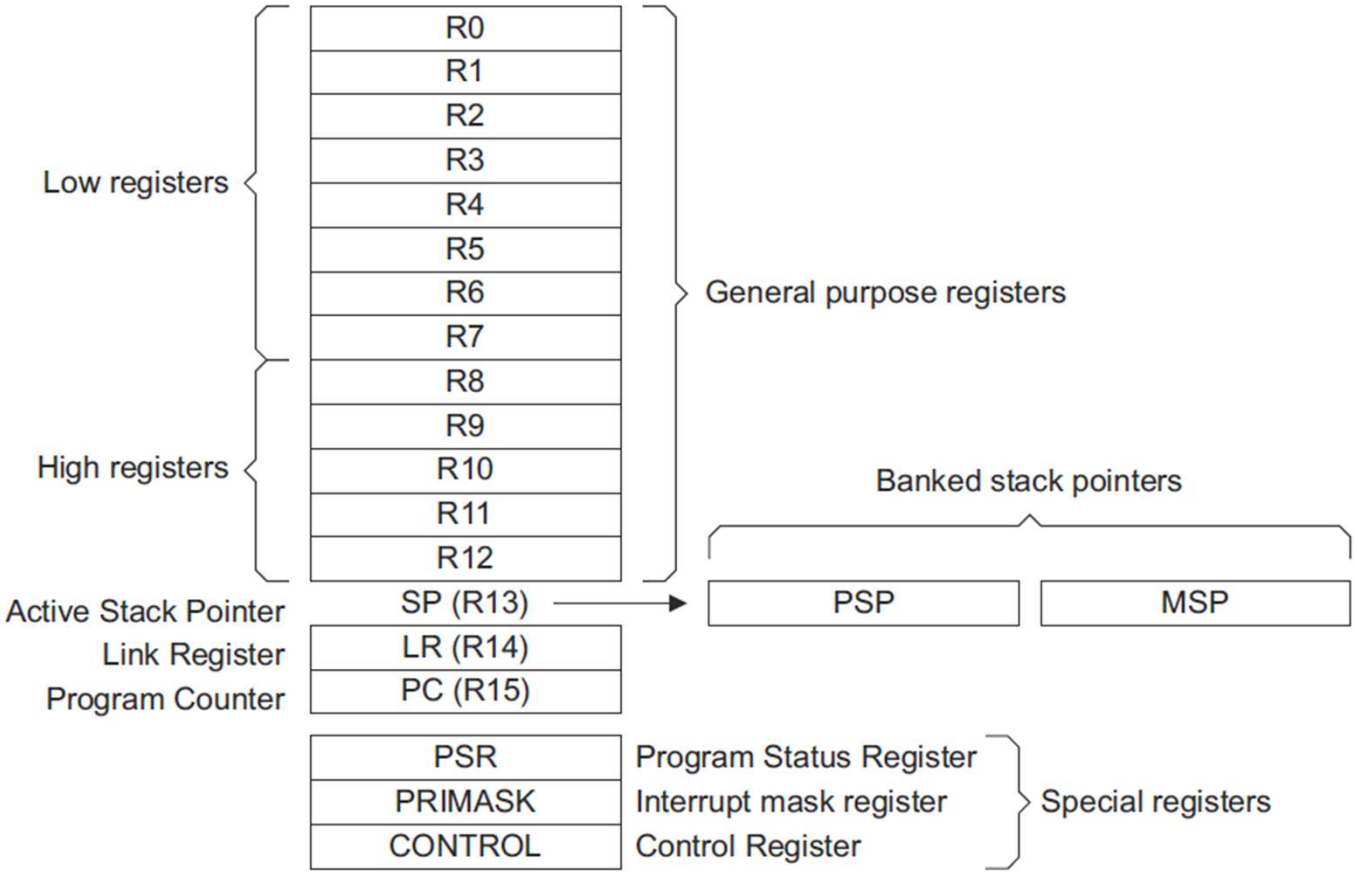
Cortex M supports only Thumb Instructions

STM32 Instruction Sets



The chosen instruction set depends on math requirements

STM32L476 Core Registers

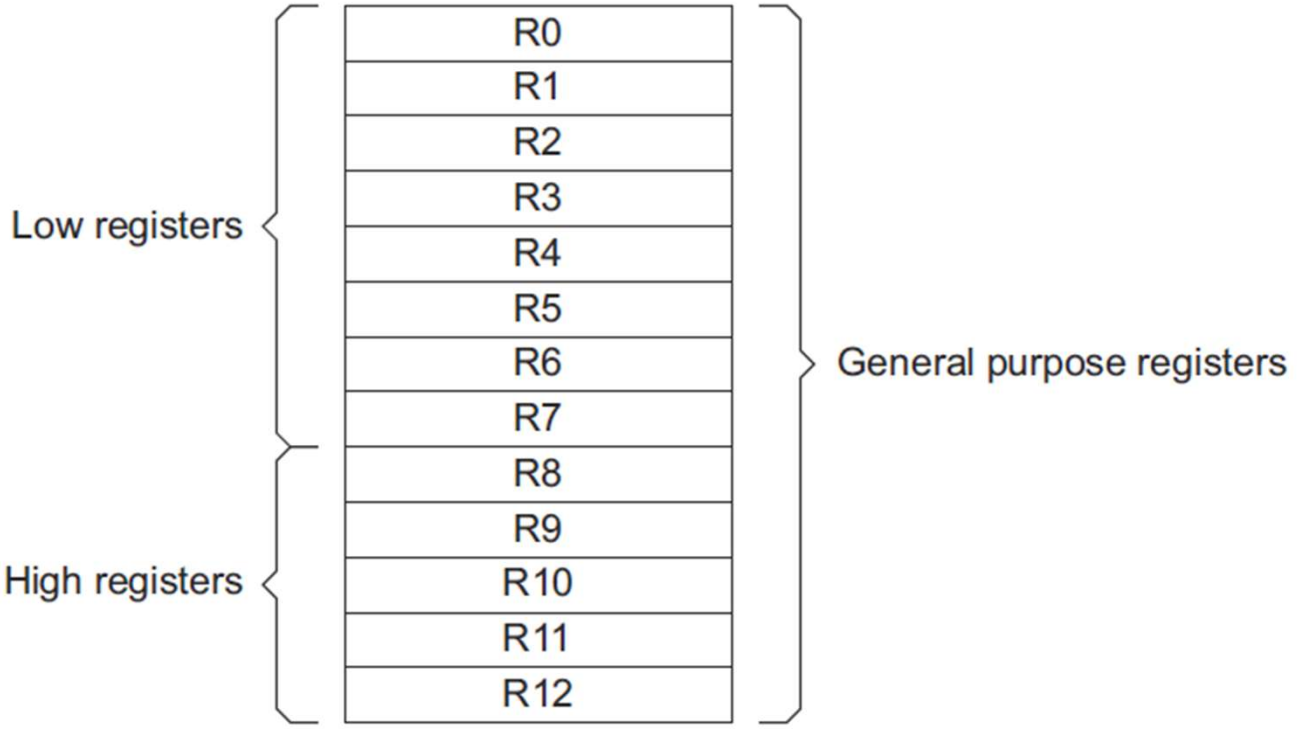


Internal registers cannot be accessed by address pointers

STM32L476 General Purpose Registers

The general purpose registers are the input of the AUL and the output (where the result of the operation is stored)

It is not possible to modify the value of a variable directly in the memory (load and store principle)



Steps to Increment a Value in Memory

1) Get the address where the variable is stored in memory
2) Retrieve the value in memory by pointing the address and store it in a register
3) Increment the value stored in the register
4) Store this new value in memory

The necessary assembly functions are LDR, ADD and STR

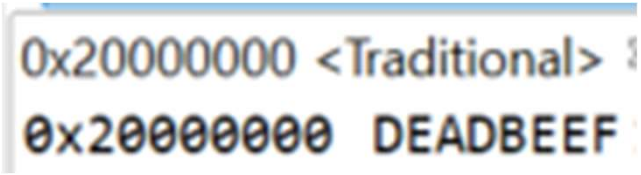
LDR instruction 1/2

Gcc code to declare a variable in the data section

```
.section .data

varindata:
    .word 0xDEADBEEF
```

1) At address 0x20000000 the value 0xDEADBEEF is stored



2) The assembly ldr instruction retrieves the address of the variable varindata

```
ldr r4, =varindata
```

3) Register r4 stores the address of varindata which is equal to 0x20000000



*0x20000000 is the beginning of the SRAM section that stores initialized variables

LDR instruction 2/2

```
ldr r5, [r4]
```

```
0x20000000 <Traditional>
0x20000000 DEADBEEF
```

[r4] means to point to the adress stored in register r4

r4	0x20000000 (Hex)
r5	0xdeadbeef (Hex)

value stored at address 0x20000000 is 0xDEADBEEF -> R5 = 0xDEADBEEF

ADD instruction

```
add r5,#1
```



r4	0x20000000 (Hex)
r5	0xdeadbeef (Hex)

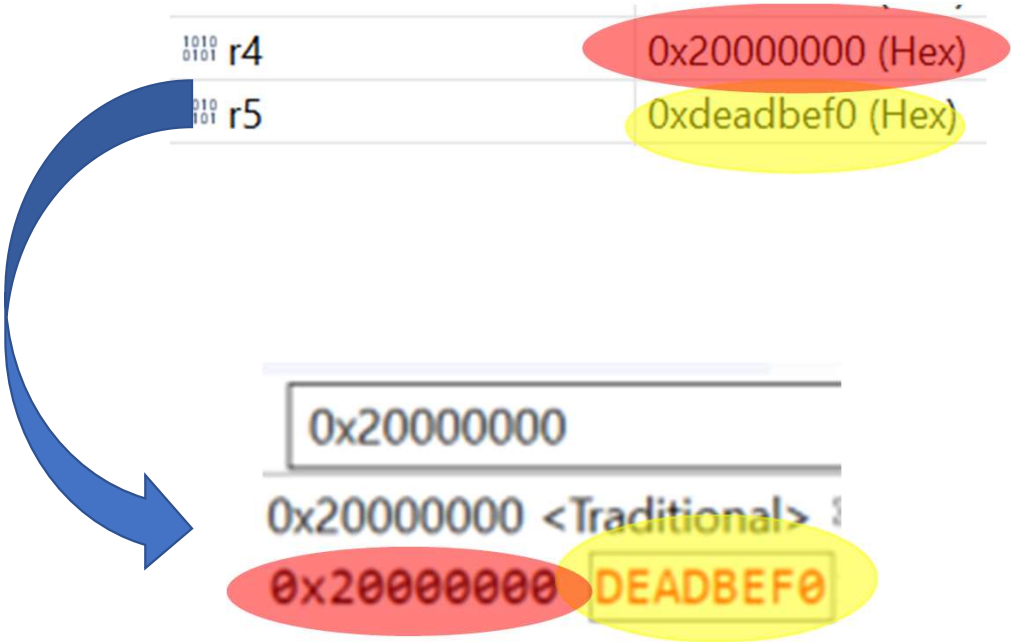
r4	0x20000000 (Hex)
r5	0xdeadbef0 (Hex)

The value in memory is still 0xDEADBEEF

```
0x20000000 <Traditional>
0x20000000 DEADBEEF
```

STR instruction

```
str r5, [r4]
```



[r4] means to point to the adress stored in register r4

value stored at address 0x20000000 becomes 0xDEADBEF0

Summary Increment Variable

```
ldr r4, =varindata
ldr r5, [r4]
add r5,#1
str r5, [r4]
```

4 instructions are necessary to increment the value of a variable stored in memory

NOP Instruction

- ✓ NOP stands for No Operation
- ✓ This instruction just does nothing (sometimes useful the application needs to wait)

PC Calculation

- ✓ PC stands for Program Counter
- ✓ The PC store the address of the next instruction to execute
- ✓ If the running instruction starts at the address AD and occupies 2 bytes, the new PC values is then AD+2 that represents the address of the following instruction.
- ✓ Instruction length in Cortex M (Thumb 2) is a mix od 16 and 32 bits → PC value is always even
- ✓ The PC of the STM32 is a 32-bit register to map 4GB of memory

PC Calculation

Code

```
ldr r4, =varindata
ldr r5, [r4]
add r5,#1
str r5, [r4]
```

Disassembly Window

Adresses of instructions

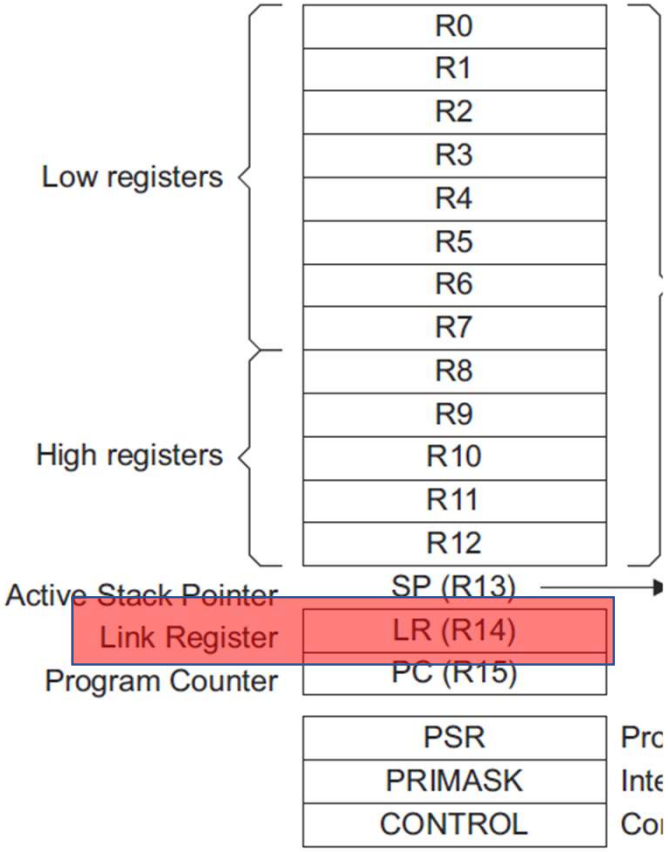
```
120                                 ldr r4, =varindata
08000198: 0x0000404c                  ldr     r4, [pc, #256]
121                                 ldr r5, [r4]
0800019a: 0x00002568                  ldr     r5, [r4, #0]
122                                 adds r5,#1
0800019c: 0x00000135                  adds    r5, #1
123                                 subs r5,#1
0800019e: 0x0000013d                  subs    r5, #1
```

In debug mode, the PC register is updated at each instruction

pc	0x8000198 <Reset_Handler+...
pc	0x800019a <Reset_Handler+...
pc	0x800019c <Reset_Handler+...
pc	0x800019e <Reset_Handler+...

Link Register

- ✓ The Link Register stores the return information for subroutines, function calls and exceptions. (On reset, the LR value is unknown)
- ✓ When there is a jump in the program to execute some code and must return at the jump level, it is necessary to retrieve the address where the jump occurred. This information is stored thanks to the Link Register



Link Register Example

1. At line 171, the program must jump to the label dosomething which is at the address 0x08000 01E6
2. Instruction movs r0,#0xAB is then executed
3. Instruction bx lr make a jump to instruction movs r1,#5

```
171 bl dosomething
172 movs r1, #5
173 dosomething:
174     movs r0,#0xAB
175     bx lr
```

```
171                     bl dosomething
080001e0: 0x00f001f8      bl      0x80001e6
172                     movs r1, #5
080001e4: 0x00000521      movs    r1, #5
174                         movs r0,#0xAB
080001e6: 0x0000ab20      movs    r0, #171
175                         bx lr
080001e8: 0x00007047      bx      lr
```

Let's see in Details the values of PC and LR registers during the execution of this program…

Link Register Code Execution (1/2)

1) The next instruction is 'bl dosomething' at address 0x0800 01FE, hence PC = 0x0800 01E0

```
245                     bl dosomething
080001fe: 0x00f001f8    bl      0x8000204
246                     movs r1, #5
08000202: 0x00000521      movs    r1, #5
248                         movs r0,#0xAB
08000204: 0x0000ab20      movs    r0, #171
249                         bx lr
08000206: 0x00007047      bx      lr
```

lr	0x8000331 (Hex)
pc	0x80001fe <Reset_...

2) The program jumps at the instruction movs, r0,#0xAB that is at address 0x8000 204. LR = 0x8000 0203 to point to the instruction following 'bl dosomething'

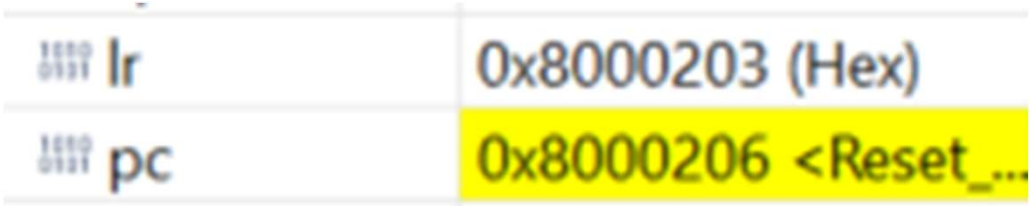
```
245                     bl dosomething
080001fe: 0x00f001f8    bl      0x8000204
246                     movs r1, #5
08000202: 0x00000521      movs    r1, #5
248                         movs r0,#0xAB
08000204: 0x0000ab20      movs    r0, #171
249                         bx lr
08000206: 0x00007047      bx      lr
```

lr	0x8000203 (Hex)
pc	0x8000204 <Reset_...

Link Register Code Execution (2/2)

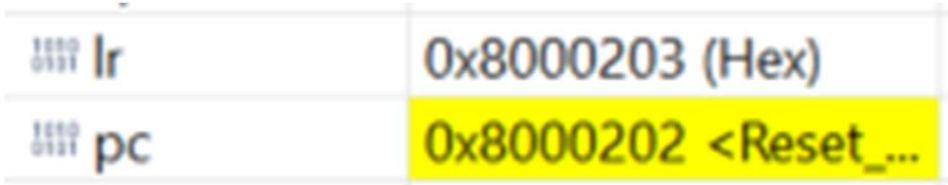
3) The next instruction is ‘bx lr’ at address 0x0800 0206, hence PC = 0x0800 0206

```
245                     bl dosomething
080001fe: 0x00f001f8      bl      0x8000204
246                     movs r1, #5
08000202: 0x00000521      movs    r1, #5
248                       movs r0,#0xAB
08000204: 0x0000ab20      movs    r0, #171
249                       bx lr
08000206: 0x00007047      bx      lr
```



4) The program jumps at the instruction movs, r1,#5 that is at address 0x8000 0202. lr does not change

```
245                     bl dosomething
080001fe: 0x00f001f8      bl      0x8000204
246                     movs r1, #5
08000202: 0x00000521      movs    r1, #5
248                       movs r0,#0xAB
08000204: 0x0000ab20      movs    r0, #171
249                       bx lr
08000206: 0x00007047      bx      lr
```



LSB of lr

- ✓ When lr = 0x0800 01E5, PC becomes 0x0800 01E4 while we could expect 0x0800 01E5
- ✓ PC always has an even value, this means the lsb is always equal to 0
- ✓ ARM processors can run with thumb instructions or ARM instructions. This information is given by the lsb of lr.
- ✓ When lr value is copied to pc register, if lsb of lr is equal to 1 this informs the processor that the next instructions is a thumb instructions
- ✓ As Cortex M supports only thumb instructions, lsb of lr is equal to 1

Branch Instructions Summary

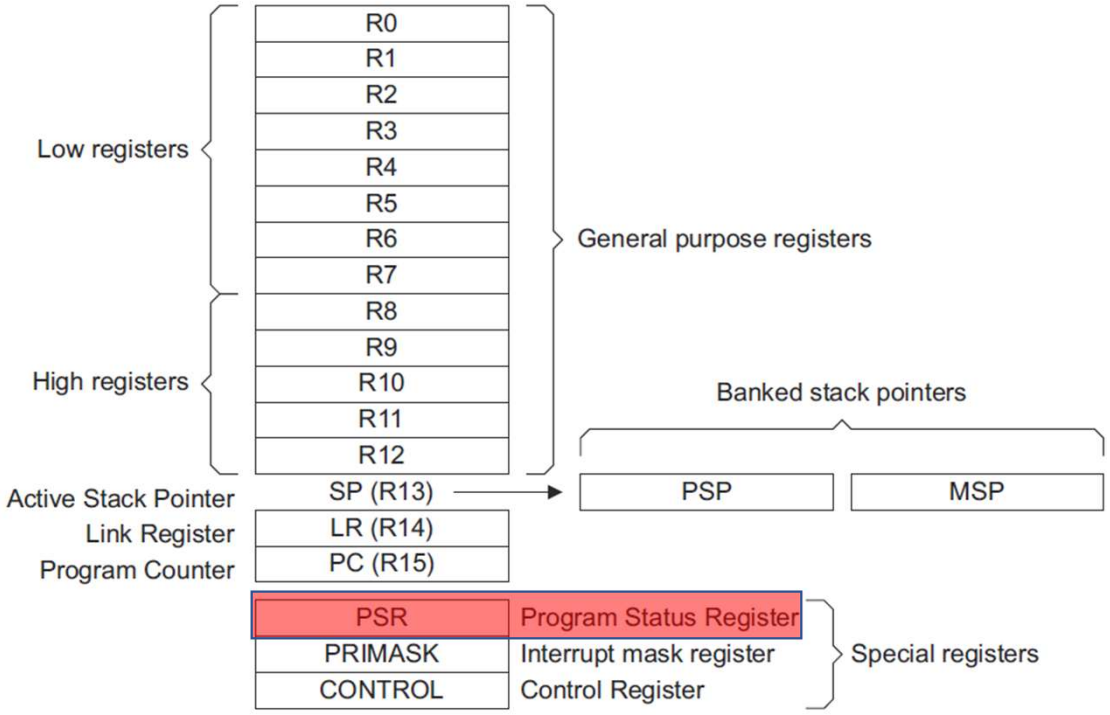
- ✓ B address : go to the address
- ✓ B<c> address : go to the address with conditions (depends on APSR register)
- ✓ BL address : save return address in lr (link register) and go to the address
- ✓ BX register : go to the address pointed in the register (often used with lr)

Conditonal Execution

EQ	Equal	Z == 1
NE	Not equal	Z == 0
CS/HS	Carry set/unsigned higher or same	C == 1
CC/LO	Carry clear/unsigned lower	C == 0
MI	Minus/negative	N == 1
PL	Plus/positive or zero	N == 0
VS	Overflow	V == 1
VC	No overflow	V == 0
HI	Unsigned higher	C == 1 and Z == 0
LS	Unsigned lower or same	C == 0 or Z == 1
GE	Signed greater than or equal	N == V
LT	Signed less than	N != V
GT	Signed greater than	Z == 0 and N == V
LE	Signed less than or equal	Z == 1 or N != V

Program Status Register (PSR)

- ✓ The Program Status Register allows to record information about the result of an operation (addition, substraction, comparison, test…)
- ✓ It allows to know if an operation comes up with a 0, negative, carry or overflow result
- ✓ During an interrupt, the interrupt number is recorded



PSR Register Contents

APSR	N	Z	C	V	Q											
IPSR												Exception Number				
EPSR						ICI/IT	T		ICI/IT							



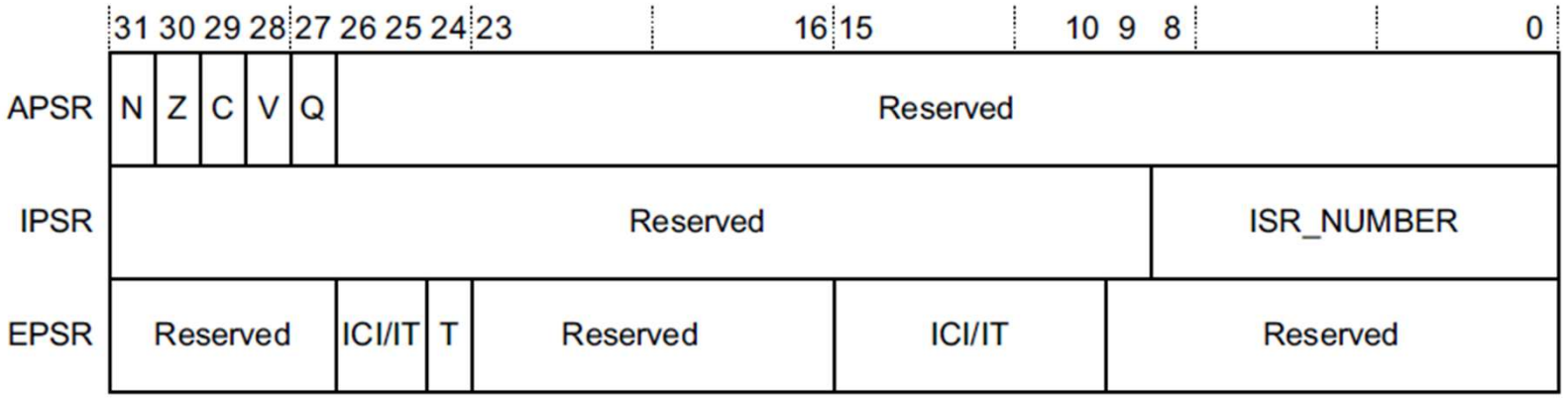
PSR	N	Z	C	V	Q	ICI/IT	T		ICI/IT	Exeption Number

- ✓ In a normal application program, the code does not make explicit access to the xPSR or any of its alias registers
- ✓ Any use of the xPSR is made by compiler generated code.
- ✓ But as a programmer it is mandatory to have an awareness of the xPSR and the flags contained in it.

xPSR Register Contents

- ✓ The most significant four bits of the xPSR are the condition code bits
  - Negative
  - Zero
  - Carry
  - oVerflow
- ✓ These are set and cleared depending on the results of a data processing instruction.
  - SUB R8,R6 perform a subtraction and do not update the condition code flags
  - SUBS R8,R6 perform a subtraction and update the conditions code flags
- ✓ This allows the compiler to perform an instruction set that updates the condition code flags, then perform instructions that do not modify the flags and then perform a conditional branch on the state of the xPSR condition codes.

PSR in Details



The PSR register is split in 3 sub-registers:

- ✓ Application Program Status Register (negative or null result….)
- ✓ Interrupt Program Status Register : interrupt number
- ✓ Execution Program Status Register: T = 1 in Cortex M processors as only thumb instructions are supported

PSR Application

```
varindata:
    .word 0xFFFFFFFF

ldr r4, =varindata
ldr r5, [r4]
adds r5,#1
subs r5,#1
str r5, [r4]
```

```
Name : xpsr
    Hex:0x61000000
```

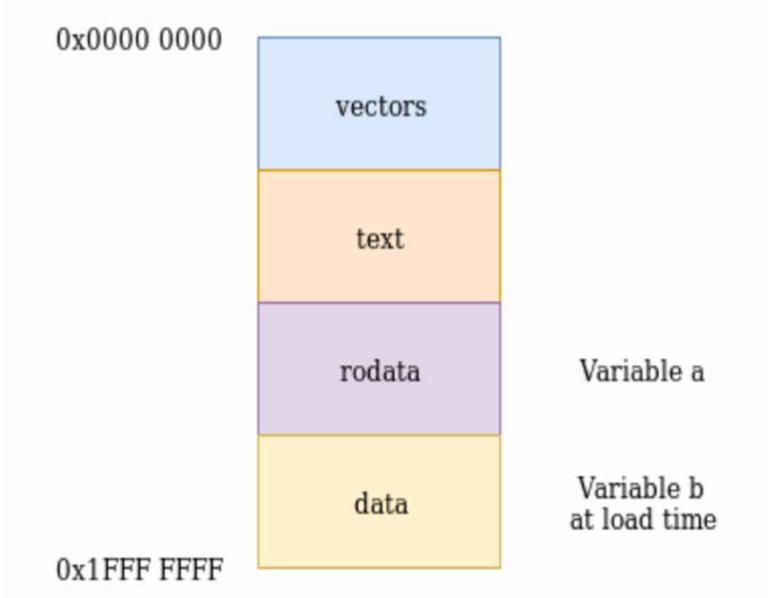
Result is zero with carry

```
Name : xpsr
    Hex:0x81000000
```

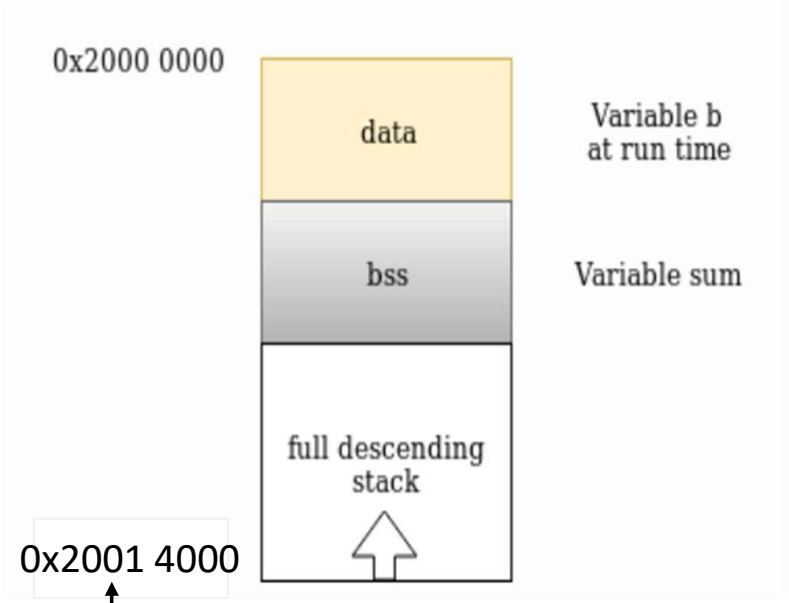
Result is negative

Memory Organization

Flash Memory

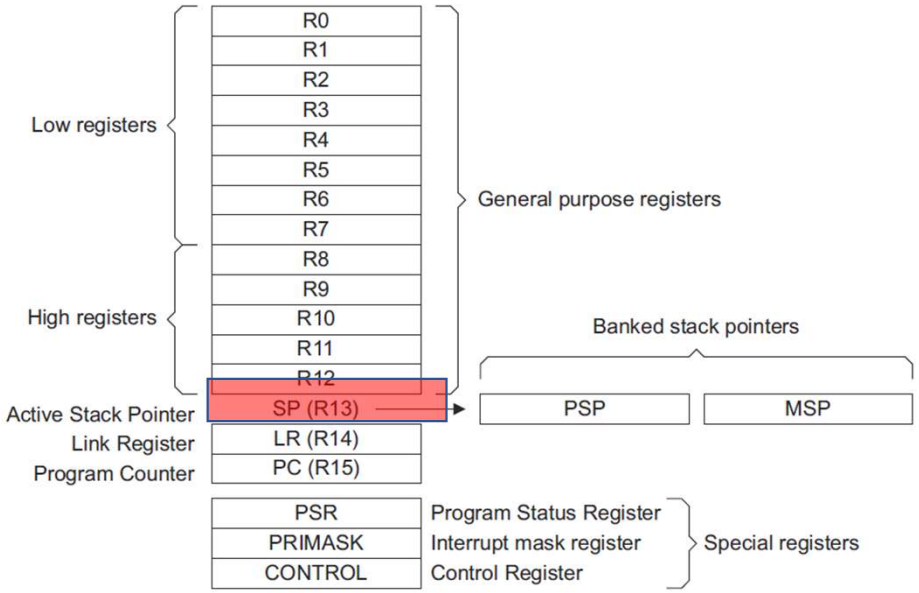


SRAM Memory



STM32L152 has 80KBytes SRAM. 80*1024 = 0x1 4000 = 81 920

Stack Pointer



A special memory area: the stack

- ✓ Area in the RAM
- ✓ LIFO (Last In First Out) type operation (Vs FIFO)
- ✓ Stores data temporarily
- ✓ Allows to save the context (PC, …) when a routine or an interrupt is called

→ Need a counter that always points to the next available cell (byte) in the stack

→ The stack pointer, decreased after stacking, incremented before unstacking

Stack Initialisation

sp	0x20014000

- ✓ At power up, the SP on the top of the SRAM
- ✓ '??' means it is no possible to read data. In fact, there is no silicon(transistors) as the STM32L152 has 80KBytes of SRAM and SRAM region starts at 0x2000 0000
- ✓ Values in SRAM are random and can depend on previous programs that run on the microcontroller
- ✓ As SP = 0x2001 4000, the next available memory space is at 0x2001 3FFF



```
0x20013ffb - 0x20014
0x20013FFB 00  .
0x20013FFC 97  .
0x20013FFD 01  .
0x20013FFE 00  .
0x20013FFF 08  .
0x20014000 ??  ?
0x20014001 ??  ?
0x20014002 ??  ?
0x20014003 ??  ?
```

Stack Program Example

```
/* met les valeurs dans la pile *,
movs r0, #0xBB
push {r0, lr}
movs r0, #0xAA
push {r0, lr}

pop {r1, r2, r3}
```

This program performs the following actions:

1) Load 0xBB in General Register r0
2) Push values in r0 and lr in the stack
3) Load 0xAA in the stack
4) Retrieve the 3 values of the stack and stores them in the three general registers r1, r2, r3

Stack Program Example Execution (1/2)

Before the push instruction

sp	0x20014000
lr	0x8000311 (Hex)
r0	0xbb (Hex)

0x20013FFA	00
0x20013FFB	00
0x20013FFC	97
0x20013FFD	01
0x20013FFE	00
0x20013FFF	08
0x20014000	??
0x20014001	??

After the push instruction

r0	0xbb (Hex)
sp	0x20013ff8
lr	0x8000311 (Hex)

0x20013FF8	BB
0x20013FF9	00
0x20013FFA	00
0x20013FFB	00
0x20013FFC	11
0x20013FFD	03
0x20013FFE	00
0x20013FFF	08
0x20014000	??

Stack Program Example (2/2)

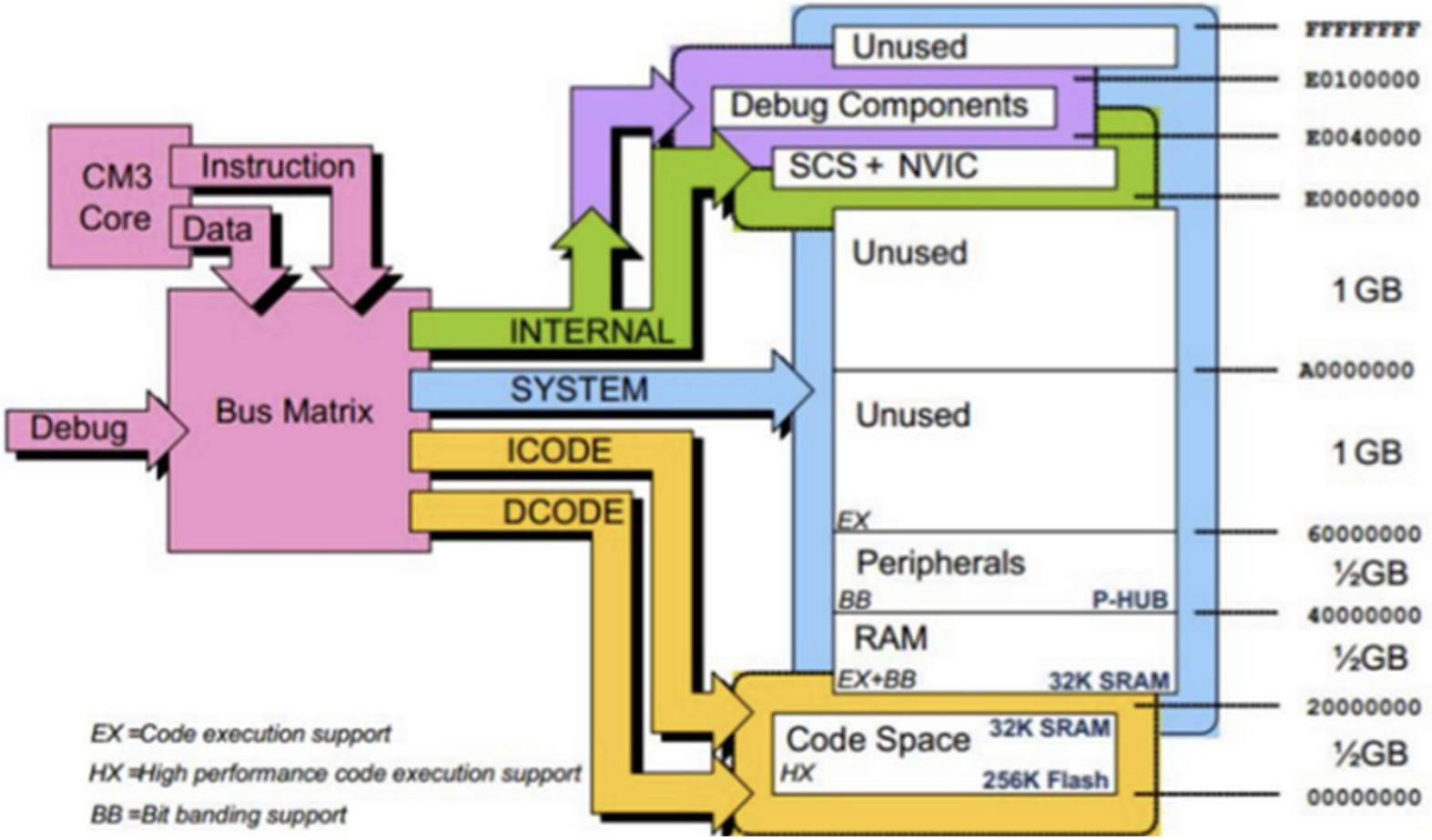
After the Pop instruction

r1	0xaa (Hex)
r2	0x8000311 (Hex)
r3	0xbb (Hex)
sp	0x20013ffc
lr	0x8000311 (Hex)

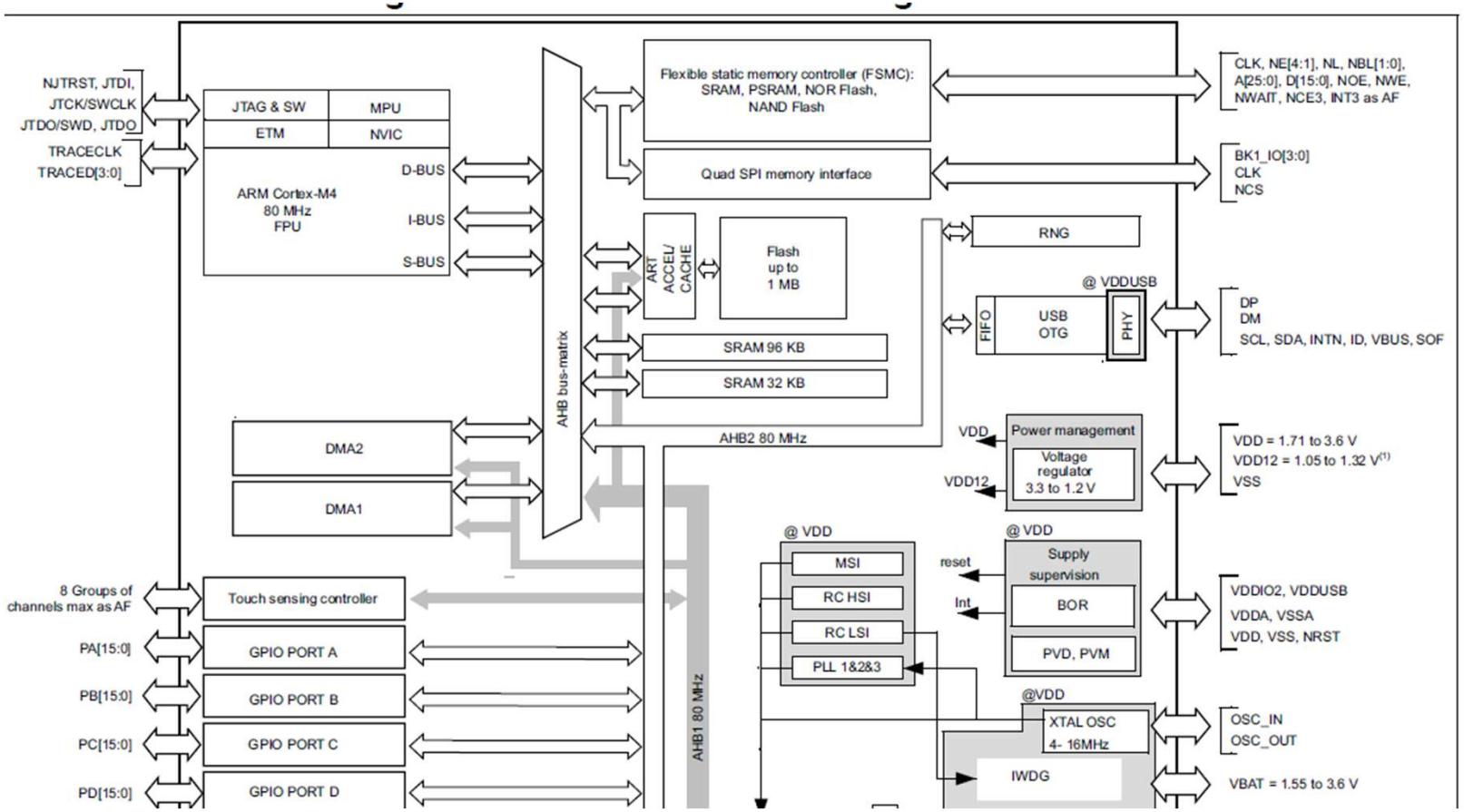
```
0x20013FF7 08
0x20013FF8 BB
0x20013FF9 00
0x20013FFA 00
0x20013FFB 00
0x20013FFC 11
0x20013FFD 03
0x20013FFE 00
0x20013FFF 08
0x20014000 ??
0x20014001 ??
0x20014002 ??
```

Data is not erased but stack pointer allows to write on this memory space

Cortex M internal Buses

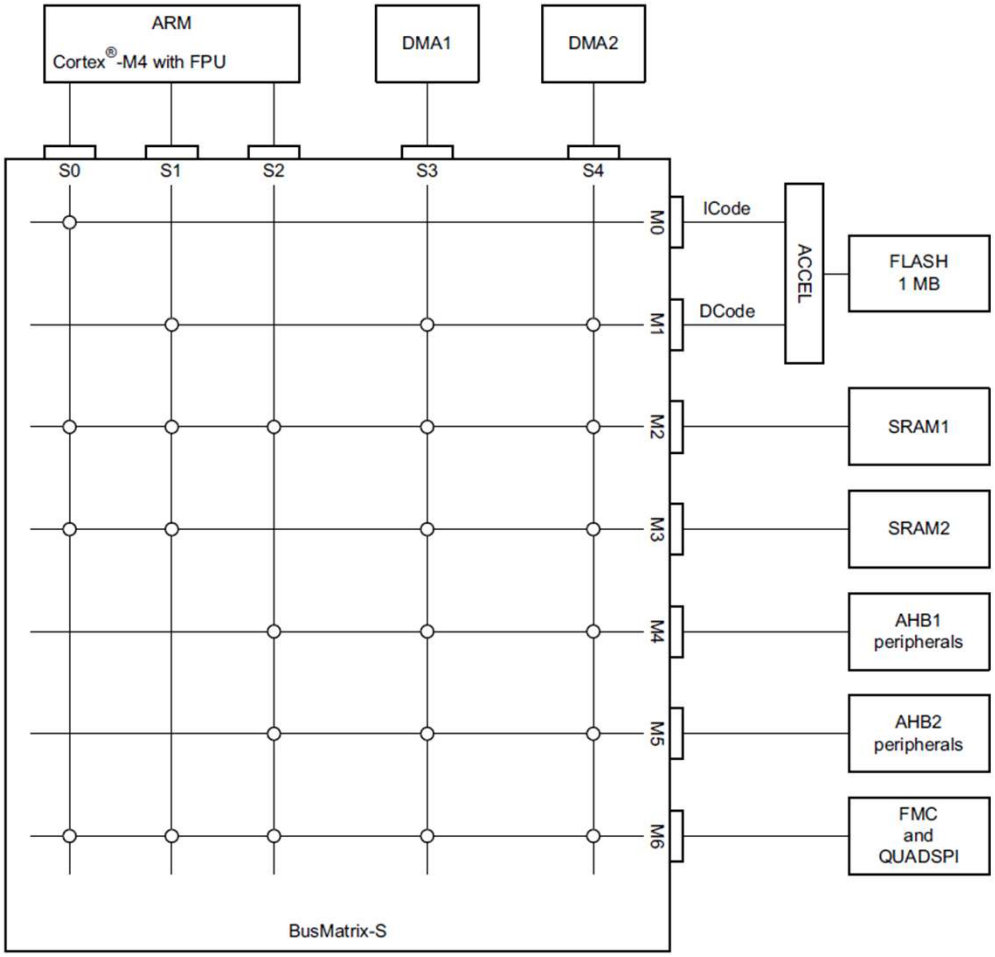


STM32L476 Schematic



*See Full Schemaric in Datasheet

Rôle of Internal Buses

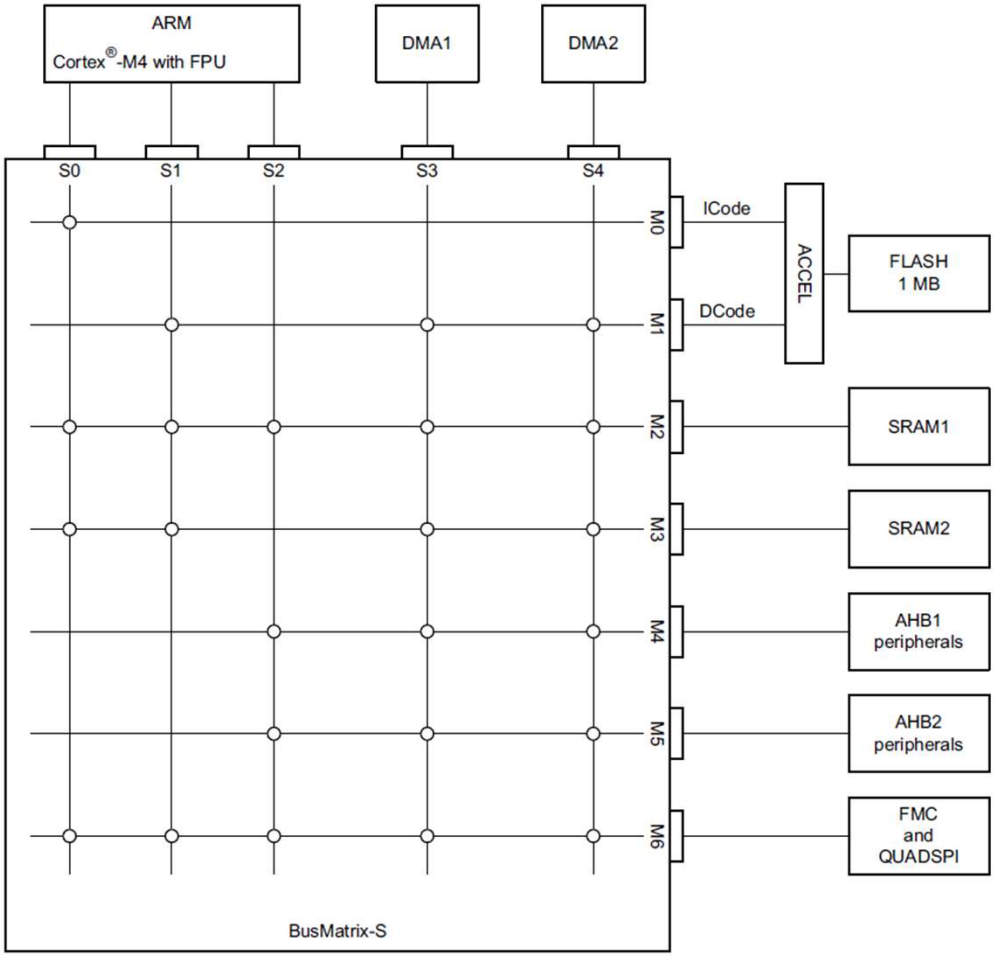


I-Bus: Instruction Bus

D-Bus: data bus

S-Bus: System bus

STM32L476 Bus Matrix



Security features inside in the microcontroller monitor and block data going through these buses

Memory Sections

```
MEMORY
{
  RAM     (xrw)     : ORIGIN = 0x20000000,   LENGTH = 96K
  RAM2    (xrw)     : ORIGIN = 0x10000000,   LENGTH = 32K
  FLASH   (rx)      : ORIGIN = 0x8000000,    LENGTH = 1008K
  SecureFlash (rx)  : ORIGIN = 0x080FC004,   LENGTH = 16K
  SecureSRAM (xrw)  : ORIGIN = 0x2000FE00,   LENGTH = 512
}
```

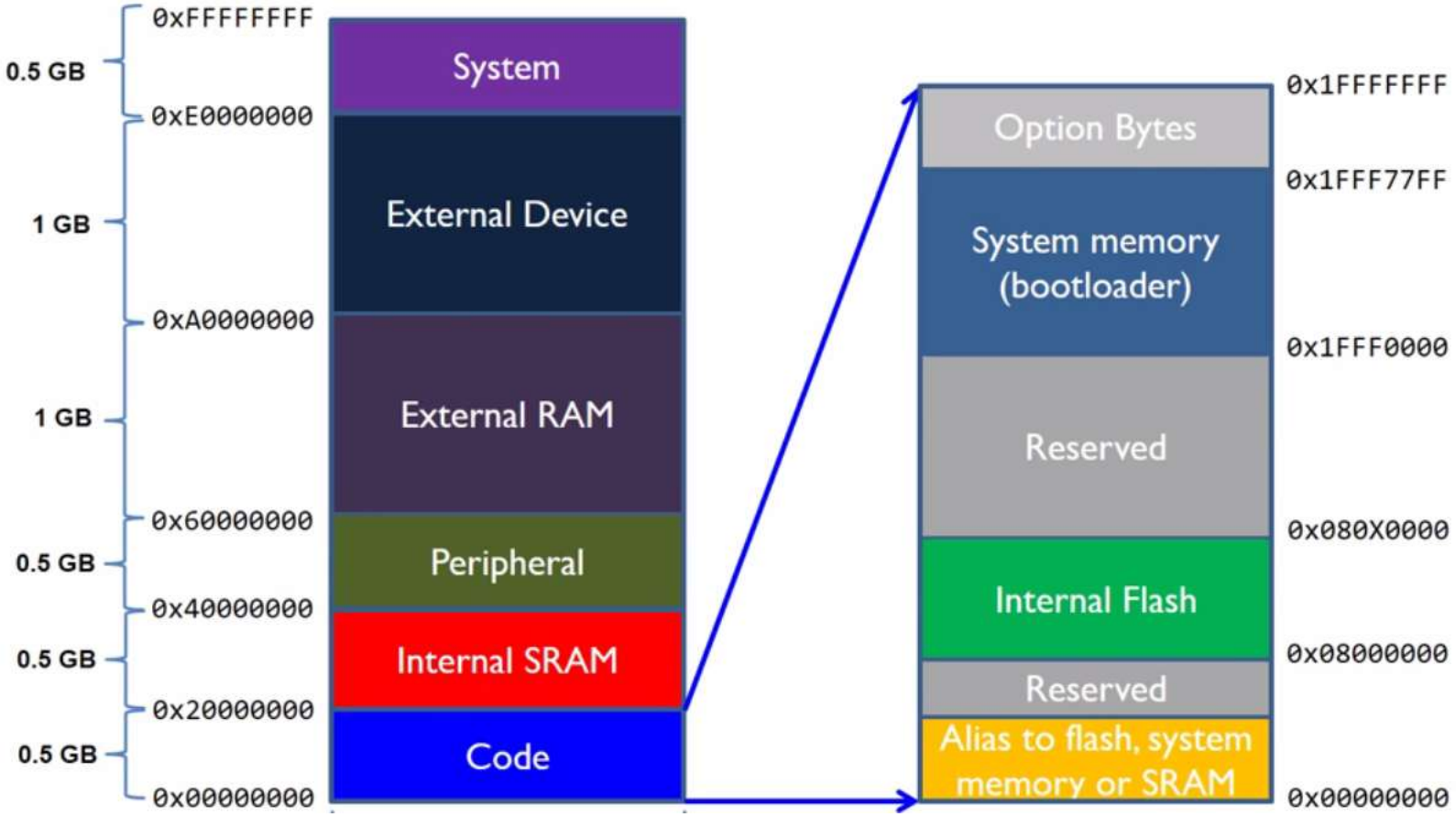
In the linker script, each memory area is configured as read, write, or execute. Depending on the configuration, a different bus is used.

Memory Areas

To secure a microcontroller, it is necessary to know where information passes through

- From memory to memory
- Between peripheral and memory
- Between core registers and memory
- External communication
- …

STM32 Memory Organization



!!! addresses are different depending on components

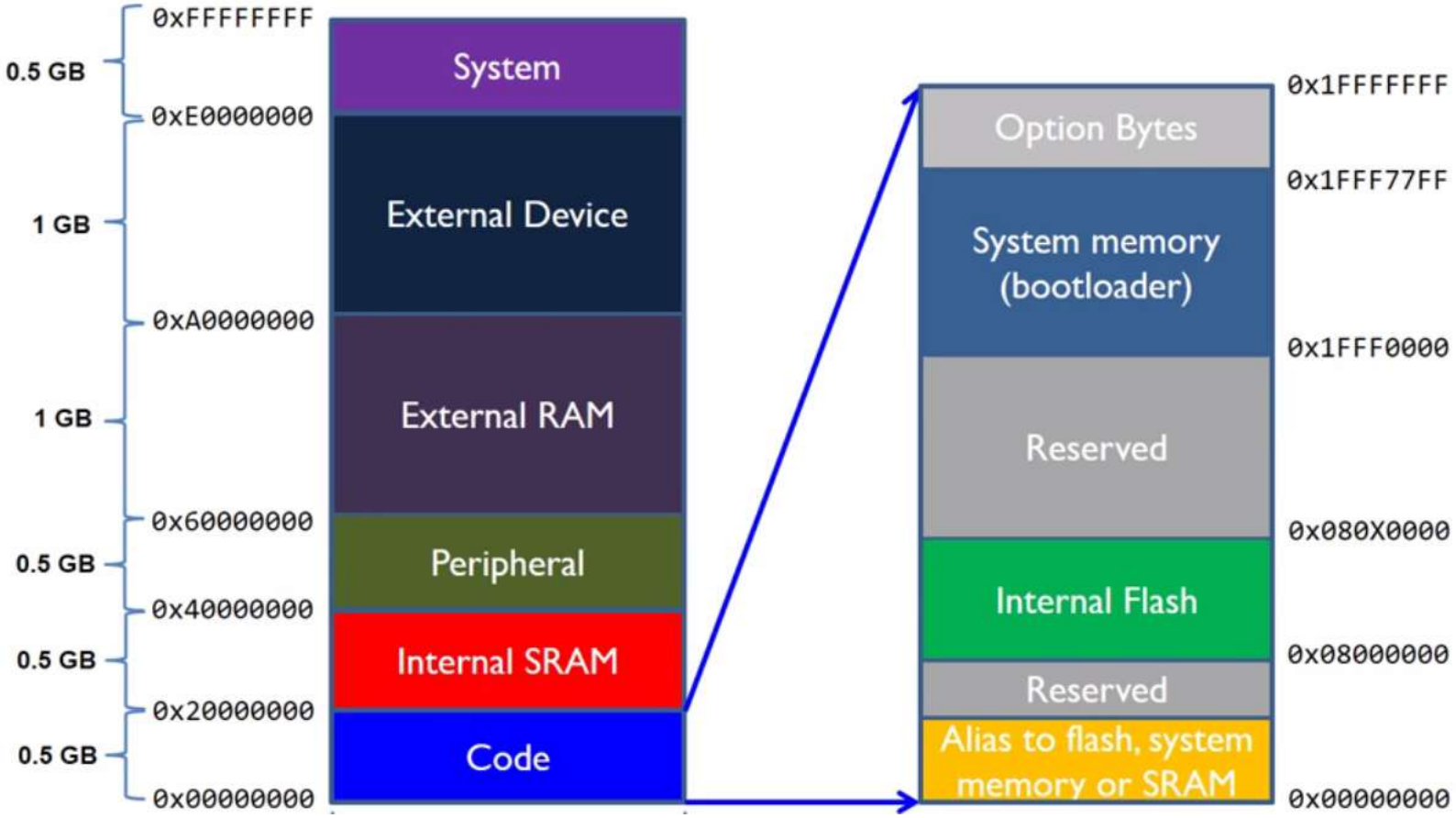
Flash Memory in STM32L476

The STM32L476xx devices feature up to 1 MB of embedded Flash memory available for storing programs and data. The Flash memory is divided into two banks that allow read-while-write operations. This feature allows a read operation to be performed from one bank while an erase or program operation is performed on the other bank.

Allows you to update programs

→ Possibility of attack when downloading a new program

Option Bytes



Option Bytes

Option bytes are static configurations:

- ✓Controlled voltage level
- ✓Boot used
- ✓Watchdog in stand by mode
- ✓Static Protections (Write Protection, PCROP…)

The STM32CubeProgrammer- will allow us to configure our option bytes

Starting the Microcontroller

A microcontroller has 3 options to start:

- Program in the Flash at address 0x0800 0000
- Program in SRAM
- From bootloader

When no program is available in the microcontroller, it must necessarily start with the bootloader which allows it to be programmed

Security requires to write its own bootloader to check security keys for instance

Note: icp (in-circuit programming) is the ability to program flash memory using the debugger protocol (SWD or JTAG for the STM32) or the bootloader

Boot Modes

At boot, the BOOT0 pin and the BOOT1 option bit are used to select one of three boot options:

- ✓ Boot from User Flash
- ✓ Boot from System Memory
- ✓ Boot from Onboard SRAM

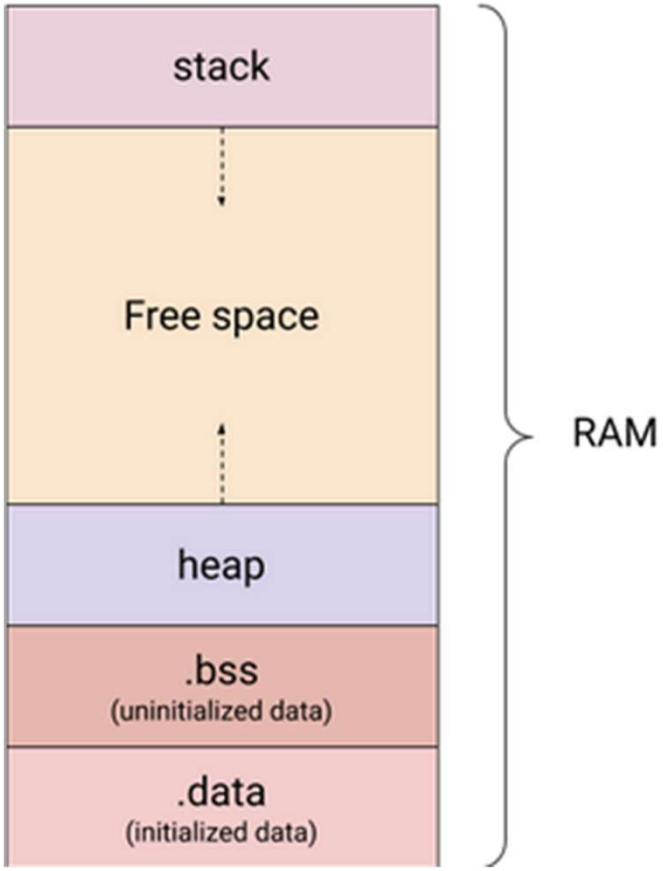
The bootloader is located in system memory. It is used to reprogram the Flash memory using USART, I2C, SPI, CAN or USB OTG FS in Device mode via DFU (Device Firmware Upgrade).

Note: Boot modes are more complex with the STM32L5x family

Organisation Code Organization



SRAM Organization





4: Hardware Security

Compilation Stages: Understanding of linker file

Files in C program

Two types of files are included

- ✓ .c files : these files include the functions and the c programs
- ✓ .h files : called header files (or include files), it includes most of variables and definitions (see #define)
- ✓ Optionally assembly files (.s)

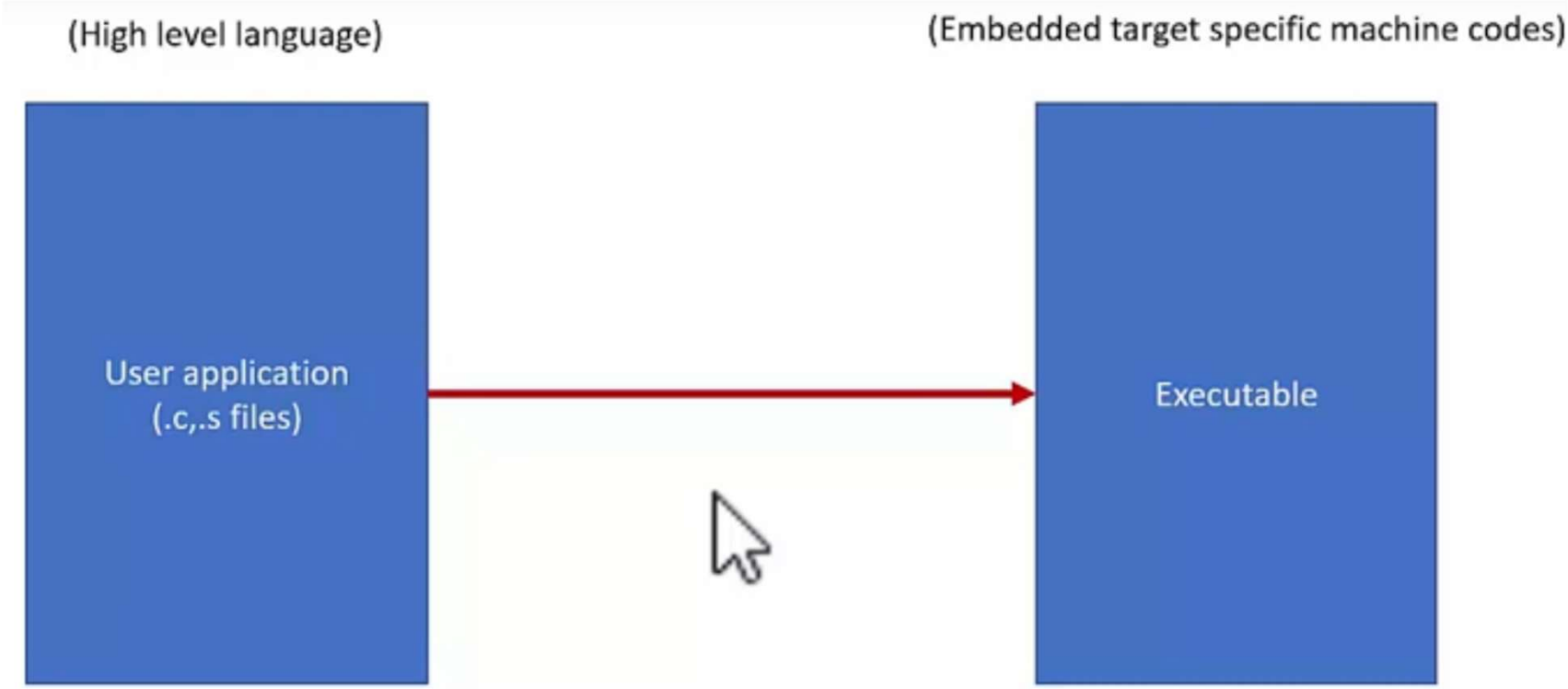
More files are involved in the compilation process!

Compilation

Compilation is processed in various phases to create several files from a text file (the C file) to get the language machine file

- ✓ Preprocessing: the source file is analyzed by a program called preprocessor that performs purely textual transformations (deletion of comments, replacement of the #define…)
- ✓ Compilation / assembly: the file generated by the preprocessor is translated into assembler, then into machine code
- ✓ Link editing: a program is often separated into several source files. The linker allows you to link the different files (i.e. defining memory location)
- ✓ Boot Program: placed at the beginning of the program before the main to initialize the stack pointer and if necessary, the variables

Create Executable



The goal is to create a high level language to an executable (binary file)

Cross Compilation

Cross-compilation toolchains

- ✓ Toolchain or a cross-compilation toolchain is a collection of binaries which allows you to compile, assemble, link your applications.
- ✓ It also contains binaries to debug the application on the target
- ✓ Toolchain also comes with other binaries which help you to analyze the executables
  - dissect different sections of the executable
  - disassemble
  - extract symbol and size information
  - convert executable to other formats such as bin, ihex
  - Provides 'C' standard libraries

ISEN ALL IS DIGITAL! OUEST | yncréa

Cross Compilation and toolchains

Popular Tool-chains:

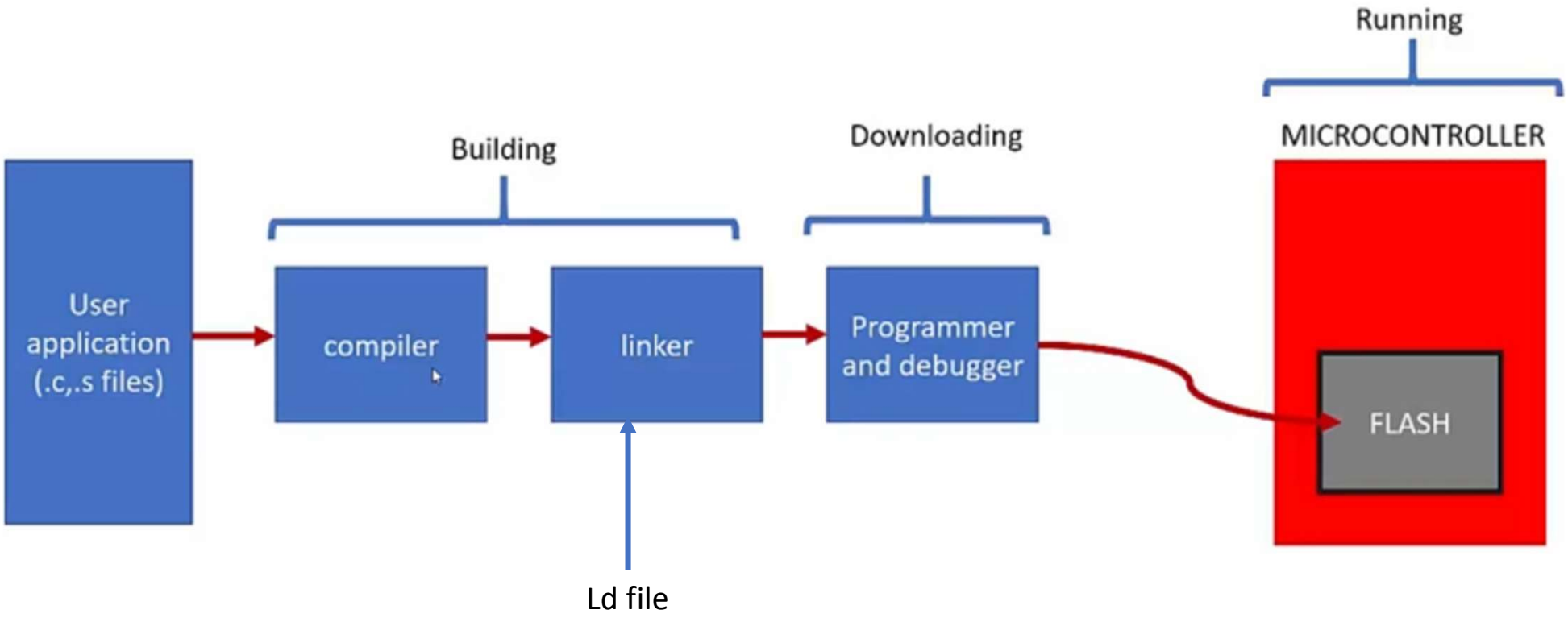
1. GNU Tools(GCC) for ARM Embedded Processors (free and open-source)
2. armcc from ARM Ltd. (ships with KEIL , code restriction version, requires licensing)

Throughout this course, we have been using,
GNU's Compiler Collections(GCC) Toolchain

Downloading GCC Toolchain for ARM embedded processors

- If you have installed STM32CubeIDE, then this toolchain would have already installed as a part of installation
- Official website to download GCC toolchain for ARM
- https://developer.arm.com/tools-and-software/open-source-software/developer-tools/gnu-toolchain/gnu-rm

Summary Steps Compilation



Relocatable Object Files



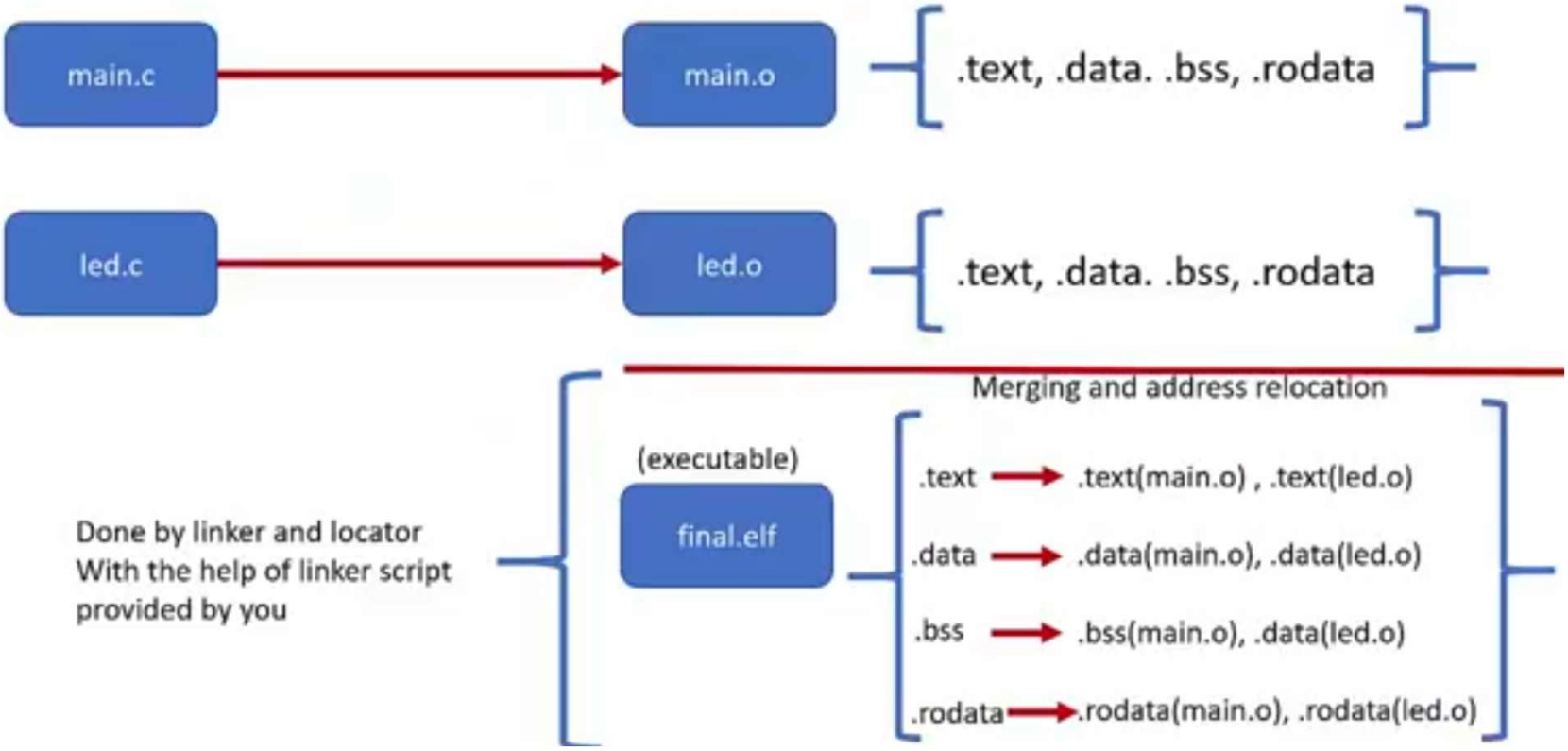
Example of Object File

```
Disassembly of section .text:

00000000 <main>:
   0:   b580        push    {r7, lr}
   2:   af00        add r7, sp, #0
   4:   f7ff fffe   bl  240 <enable_processor_faults>
   8:   4807        ldr r0, [pc, #28]   ; (13c <init_tasks_stack+0x16>)
   a:   f7ff fffe   bl  11c <init_scheduler_stack>
   e:   f7ff fffe   bl  126 <init_tasks_stack>
  12:   f7ff fffe   bl  0 <led_init_all>
  16:   f44f 707a   mov.w   r0, #1000   ; 0x3e8
  1a:   f7ff fffe   bl  b2 <init_systick_timer>
  1e:   f7ff fffe   bl  34c <switch_sp_to_psp>
  22:   f7ff fffe   bl  32 <task1_handler>
  26:   e7fe        b.n 26 <main+0x26>
  28:   2001ec00    .word   0x2001ec00
```

Addresses are not yet allocated in an object file

Combinaison Fichiers Objets



Using the Linker File, different sections of object files are combined

Example of Linker Script

```
/* Memories definition */
MEMORY
{
  RAM    (xrw)    : ORIGIN = 0x20000000,   LENGTH = 96K
  FLASH    (rx)    : ORIGIN = 0x8000000,   LENGTH = 512K
}

/* Sections */
SECTIONS
{
  /* The startup code into "FLASH" Rom type memory */
  .isr_vector :
  {
    . = ALIGN(4);
    KEEP(*(.isr_vector)) /* Startup code */
    . = ALIGN(4);
  } >FLASH

  /* The program code and other data into "FLASH" Rom type memory */
  .text :
  {
    . = ALIGN(4);
    *(.text)           /* .text sections (code) */
    *(.text*)          /* .text* sections (code) */
    *(.glue_7)         /* glue arm to thumb code */
    *(.glue_7t)        /* glue thumb to arm code */
    *(.eh_frame)

    KEEP (*(.init))
    KEEP (*(.fini))
```

Memory Reading/Writing

- ✓ Directly from the memory (internal to the microcontroller)
- ✓ From the bootloader
- ✓ Via the SWD in debug mode
- ✓ From the DMA with a serial communication

- ✓ Requirement to understand the linker script
- ✓ Understand build analyzer



ISEN ALL IS DIGITAL! OUEST | yncréa

5 : Hardware Security

C programming and impact on variable location in memory

Hardware Security

```
#include "STM32L1xx.h"      /* I/O port/register names/addresses for the STM32L1xx microcontrollers */

/* Global variables – accessible by all functions */
int count, bob;             //global (static) variables – placed in RAM

/* Function definitions*/
int function1(char x) {     //parameter x passed to the function, function returns an integer value
  int i,j;                  //local (automatic) variables – allocated to stack or registers
  -- instructions to implement the function
}

/* Main program */
void main(void) {
  unsigned char sw1;        //local (automatic) variable (stack or registers)
  int k;                    //local (automatic) variable (stack or registers)
/* Initialization section */
  -- instructions to initialize  variables, I/O ports, devices, function registers
/* Endless loop */
  while (1) {           //Can also use:  for(;;) {
   -- instructions to be repeated
  } /* repeat forever */
}
```

Declare local variables

Initialize variables/devices

Body of the program

Variable Location

```
/* USER CODE BEGIN PV */
volatile uint32_t myvarOutMain;
```

Memory Monitor: &(myvarOutMain) : 0x20000028

myvarOutMain saved in the data section of the RAM

```
int main(void)
{
  /* USER CODE BEGIN 1 */
  volatile uint32_t myvarInMain = 0xDEADBEEF;
  myvarOutMain = 0x12345678;
```

Memory Monitor: &(myvarInMain) : 0x20013FF4

myvarInMain saved in the stack

The location to store a variable depends on the location it is declared in the program how it is declared and the optimization to compile the program

Variable in Internal Registers

```
5  uint32_t incrementV1(uint32_t var1) {
6      uint32_t var2;
7      var2 = var1 + 1;
8      return var2;
9  }
```

- Arguments are passed through internal registers:
  - Up to 4 arguments in function parameters are passed through registers (R0, R1, R2 and R3). If more than 4 arguments, extra arguments are passed thanks to the stack
  - Return arguments are passed through registers R0 and R1

Requirement to be aware of variables size

- ✓ Langage C does not define sizes of all variables and depend on platforms
- ✓ Issue with pointers: weird behaviors and bugs are triggered when sizes of variables is not fixed

Size of Int

In many languages, it's common to declare an integer as an int

```
int entier = 5;
int tailleEntier;

main()
{
  tailleEntier = sizeof(entier);
  while (1);

}
```

Variable	Value	Type	Address
tailleEntier	2	int	0x0

tailleEntier	int	4

This small program allows to check the actual size of an integer

Size of an integer is equal to 2 bytes on the STM8

Size of an integer is equal to 4 bytes on the STM32

Size of Data Types in C

✓ On the STM8

Variable	Size in Bytes
char	1
int	2
Long	4

Variable	Size in Bytes
char	1
Short int	2
int	4
long	8

Type	C-Standard Type Size	Value Range (**min**)
signed char	At least 8-bits	$[-2^7, +2^7 - 1]$
unsigned char	At least 8-bits	$[0, +2^8 - 1]$
signed short int	At least 16-bits	$[-2^{15}, +2^{15} - 1]$
unsigned short int	At least 16-bits	$[0, +2^{16} - 1]$
signed long int	At least 32-bits	$[-2^{15}, +2^{15} - 1]$
unsigned long int	At least 32-bits	$[0, +2^{32} - 1]$

✓ Sizes of C data types are ambiguous and vary among architectures.

✓ C-standard specifies a minimum each variable can be

Problem with size of Int

- ✓ The size of int depends on platform
  - 2 bytes on the STM8
  - 4 bytes on a Cortex-M 32-bit microcontroller
- ✓ Portability of programs gets impossible (purpose of C-program is to give this flexibility vs assembly)
- ✓ Any microcontroller has many 8-bit operations (like reading a register in an STM8)

Do not Declare any variable as int in embedded C programming

Solving Int issue

- ✓ Create new types that impose the size of integer
  - int8_t : signed integer of exactly 8 bits
  - int16_t : signed integer of exactly 16 bits
  - int32_t : signed integer of exactly 32 bits

  - uint8_t : unsigned integer of exactly 8 bits
  - uint16_t : unsigned integer of exactly 16 bits
  - uint32_t : unsigned integer of exactly 32 bits

- ✓ These types must be defined in a header file with the intrinsic types of the platform

How to Use Pointer

GPIOA address is : 0x4002 0000
Address offset of ODR: 0x14

// Way to write 0x0000020 at the address 0x4002 0000
*((unsigned int *)0x40020014) = 0x00000020;

(unsigned int *)0x40020014 means 0x40020014 is cast a pointer to a 32-bit value
*((unsigned int *)0x40020014) means we point to the address 0x40020014
*((unsigned int *)0x40020014) = means we assign a value to the address 0x40020014

Pointer: * Symbol

```
/* USER CODE BEGIN 1 */
  uint32_t myvar = 0xDEADBEEF;
  uint32_t * pointer_myvar;
  pointer_myvar = &myvar;
  * pointer_myvar = 0x12345678;
```

- ✓ myvar is declared as an unsigned 32-bit integer (4 bytes on stm32)
- ✓ pointer_myvar is declared as a pointer pointing to an uint32_t
- ✓ point_myvar is loaded with the address of myvar
- ✓ This location can be pointed to change the value at this specific location

Struct

- ✓ Structure is a data type that allows to combine items of the same or different kinds
- ✓ The struct tag is optional, and each member is a normal variable definition.
- ✓ At the end of the structure definition, before the final semicolon, you can specify one or more structure variables, but it is optional.

```
struct [structure tag] {

   member definition;
   member definition;
   ...
   member definition;
} [one or more structure variables];
```

Struct for GPIO

- ✓ All the ports have a common structure with the same 11 registers.
- ✓ It makes sense to combine these 11 registers in one structure and apply this structure to all the ports

```
/* USER CODE BEGIN 0 */
typedef struct
{
  uint32_t MODER;        /*!< GPIO port mode register,                  Address offset: 0x00      */
  uint32_t OTYPER;       /*!< GPIO port output type register,           Address offset: 0x04      */
  uint32_t OSPEEDR;      /*!< GPIO port output speed register,          Address offset: 0x08      */
  uint32_t PUPDR;        /*!< GPIO port pull-up/pull-down register,     Address offset: 0x0C      */
  uint32_t IDR;          /*!< GPIO port input data register,            Address offset: 0x10      */
  uint32_t ODR;          /*!< GPIO port output data register,           Address offset: 0x14      */
  uint32_t BSRR;         /*!< GPIO port bit set/reset registerBSRR,     Address offset: 0x18      */
  uint32_t LCKR;         /*!< GPIO port configuration lock register,    Address offset: 0x1C      */
  uint32_t AFR[2];       /*!< GPIO alternate function register,         Address offset: 0x20-0x24 */
  uint32_t BRR;          /*!< GPIO bit reset register,                  Address offset: 0x28      */
} GPIO_struct;
```

This structure matches the hardware of a GPIO on the STM32L152

Access to GPIO W/ struct

- ✓ GPIO_portB is declared as a pointer to a structure GPIO_struct
- ✓ In the STM32L152RE8, the address of port B is at address 0x40020400
- ✓ Pointing to the GPIO_portB, it is possible to select MODER element and changing the value of this register

```
GPIO_struct * GPIO_portB;
GPIO_portB = (GPIO_struct *) 0x40020400;
(* GPIO_portB).MODER = 0xFFFFFFFF;
```

ISEN ALL IS DIGITAL! OUEST | yncréa

Improvment (1/2)

The annotaton (*typeStructure).structureElement is tedious to write and usually replaced by typeStructure→structureElement

```
GPIO_struct * GPIO_portB;
GPIO_portB = (GPIO_struct *) 0x40020400;
(* GPIO_portB).MODER = 0xFFFFFFFF;
```

```
GPIO_struct * GPIO_portB;
GPIO_portB = (GPIO_struct *) 0x40020400;
GPIO_portB->MODER = 0xFFFFFFFF;
```

Improvment (2/2)

The GPIO_portB can be initialized in one line

```
GPIO_struct * GPIO_portB;
GPIO_portB = (GPIO_struct *) 0x40020400;
(* GPIO_portB).MODER = 0xFFFFFFFF;
```

```
GPIO_struct * GPIO_portB = (GPIO_struct *) 0x40020400;
GPIO_portB->MODER = 0xFFFFFFFF;
```

ISEN ALL IS DIGITAL! OUEST | yncréa

C-Program Compiled

```
/* USER CODE BEGIN 0 */
typedef struct
{
  uint32_t MODER;
  uint32_t OTYPER;
  uint32_t OSPEEDR;
  uint32_t PUPDR;
  uint32_t IDR;
  uint32_t ODR;
  uint32_t BSRR;
  uint32_t LCKR;
  uint32_t AFR[2];
  uint32_t BRR;
} GPIO_struct;
```

```
GPIO_struct * GPIO_portB = (GPIO_struct *) 0x40020400;
GPIO_portB->MODER = 0xFFFFFFFF;
```

```
118
08000192: 0x0000054b   GPIO_struct * GPIO_portB = (GPIO_struct *) 0x40020400;
08000194: 0x00007b60   ldr     r3, [pc, #20]   ; (0x80001a8 <main+44>)
119                    str     r3, [r7, #4]
08000196: 0x00007b68     GPIO_portB->MODER = 0xFFFFFFFF;
08000198: 0x4ff0ff32   ldr     r3, [r7, #4]
0800019c: 0x00001a60   mov.w   r2, #4294967295
                       str     r2, [r3, #0]
```

✓ The compilation of the code shows the usage of ldr, str and mov instructions

How to Write Shift

Preferred version

```
// initilisation PC2
PC_DDR |= 1<<2; // pin2 in output mode
PC_CR1 |= 1<<2; // pin2 in push-pull
```

Instead of

```
PC_DDR |= 0x04;
PC_CR1 |= 0x02
```

Masking in C (1/2)

Masking bits to 1 :

```
PA_ODR=PA_ODR | 0b00011000; // set bits 3 and 4 of PA_ODR
```

or more readable

```
PA_ODR |= 1<<4| 1<<3;
```

Masking bits to 0 :

```
PA_ODR=PA_ODR & 0b11100111;// reset bits 3 and 4 of PA_ODR
```

Or more readable

```
PA_ODR &= ~(1<<4 | 1<<3)
```

```
#define SetBit(var,place)      (var|=(1<<place))
#define ClrBit(var,place)      ….
```

Masking in C (2/2)

Querying Status bit:

```
var = (PA_IDR & (1<<12)) >> 12; // Query value of bit 12 of value called PA_IDR
```

Toggling a bit :

```
PA_ODR=PA_ODR ^ 0b00011000; // toggle bits 3 and 4 of PA_ODR
```

or more readable

```
PA_ODR ^= (1<<4| 1<<3);
```

Start up File

- The startup file is responsible for setting up the right environment for the main user code to run
- Code written in startup file runs before main(). So, you can say startup file calls main()
- Some part of the startup code file is the target (Processor) dependent
- Startup code takes care of vector table placement in code memory as required by the ARM cortex Mx processor
- Startup code may also take care of stack reinitialization
- Startup code is responsible of .data, .bss section initialization in main memory

Example Start up File

```
/* Call the clock system intitialization function.*/
  bl  SystemInit
/* Call static constructors */
    bl __libc_init_array
/* Call the application's entry point.*/
  bl  main
  bx  lr
.size  Reset_Handler, .-Reset_Handler

/**
 * @brief  This is the code that gets called when the processor receives an
 *         unexpected interrupt.  This simply enters an infinite loop, preserving
 *         the system state for examination by a debugger.
 * @param  None
 * @retval None
*/
    .section  .text.Default_Handler,"ax",%progbits
Default_Handler:
Infinite_Loop:
  b  Infinite_Loop
  .size  Default_Handler, .-Default_Handler
/******************************************************************************
*
* The minimal vector table for a Cortex M3. Note that the proper constructs
* must be placed on this to ensure that it ends up at physical address
* 0x0000.0000.
*
******************************************************************************/
   .section  .isr_vector,"a",%progbits
  .type  g_pfnVectors, %object
  .size  g_pfnVectors, .-g_pfnVectors

g_pfnVectors:
  .word  _estack
  .word  Reset_Handler
  .word  NMI_Handler
  .word  HardFault_Handler
```

- ✓ A startup file can be written in C or assembler
- ✓ This file initializes the interrupt vector table
- ✓ This file calls main.c
- ✓ Initializes the SRAM partition (linked to the link file)



6: Hardware Security

Microcontroller protections: Write Protection, Read Protection (PCROP, RDP), Firewall, TrustZone….

Memory Protection Unit

The Memory Protection Unit (MPU) is used to manage processor access to memory to prevent one task from accidentally corrupting memory or resources used by any other active task.

The MPU is particularly useful for applications where critical or certified code must be protected from misbehavior by other tasks. It is typically managed by an RTOS (real-time operating system).

This protection has been available for many years and this class is focusing on more updated features that are required even without OS.

RDP (Readout Protection)

Read-protect (RDP) to protect all memory. Three levels are available :

- ✓ Level 0 : no read protection
- ✓ Level 1 : Memory read protection: Flash memory cannot be read or written if either debugging functionality is connected, booting into RAM, or the bootloader is chosen
- ✓ Level 2 : Chip read protection: debugging features (Cortex-M4 JTAG and serial wire), RAM boot and bootloader selection are disabled (JTAG fuse). This selection is irreversible.

- ✓ **NEVER USE LEVEL 2 DURING OUR PRACTICAL WORK!!! WE CANNOT RECOVER NORMAL USAGE OF THE MICROCONTROLLER**

Access State / RDP and Execution Modes

Area	Protection level	User execution			Debug, boot from RAM or boot from system memory (loader)		
		Read	Write	Erase	Read	Write	Erase
Main memory	1	Yes	Yes	Yes	No	No	No
	2	Yes	Yes	Yes	N/A	N/A	N/A
System memory	1	Yes	No	No	Yes	No	No
	2	Yes	No	No	N/A	N/A	N/A
Option bytes	1	Yes	Yes	Yes	Yes	Yes	Yes
	2	Yes	No	No	N/A	N/A	N/A
Backup registers	1	Yes	Yes	N/A(1)	No	No	N/A(1)
	2	Yes	Yes	N/A	N/A	N/A	N/A
SRAM2	1	Yes	Yes	Yes(1)	No	No	No(1)
	2	Yes	Yes	Yes	N/A	N/A	N/A

1. Erased when RDP change from Level 1 to Level 0.

WRP (Write Protection)

The protected area is protected against erasure and programming. Two areas per bank can be selected, with a granularity of 2 KB

PCROP (Proprietary Code Readout Protection)

Part of the flash memory may be protected against reading and writing by third parties.

- ✓ The protected area is execute-only: it can only be accessed by the STM32 CPU, as instruction code, while all other accesses (DMA, debugging and reading, writing and erasing CPU data) are strictly prohibited.

- ✓ One zone per bank can be selected, with a granularity of 64 bits. An additional option bit (PCROP_RDP) allows you to select whether or not the PCROP zone is cleared when RDP protection goes from level 1 to level 0.

PCROP Deletion

- ✓ PCROP can only be removed when 2 conditions are covered
  - PCROP_RDP option bit is set set to 1
  - Perform RDP1 eto RDP0 regression

⇒ In that case a mass erase is performed and all PCROP regions removed

✓ If PCROP_RDP option is set to 0, the regression RDP1 to RDP0 will perform a mass erase without altering the PCROP regions setting and content

PCROP Usage

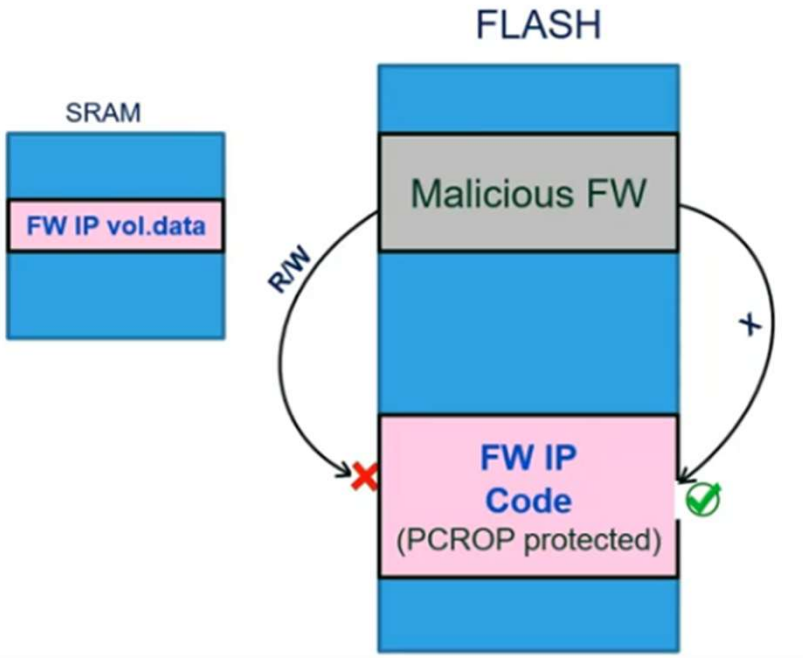
- ✓ A large ecosystem was built around the Cortex M that is de facto a standard on the market
- ✓ Companies make business by selling licences and libraries
- ✓ Libraries must be able to executed, but the user must be able to read the library to avoid reverse engineering.

PCROP software constraint

- ✓ The code in PCROP has to be compiled to remove all data access
  - GCC option : No data reads in code memory
  - Keil option : Execure Code Only
  - IAR option : No data reads in code memory

- ✓ Code running in PCROP must be put in a specific area in the scatter file (.ld file) to place this read-only code in the good area of the flash

PCROP Principle

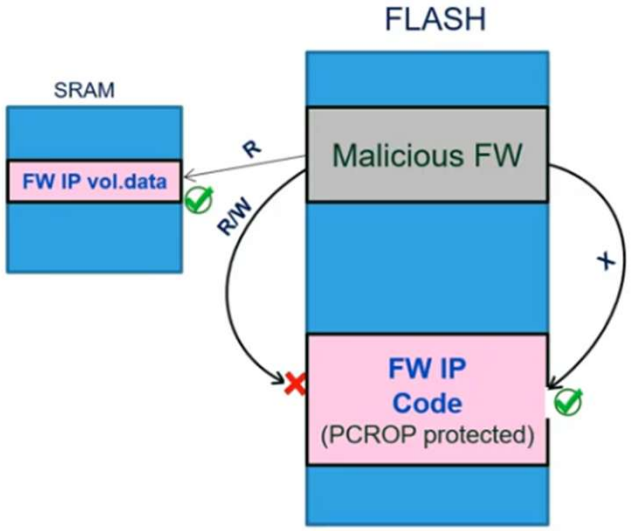


A code can launch the FW IP but cannot read or write the code

PCROP Benefits

Proprietary Code ReadOut Protection **PCROP**

- Benefits
  - FW IP protection
  - Mutual protection of FW IPs
  - Interesting feature in RDP level0
  - Code integrity
- Weakness
  - Volatile data (SRAM) not protected
  - Specific compilation required



PCROP can be used in level 0. Hence it can be supplied to a third party to develop its application, the FW is completely hidden

PCROP TIPS Application

Tips: Possibility to hide some data in PCROP region
Use assembly code which could generate a value without data access. This could be used to protect cryptographic key values

```
MOVS   R1, #0x5f
LSLS   R1, R1, #24
MOVS   R5, #0x4d
LSLS   R5, R5, #16
ADD    R1, R1, R5
MOVS   R5, #0x45
LSLS   R5, R5, #8
ADD    R1, R1, R5
MOVS   R5, #0x4f
ADD    R1, R1, R5
```



Executing this code, you have 0x5f4d454f in R1 without any data access.

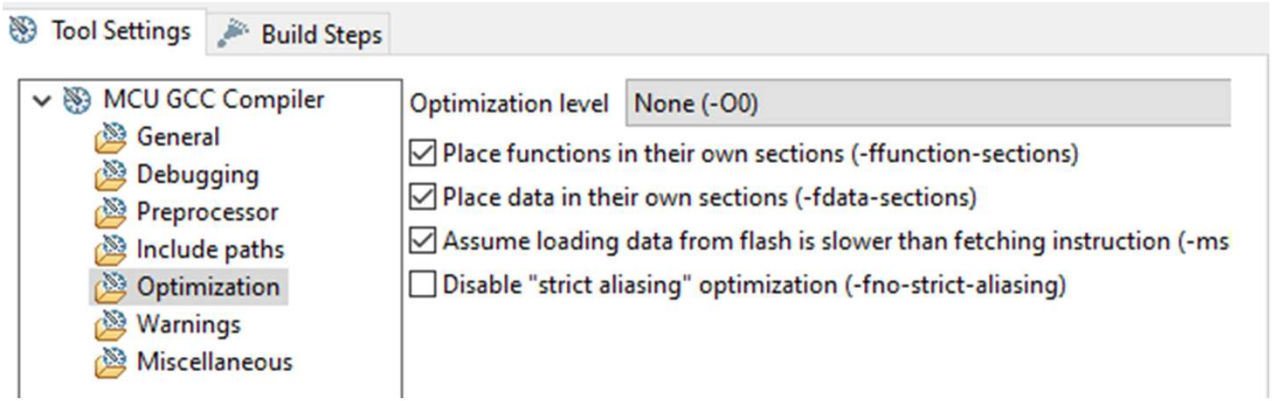
You have the data only in the registers! No data in SRAM!

PCROP Optimisation requirement

`-mslow-flash-data`

Assume loading data from flash is slower than fetching instruction. Therefore literal load is minimized for better performance. '

This option is only supported when compiling for ARMv7 M-profile and off by default. It conflicts with `-mword-relocations`.

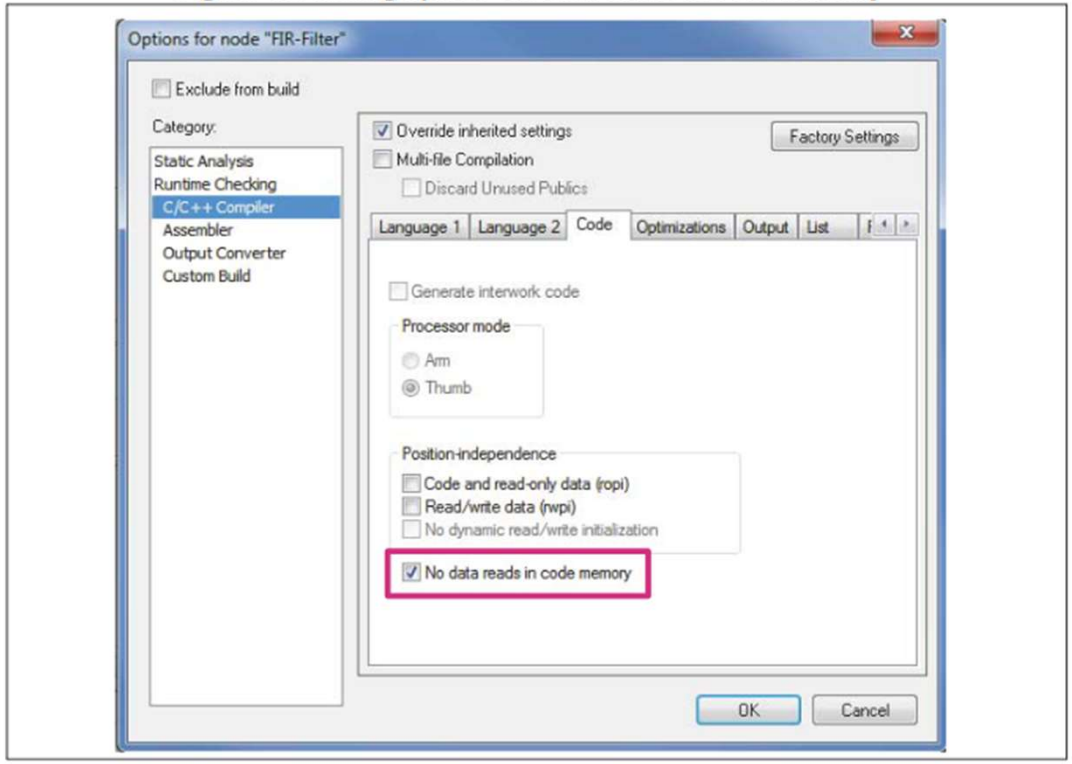


Generating an execute-only IP code

Each tool chain has its own options to prevent compiler from generating literal pools and branch tables. For instance, Keil® has the Execute-only code option while IAR™ has the No data reads in code memory option. For STM32CubeIDE the option is called “Assume loading data from flash is slower than fetching instruction”.

PCROP Optimisation requirement

Figure 13. Setting option "no data reads in code memory"



SW4STM32: No data reads in code memory

The same option "No data reads in code memory" must be used with SW4STM32 to prevent the compiler from generating any data accesses to code sections. To activate this option, right click on the file group "FIR-Filter" containing IP-code source files then (see *Figure 14*) select "Properties".

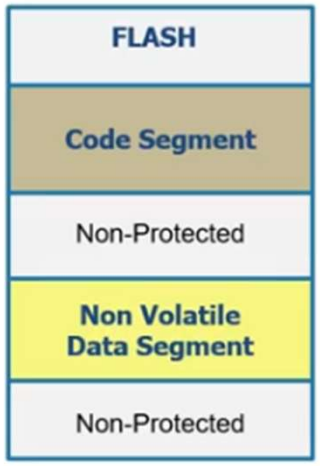
Secure User Memory

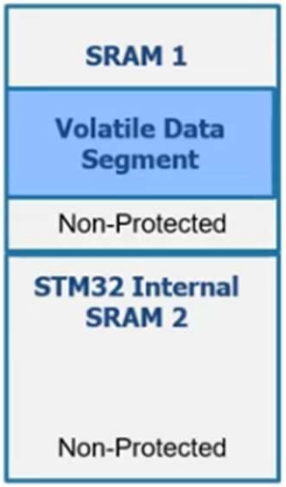
- Purpose : code, data and execution isolation of secure user firmware
- Principle : a user flash region executed at boot time, whatever the boot mode configuration. Once executed, the secure user part is closed and cannot be accessed anymore by any mean until the next boot.
- Configured thanks option bytes
- Supported on
  - STM32G0/G4
  - STM32H7
  - STM32L5 (mechanism called HDP)

Firewall

- Firewall : it's is an hardware IP, which allow to create a security enclave (Flash/RAM) with a unique entry point
- This protection is dynamically managed at runtime.

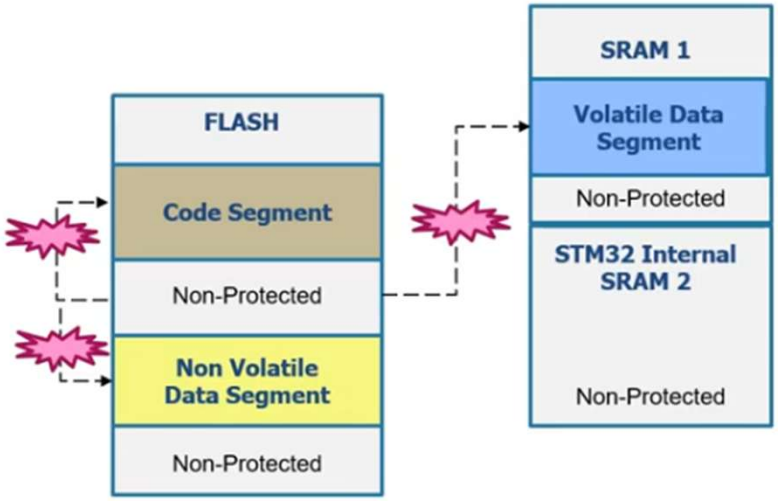
Firewall





- The Firewall monitors accesses to trusted areas, referred to as segments:
  - **Code Segment (FLASH)**
    - **Instruction fetches** or **data read accesses** can only occur when Firewall is open
  - **Non Volatile Data Segment (FLASH)**
    - Usually contains **sensitive constants** (e.g. cryptographic keys)
  - **Volatile Data Segment (SRAM 1)**
    - Contains potentially variable data used by the protected code.
    - **Data can only be accessed during the protected execution state**

Firewall

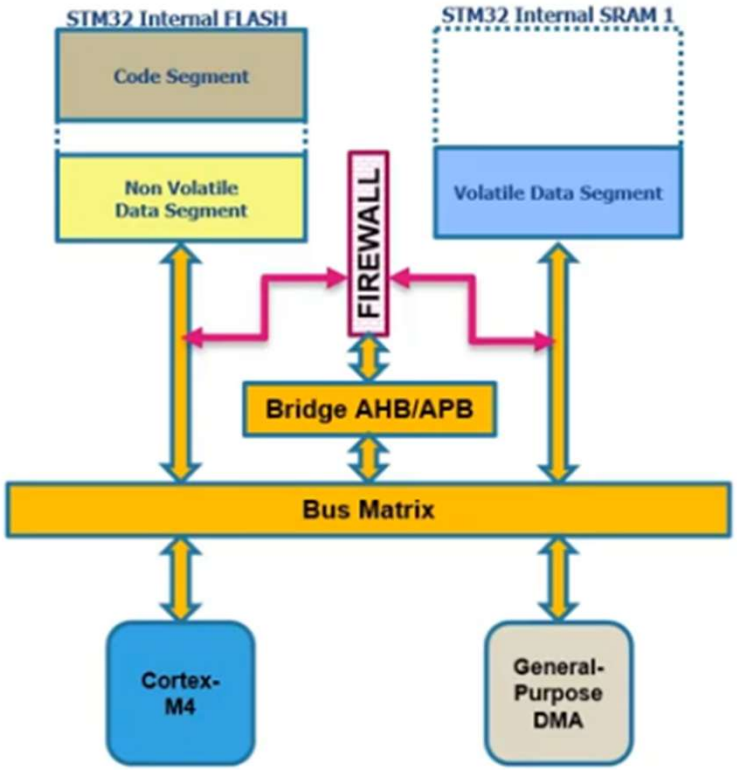


- The Firewall monitors accesses to trusted areas, referred to as segments:
  - **Code Segment (FLASH)**
    - **Instruction fetches** or **data read accesses** can only occur when Firewall is open
  - **Non Volatile Data Segment (FLASH)**
    - Usually contains **sensitive constants** (e.g. cryptographic keys)
  - **Volatile Data Segment (SRAM 1)**
    - Contains potentially variable data used by the protected code.
    - **Data can only be accessed during the protected execution state**



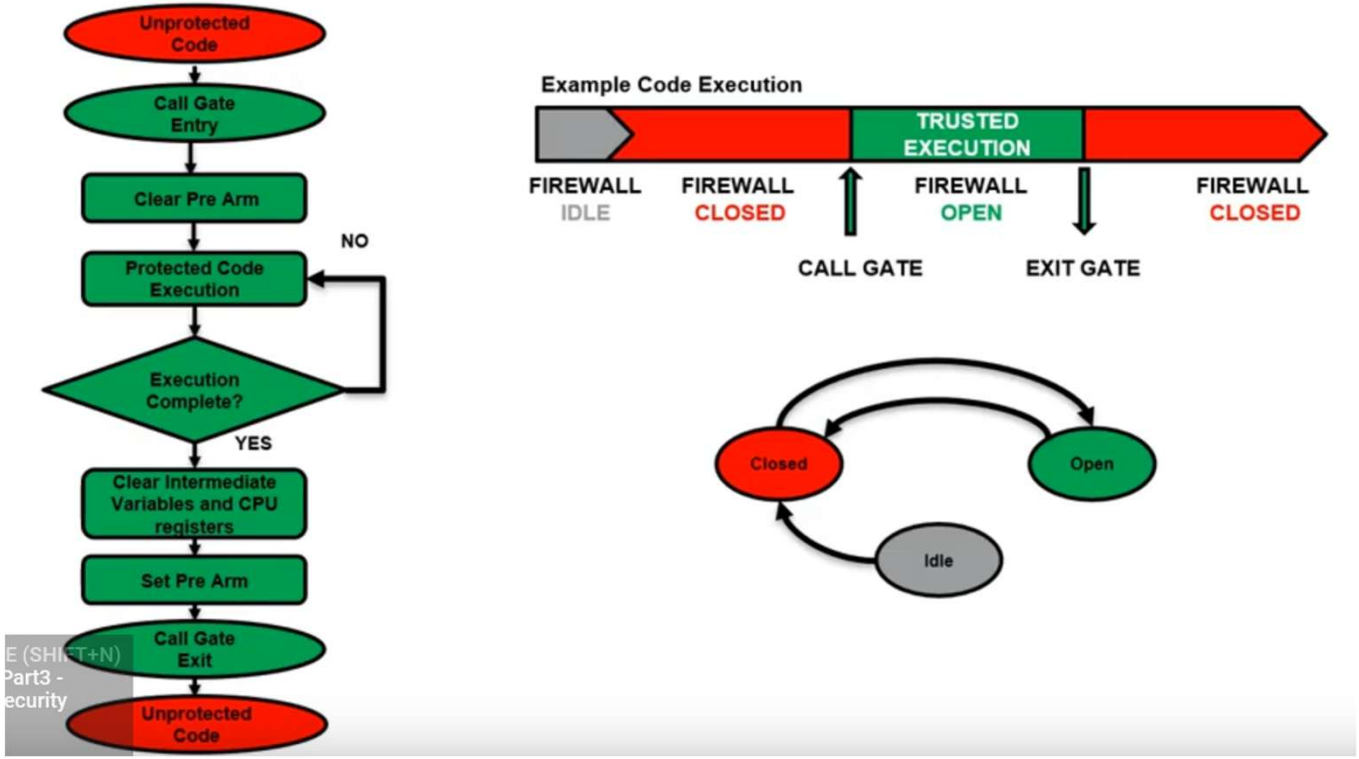
Illegal accesses based on the Segment properties and current Firewall State will cause a **RESET**

Firewall Architecture



- The Firewall is an hardware peripheral monitoring the connected memories (FLASH and SRAM 1)

Firewall Architecture



Firewall PreArm

- FPA: Firewall prearm

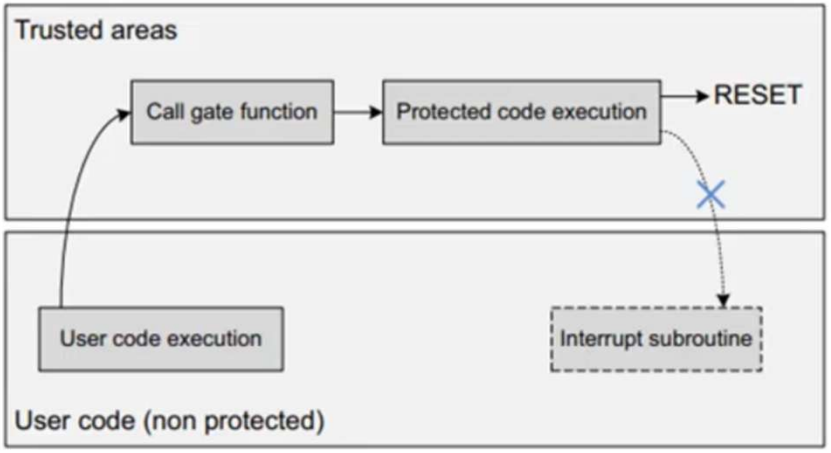
  0 : Any code executed outside the protected segment when the Firewall is opened will generate a system reset.

  1 : any code executed outside the protected segment will close the Firewall.

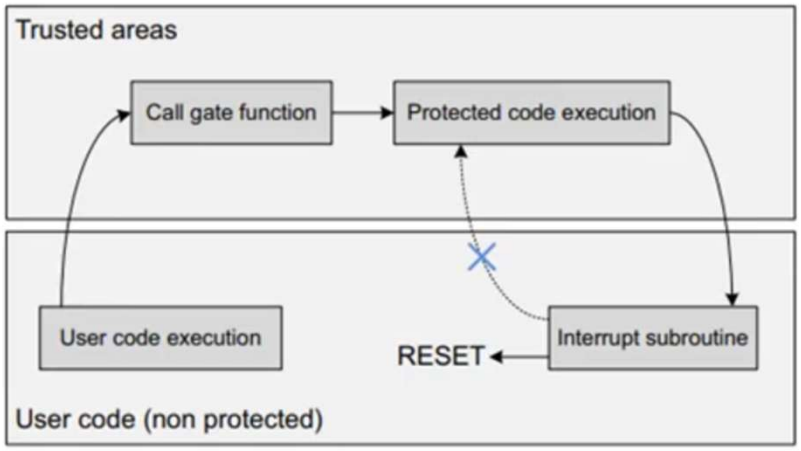
Firewall And Interrupt

- When the FIREWALL is OPEN, no interrupt must take place during the execution of the protected code.

FIREWALL is OPEN with FPA = 0



FIREWALL is OPEN with FPA = 1



ISEN ALL IS DIGITAL! OUEST | yncréa

Firewall TIPS

- The code protected by the Firewall must not be interruptible. It is up to the user code to disable any interrupt source before executing the code protected by the Firewall.

```
SecureFlash (rx) : ORIGIN = 0x080fc004, LENG
```

- To open the Firewall, the code currently executed must jump to the 2nd word of the “call gate” and execute the code from this point.

ISEN ALL IS DIGITAL! OUEST | yncréa

Firewall TIPS

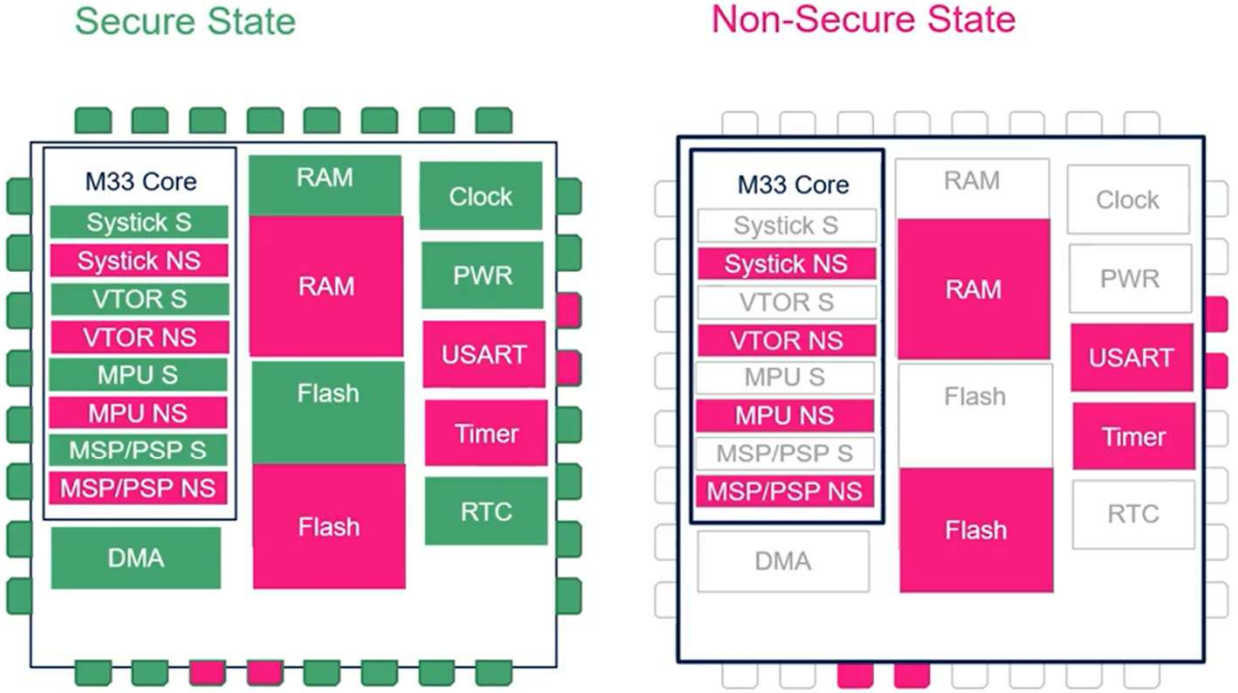
```
MEMORY
{
  RAM    (xrw)    : ORIGIN = 0x20000000,   LENGTH = 96K
  RAM2   (xrw)    : ORIGIN = 0x10000000,   LENGTH = 32K
  FLASH  (rx)     : ORIGIN = 0x8000000,    LENGTH = 1024K
  SecureFlash (rx) : ORIGIN = 0x080fc004, LENG
}
```

0x080FC004 because the call gate is at 0x080FC004

Security Part6 - STM32 security ecosystem - 07 TrustZone presentation

Cortex M33 and TrustZone Key features

- New processor state
- Real time
  - Low interrupt latency
  - Low state switching overhead
  - Deterministic

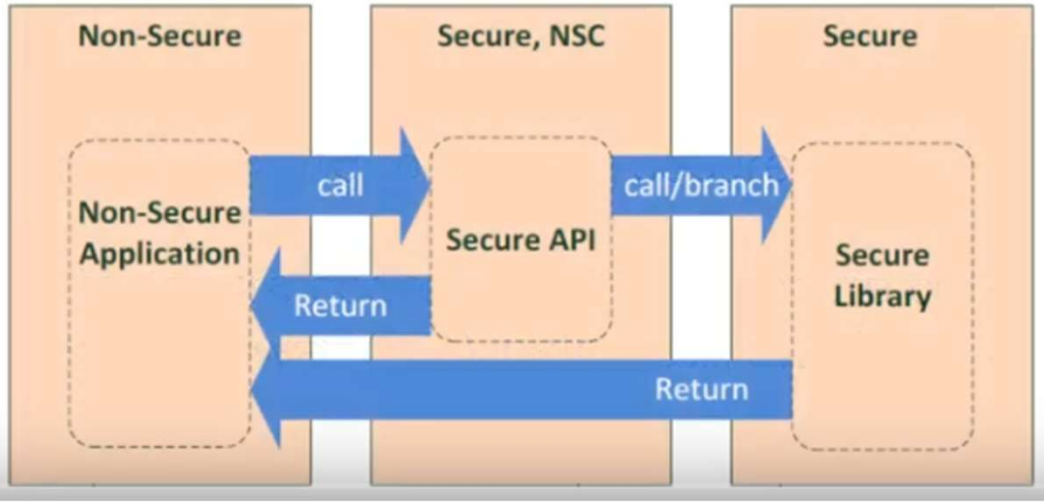


07-6

6:21 / 10:15 • Cortex M33 ›

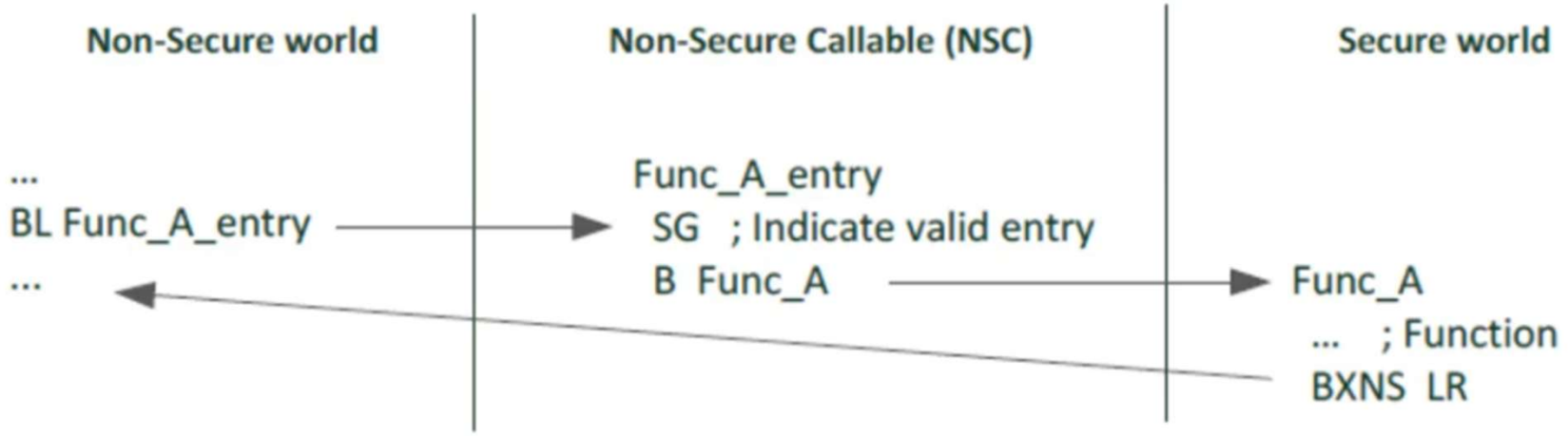
TrustZone

- 3 different attribute in SAU (security attribution unit) and IDAU (implementation defined attribution unit)
  - Secure
  - Non-Secure Callable (NSC)
  - Non-Secure



TrustZone

Non-secure program calling a secure function

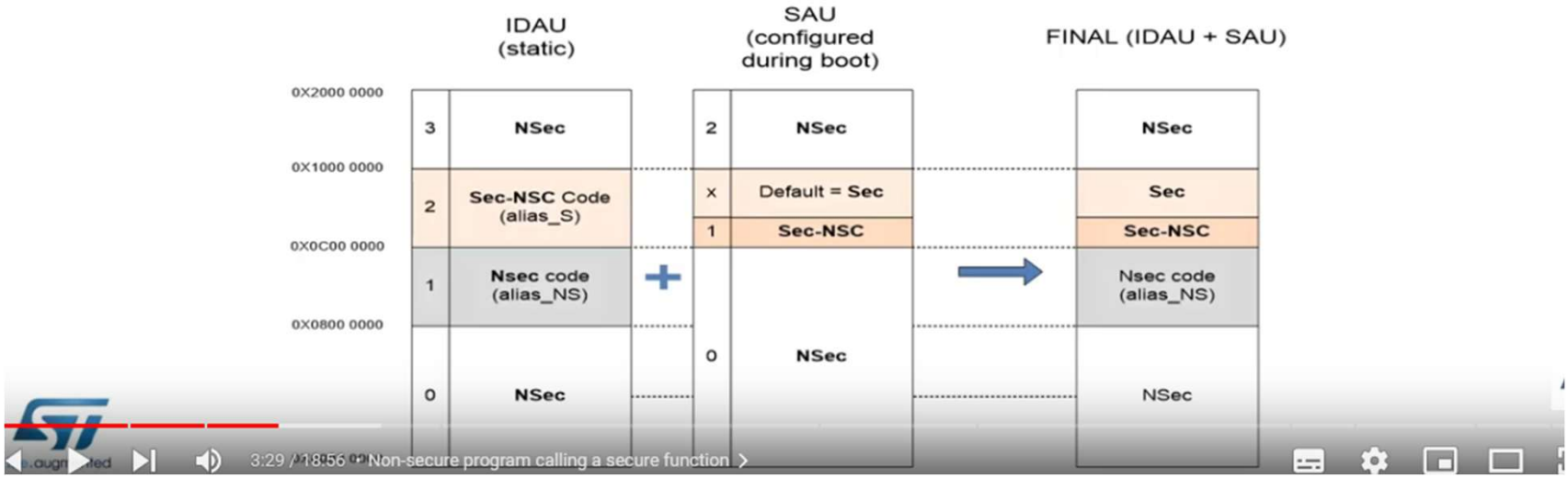


SG : Secure Gate
BXNS: Branch to non secure world

TrustZone

Non-secure program calling a secure function

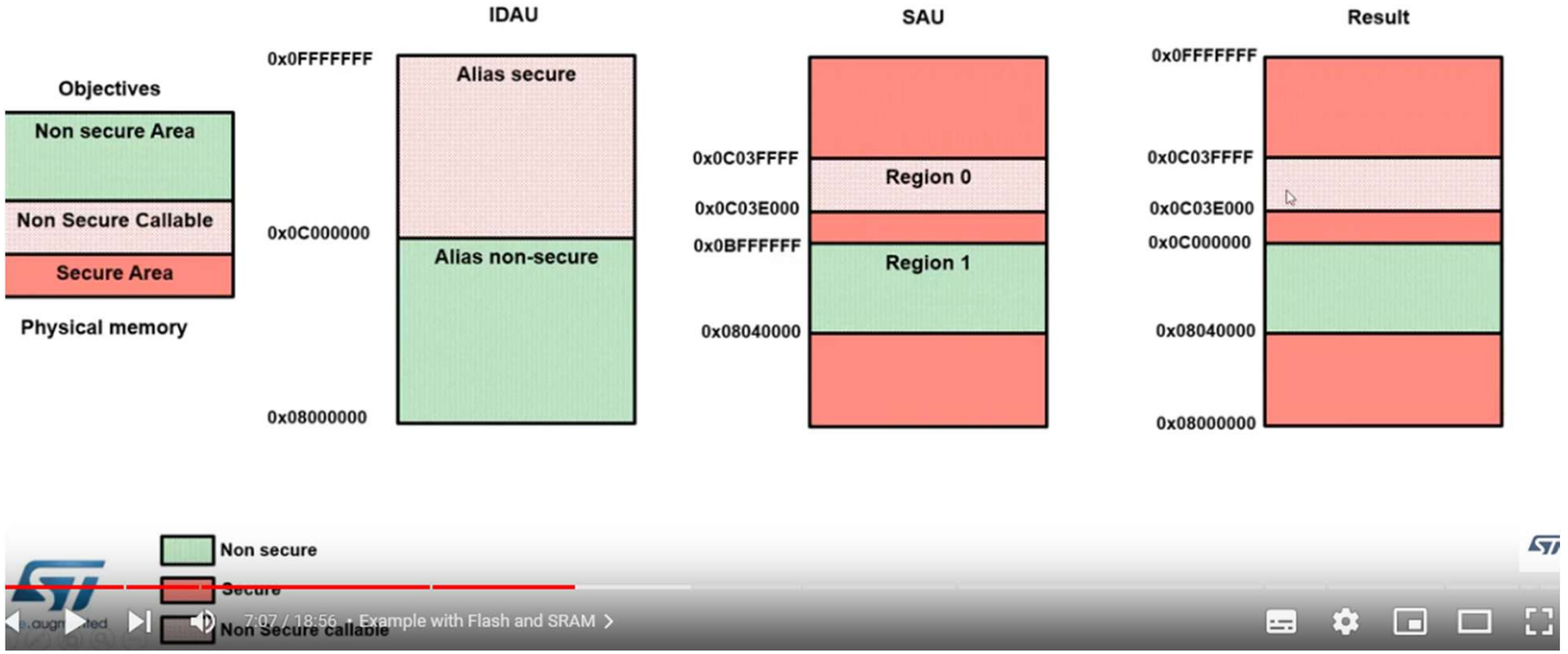
- IDAU define Secure-NSC, Non-Secure and exempted regions / granularity = 64 MB
- SAU = 8 regions, used to overwrite IDAU in order to set secure areas and confirm Non-sec ones
- As showed below, the code area as defined in the memory map (final) requires 3 SAU regions



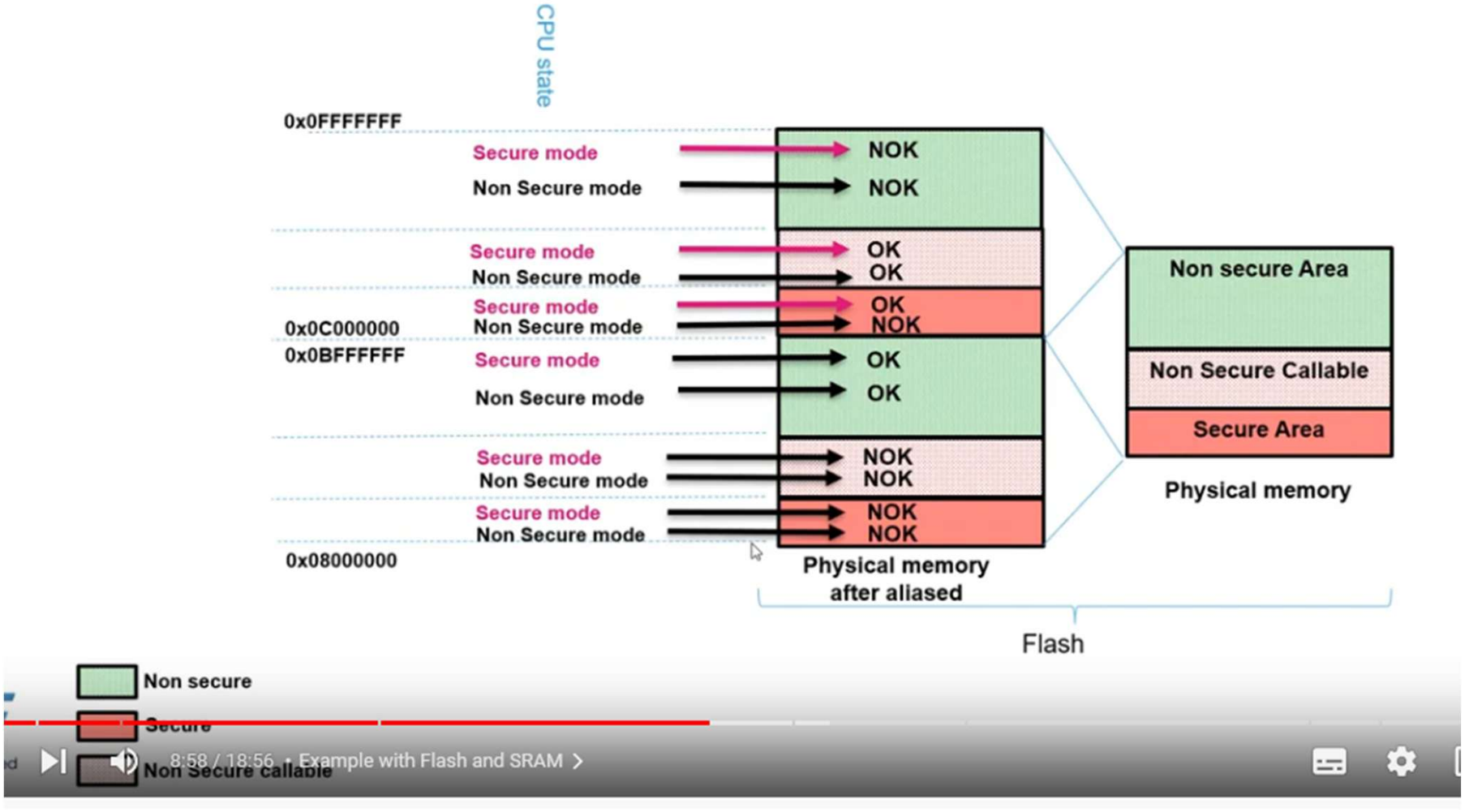
TrustZone

Example with Flash and SRAM

12



TrustZone


CPU state
0x0FFFFFFF
Secure mode
Non Secure mode
NOK
NOK
Secure mode
Non Secure mode
OK
OK
Secure mode
Non Secure mode
OK
NOK
0x0C000000
0x0BFFFFFF
Secure mode
Non Secure mode
OK
OK
Secure mode
Non Secure mode
NOK
NOK
Secure mode
Non Secure mode
NOK
NOK
0x08000000
Physical memory after aliased
Flash
Non secure Area
Non Secure Callable
Secure Area
Physical memory
Non secure
Secure
Non Secure callable
8:58 / 18:56 • Example with Flash and SRAM >

TrustZone Activation

- One option byte to activate : TZEN
- SAU configuration to be done at boot time
- Additional flash security features are available:
  - Secure watermark-based user options bytes defining secure areas and HDP areas.
  - Secure or non-secure block-based areas can be configured on-the-fly after reset
  - An additional RDP protection: RDP level 0.5.

TrustZone Security

- The debug access to secure area is prohibited.
- Debug access to non- secure area are possible

